A. Prepare Multiple Nodes on Alibaba Cloud

A.1. Purchase Single Node on Alibaba Cloud

Step 1: Click https://homenew.console.aliyun.com/home/dashboard/ProductAndService, choose the ECS service,

Step 2: Create a new instance, choose pay as you go, choose ecs.c7.2xlarge, choose Ubuntu 16.04 64 bits, choose ESSD, choose no IPv4 address, choose your password Y0urpassword,

Step 3: Double-check the order,

Step 4: Pay the order,

Step 5: Access the ECS instance via Alibaba Workbench, where the default username is root,

So far, the ECS instance cannot access the Internet because it has no IP address.

A.2. Purchase a NAT Gateway

Step 1: Click https://vpc.console.aliyun.com/nat/cn-guangzhou/nats, purchase 2 public IP addresses, create a NAT gateway,

Step 2: After purchasing 2 public IP addresses, we have 8.163.56.8 and 8.138.164.210,

A.3. Purchase Multiple Nodes on Alibaba Cloud

Step 1: Click https://vpc.console.aliyun.com/vpc/cn-guangzhou/vpcs, purchase another 10 ECS nodes,

choose pay as you go, choose ecs.c7.2xlarge, choose Ubuntu 16.04 64 bits, choose ESSD, choose no IPv4 address, choose your password Y0urpassword,

A.4. Configure the NAT Gateway

Step 1: Click https://vpc.console.aliyun.com/nat/cn-guangzhou/nats, configure the NAT gateway, where the IP address 8.163.56.8 for DNAT and 8.138.164.210 for SNAT (DNAT for Internet users and SNAT for VM inside). Currently, we have 1 NAT, 2 Public IP addresses, and 11 ECS, our target is as follows:

For NAT (Public IP):

SNAT	DNAT
8.138.164.210	8.163.56.8

For ECS maps (Inside IP):

SNAT	DNAT
-	172.31.210.67
	172.31.210.70
	172.31.210.71
	172.31.210.72
	172.31.210.73
	172.31.210.74
	172.31.210.75
	172.31.210.76
	172.31.210.77
	172.31.210.78
	172.31.210.79

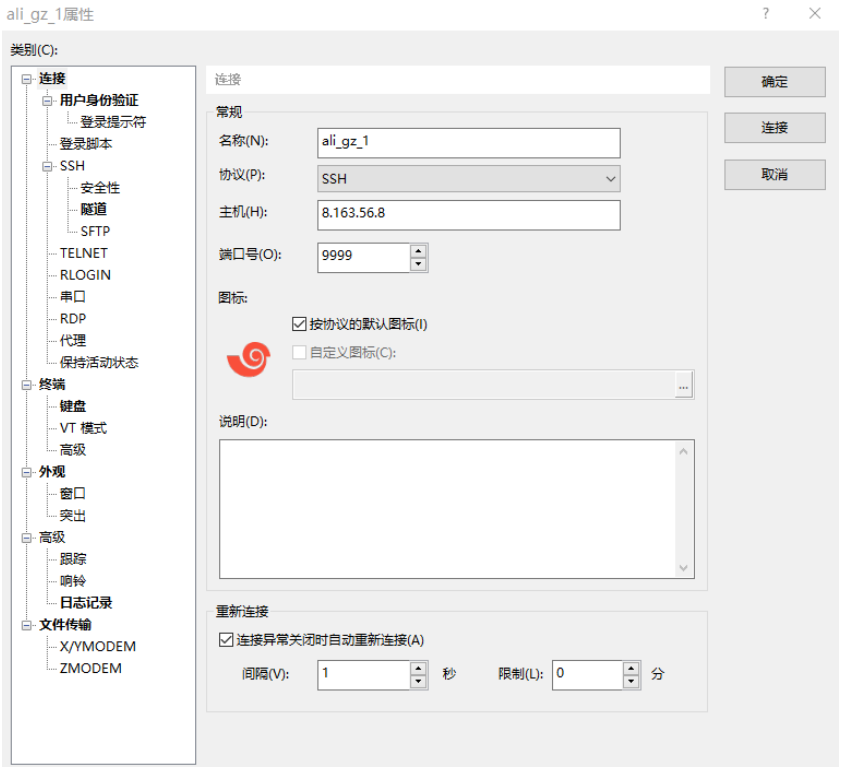
Step 2: Configure the SNAT,

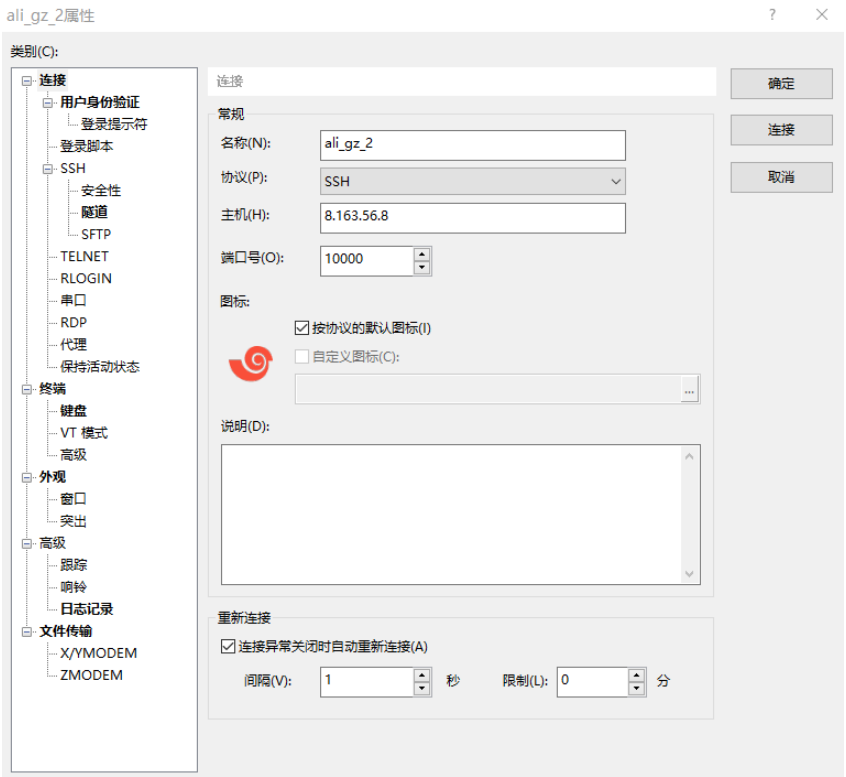
Step 3: Configure the DNAT,

ECS maps:

DNAT Public IP and Port Number	DNAT Inside IP
8.163.56.8:9999	172.31.210.67
8.163.56.8:10000	172.31.210.70
8.163.56.8:10001	172.31.210.71
8.163.56.8:10002	172.31.210.72
8.163.56.8:10003	172.31.210.73
8.163.56.8:10004	172.31.210.74
8.163.56.8:10005	172.31.210.75
8.163.56.8:10006	172.31.210.76
8.163.56.8:10007	172.31.210.77
8.163.56.8:10008	172.31.210.78
8.163.56.8:10009	172.31.210.79

Access the inside IPs via Xshell,





A.5. Configure the password-free SSH

Step 1: The target is as follows,

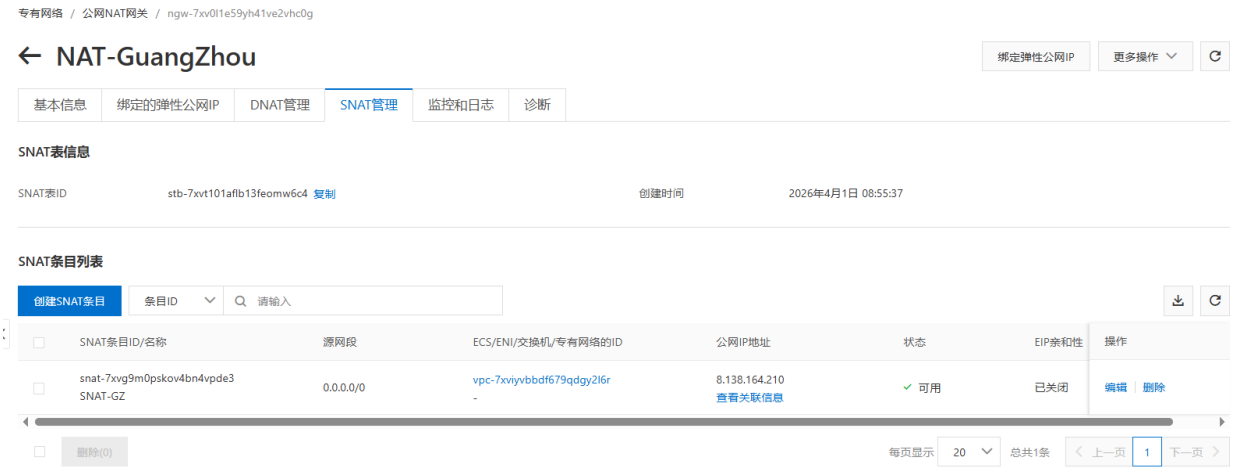
Host with free-password SSH to	
172.31.210.67	-
	172.31.210.70
	172.31.210.71
	172.31.210.72
	172.31.210.73
	172.31.210.74
	172.31.210.75
	172.31.210.76
	172.31.210.77
	172.31.210.78
	172.31.210.79

Step 2: Configure the 172.31.210.67,

```
$ ssh-keygen

$ ssh root@172.31.210.70 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.71 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.72 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.73 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.74 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.75 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.76 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.77 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.78 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub
$ ssh root@172.31.210.79 'mkdir -p .ssh && cat > .ssh/authorized_keys' < ~/.ssh/id_rsa.pub

$ ssh 172.31.210.70 "echo 'hi'"
$ ssh 172.31.210.71 "echo 'hi'"
$ ssh 172.31.210.72 "echo 'hi'"
$ ssh 172.31.210.73 "echo 'hi'"
$ ssh 172.31.210.74 "echo 'hi'"
$ ssh 172.31.210.75 "echo 'hi'"
$ ssh 172.31.210.76 "echo 'hi'"
$ ssh 172.31.210.77 "echo 'hi'"
$ ssh 172.31.210.78 "echo 'hi'"
$ ssh 172.31.210.79 "echo 'hi'"
```

A.6. Free the NAT Gateway

Step 1: Click https://vpc.console.aliyun.com/nat/cn-guangzhou/nats/ngw-7xv0l1e59yh41ve2vhc0g/snats, delete SNAT,



Step 2: Click https://vpc.console.aliyun.com/nat/cn-guangzhou/nats, unbind public IP, free public IP,

Step 3: Click https://homenew.console.aliyun.com/home/dashboard/ProductAndService, delete ECS,

Step 4: Click https://vpc.console.aliyun.com/nat/cn-guangzhou/nats, delete NAT

B. Run Single-Node Fabric

B.1. Configure Each Node

Step 1: Configure the host names,

```
# sudo vi /etc/hosts
:set paste
I
```

```
172.31.210.67 peer0.org1.example.com

172.31.210.70 client1.peer
172.31.210.71 client2.peer
172.31.210.72 client3.peer
172.31.210.73 client4.peer
172.31.210.74 client5.peer
172.31.210.75 client6.peer
172.31.210.76 client7.peer

172.31.210.77 order.example.com
172.31.210.78 order2.example.com
172.31.210.79 order3.example.com
```

B.2. Install Dependencies on Each Node

Step 1: Install the prerequisites on each node,

```
// Prerequisite: Git
$ sudo apt-get update
$ sudo apt purge -y git
$ sudo apt-get install -y git // =1:2.7.4-0ubuntu1.9
$ git version // 2.7.4
$ export PATH=$PATH:/usr/lib/git-core
$ echo 'export PATH=$PATH:/usr/lib/git-core' >> ~/.bashrc
$ source ~/.bashrc
```

Step 2: Install the cURL on each node,

```
// Prerequisite: cURL
$ sudo apt-get install -y curl
$ curl –version // 7.47.0
```

Step 3: Install the wget on each node,

```
// Prerequisite: wget
```

```
$ sudo apt-get install -y wget=1.17.1-1ubuntu1.5
```

Step 4: Install Docker on each node,

```
// Prerequisite: Docker 17.06.2-ce or greater: Docker 17.12.0-ce
$ sudo apt-get purge -y docker-ce docker-ce-cli
$ wget https://download.docker.com/linux/ubuntu/gpg
$ sudo apt-key add gpg
$ echo 'deb [arch=amd64] https://download.docker.com/linux/ubuntu xenial stable' | sudo tee
/etc/apt/sources.list.d/docker.list
$ sudo apt-get update
$ apt-cache madison docker-ce
$ sudo apt-get install docker-ce=17.12.0~ce-0~ubuntu
$ sudo usermod -aG docker $USER
$ docker version
```

Step 5 (if outside the Great Wall): Install Docker Compose on each node,

```
// Prerequisite: Docker Compose 1.14.0 or greater: 1.21.0
$ sudo apt-get purge -y docker-compose
$ sudo curl -L https://github.com/docker/compose/releases/download/1.21.0/docker-compose-
`uname -s`-`uname -m` -o /usr/local/bin/docker-compose
$ sudo chmod +x /usr/local/bin/docker-compose
$ docker-compose version
```

Step 5 (if within the Great Wall): Install Docker Compose on each node,

```
// Prerequisite: Docker Compose 1.14.0 or greater: 1.21.0
$ sudo apt-get purge -y docker-compose
$ mkdir downloads
$ cd $HOME/downloads
$ wget https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/docker-compose
$ sudo chmod +x docker-compose
$ sudo mv docker-compose /usr/local/bin
$ docker-compose version
```

Step 6 (if outside the Great Wall): Install Golang on each node,

```
// Prerequisite: Go v1.14.8
$ cd $HOME/downloads
$ wget https://golang.org/dl/go1.14.8.linux-amd64.tar.gz
$ tar -xzf go1.14.8.linux-amd64.tar.gz
$ sudo rm -rf /usr/local/go
$ sudo mv go /usr/local
$ mkdir -p $HOME/go
$ echo 'export PATH=$PATH:/usr/local/go/bin' >> ~/.bashrc
```

```
$ echo 'export GOPATH=$HOME/go' >> ~/.bashrc
$ source ~/.bashrc
$ go version // go1.14.8 linux/amd64
```

Step 6 (if within the Great Wall): Install the Golang on each node,

```
// Prerequisite: Go v1.14.8
$ cd $HOME/downloads
$ … // download go1.14.8.linux-amd64.tar.gz on node one, e.g., via winscp,
$ scp go1.14.8.linux-amd64.tar.gz root@172.31.210.70:$HOME/downloads
$ scp go1.14.8.linux-amd64.tar.gz root@172.31.210.71:$HOME/downloads
$ …
$ scp go1.14.8.linux-amd64.tar.gz root@172.31.210.79:$HOME/downloads

$ tar -xzf go1.14.8.linux-amd64.tar.gz
$ sudo mv go /usr/local
$ mkdir -p $HOME/go
$ echo 'export PATH=$PATH:/usr/local/go/bin' >> ~/.bashrc
$ echo 'export GOPATH=$HOME/go' >> ~/.bashrc
$ source ~/.bashrc
$ go version // go1.14.8 linux/amd64
```

Step 7: Install Node.js on each node,

```
// Prerequisite: Node.js v12.20.1 or higher
$ cd $HOME/downloads
$ curl -sL https://deb.nodesource.com/setup_12.x -o nodesource_setup.sh
$ sudo chmod +x nodesource_setup.sh
$ sudo bash nodesource_setup.sh
$ sudo apt-get install -y nodejs --allow-unauthenticated
$ nodejs -v // v12.22.12

$ curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.35.3/install.sh | bash
$ // close the terminals, and open the terminals again
$ cd $HOME/downloads
$ sudo npm -g install nvm
$ nvm ls available
$ nvm install 12.20.1
$ nvm use 12.20.1
```

Step 8: Install the NTP on each node,

```
$ sudo apt-get install ntp
$ sudo service ntp start
```

B.3. Install Fabric 2.2.0

Step 1 (if outside the Great Wall): Install Fabric 2.2.0, including Fabric Samples on each node,

```
$ cd $HOME
$ sudo rm -rf fabric-samples
$ curl -sSL https://bit.ly/2ysbOFE | bash -s -- 2.2.0
```

Step 1 (if within the Great Wall): Install Fabric 2.2.0, including Fabric Samples on each node,

```
$ cd $HOME
$ sudo rm -rf fabric-samples
$ sudo apt-get install unzip
$ cd $HOME
$ wget https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/fabric-v2.2.0.zip
$ unzip fabric-v2.2.0.zip
$ cd fabric-v2.2.0/scripts
```

In bootstrap.sh -> ”pullBinaries()”,

```
download "${BINARY_FILE}"
"https://github.com/hyperledger/fabric/releases/download/v${VERSION}/${BINARY_FILE}"
download "${CA_BINARY_FILE}" "https://github.com/hyperledger/fabric-
ca/releases/download/v${CA_VERSION}/${CA_BINARY_FILE}"
```

Replace with the two lines,

```
download "${BINARY_FILE}"
"https://gitee.com/joe320653/fabric/attach_files/842230/download/${BINARY_FILE}"
download "${CA_BINARY_FILE}" "https://gitee.com/joe320653/fabric-
ca/attach_files/842289/download/${CA_BINARY_FILE}"
```

In bootstrap.sh -> ” cloneSamplesRepo()”,

```
git clone -b master https://github.com/hyperledger/fabric-samples.git && cd fabric-samples && git
checkout v${VERSION}
```

Replace with the two lines,

```
git clone -b master https://gitee.com/joe320653/fabric-samples.git && cd fabric-samples && git
checkout v${VERSION}
```

Step 2: Install docker images and fabric-samples on each node,

```
$ cd $HOME/fabric-v2.2.0/scripts
$ chmod 777 *.sh

$ // configure the docker container service within the Great Wall
$ cd $HOME
$ sudo mkdir -p /etc/docker
$ sudo tee /etc/docker/daemon.json <<-'EOF'
{
  "registry-mirrors": ["https://2htzxp9d.mirror.aliyuncs.com"]
}
EOF
$ sudo systemctl daemon-reload
$ sudo systemctl restart docker

$ cd $HOME/fabric-v2.2.0/scripts
$ sudo ./bootstrap.sh
$ mv fabric-samples $HOME
```

B.4. *Run Test-Network*

Step 1 (if outside the Great Wall): Run test network on each node,

```
$ cd $HOME/fabric-samples/test-network
$ ./network.sh up
$ ./network.sh createChannel
$ ./network.sh deployCC -ccn basic -ccp ../asset-transfer-basic/chaincode-go -ccl go
$ ./network.sh down
```

The success information is as follows,

```
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to Query committed status on peer0.org1, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2
MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to Query committed status on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2
MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required
```

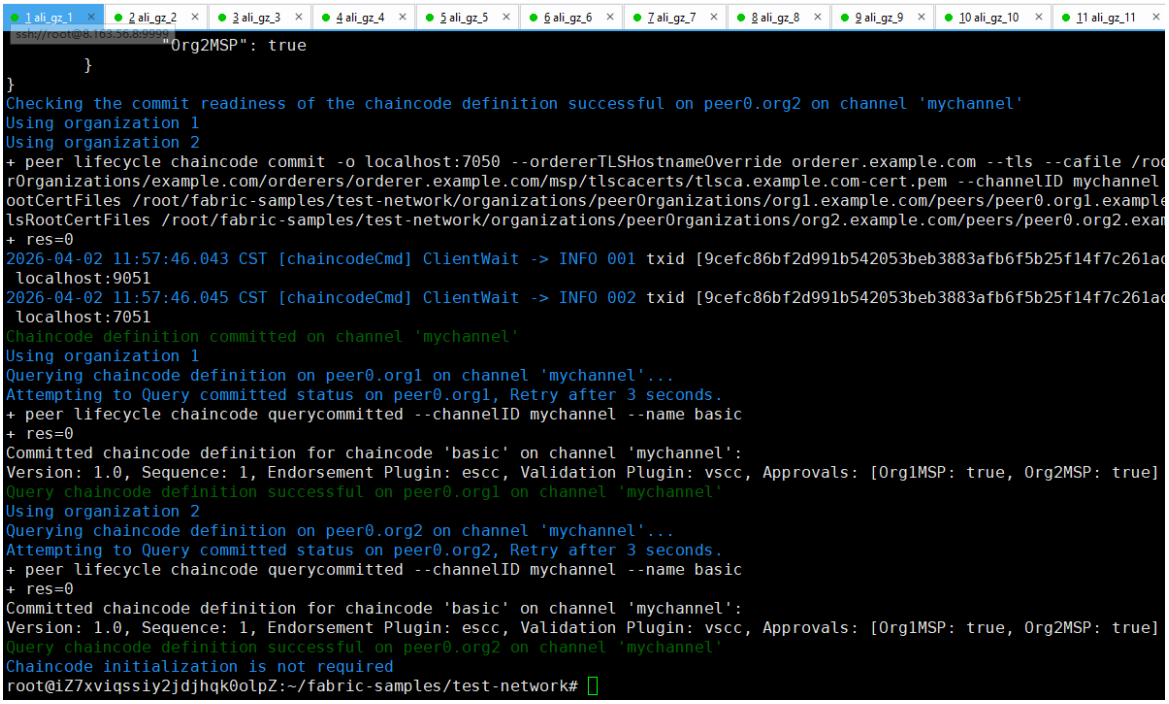
Step 1 (if within the Great Wall): Run test network on each node,

Set the Golang Proxy,

```
$ go env -w GOPROXY=https://goproxy.cn
```

Run the test network,

```
$ cd $HOME/fabric-samples/test-network
$ ./network.sh up
$ ./network.sh createChannel
$ ./network.sh deployCC -ccn basic -ccp ../asset-transfer-basic/chaincode-go -ccl go
$ ./network.sh down
```



1. Run Multi-Nodes Fabric (Exp 1: 1 CLI + 2 Peers + 1 Orderer)

1.1. Prepare the Hosts

Step 1 (Prepare): All machines prepare the host names,

```
$ vi /etc/hosts
```

```
172.31.210.67 cli
172.31.210.70 peer0.org1.example.com
172.31.210.71 peer0.org2.example.com
172.31.210.72 orderer.example.com
```

Step 2 (Prepare): CLI prepares the Test-Network files,

```
$ cd $HOME/fabric-samples
$ mkdir demo-first
$ cd $HOME/fabric-samples/demo-first
$ vi crypto-config.yaml
$ vi configtx.yaml

$ vi peer0-org1.yaml
$ vi peer0-org2.yaml

$ vi orderer1.yaml

$ vi cli.yaml // (of peer0.org1)

$ vi $HOME/fabric-samples/chaincode/abstore/go/abstore.go // our proposed key-value chaincode
$ vi $HOME/fabric-samples/chaincode/abstore/go/go.mod
```

Step 2 (Prepare): All machines prepare the timestamp alignment,

```
$ date +%s // check the timestamp

$ sudo apt-get install ntpdate // align the timestamp
$ sudo ntpdate cn.pool.ntp.org
$ sudo hwclock –systohc

$ date +%s // check the timestamp again
```

Step 3 (Run): CLI runs the initial block generation,

```
$ cd $HOME/fabric-samples/demo-first
$ chmod +x *.sh
$ ./config.sh
```

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# ./config.sh
org1.example.com
org2.example.com
2026-04-14 16:21:05.926 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 002 orderer type: etcdraft
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 003 Orderer.EtcdRaft.Options unset, setting to tick_interval:"500ms" election_
ck:10 heartbeat_tick:1 max_inflight_blocks:5 snapshot_interval_size:16777216
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] Load -> INFO 004 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 005 Generating genesis block
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 006 Writing genesis block
2026-04-14 16:21:05.957 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.973 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.973 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 003 Generating new channel configtx
2026-04-14 16:21:05.974 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 004 Writing new channel tx
2026-04-14 16:21:05.988 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.003 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.003 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.004 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
2026-04-14 16:21:06.018 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.034 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
```

```
$ cd $HOME/fabric-samples/demo-first/workload/tls
$ chmod +x *.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ chmod +x *.sh
$ wget https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/node_modules.zip
$ unzip node_modules.zip

$ cd $HOME/fabric-samples/demo-first/scripts
$ chmod +x *.sh
```

Step 4 (Run): CLI scp the contents to all machines including peers and orderers,

```
$ cd $HOME/fabric-samples/demo-first
$ ./scp.sh
```

```
priv_sk                                   100%  241     0.2KB/s   00:0
tlsca.example.com-cert.pem                100%  847     0.8KB/s   00:0
orderer.example.com-cert.pem              100%  769     0.8KB/s   00:0
tlsca.example.com-cert.pem                100%  847     0.8KB/s   00:0
Admin@example.com-cert.pem                100%  769     0.8KB/s   00:0
ca.example.com-cert.pem                   100%  839     0.8KB/s   00:0
priv_sk                                   100%  241     0.2KB/s   00:0
ca.crt                                    100%  847     0.8KB/s   00:0
server.key                                100%  241     0.2KB/s   00:0
server.crt                                100%  916     0.9KB/s   00:0
Admin@example.com-cert.pem                100%  769     0.8KB/s   00:0
tlsca.example.com-cert.pem                100%  847     0.8KB/s   00:0
Admin@example.com-cert.pem                100%  769     0.8KB/s   00:0
ca.example.com-cert.pem                   100%  839     0.8KB/s   00:0
priv_sk                                   100%  241     0.2KB/s   00:0
ca.crt                                    100%  847     0.8KB/s   00:0
client.crt                                100%  814     0.8KB/s   00:0
client.key                                100%  241     0.2KB/s   00:0
priv_sk                                   100%  241     0.2KB/s   00:0
ca.example.com-cert.pem                   100%  839     0.8KB/s   00:0
```

Step 5 (Run): Orderer1 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./orderer1.sh
```

```
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first# ./orderer1.sh
Creating volume "net_orderer.example.com" with default driver
Creating orderer.example.com ... done
```

$ docker ps

```
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                               COMMAND             CREATED             STATUS              PORTS                    NAMES
4a207f175833        hyperledger/fabric-orderer:2.2.0    "orderer"           8 seconds ago       Up 8 seconds        0.0.0.0:7050->7050/tcp   orderer.example.com
```

Step 6 (Run): Peer1 sets up the Docker container,

$ cd $HOME/fabric-samples/demo-first
$ ./**peer1.sh**

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first# ./peer1.sh
Creating volume "net_peer0.org1.example.com" with default driver
Creating peer0.org1.example.com ... done
```

$ docker ps

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                               COMMAND             CREATED             STATUS              PORTS                    NAMES
38fb1129b2ae        hyperledger/fabric-peer:2.2.0       "peer node start"   2 seconds ago       Up 1 second         0.0.0.0:7051->7051/tcp   peer0.org1.example.com
```

Step 7 (Run): Peer2 sets up the Docker container,

$ cd $HOME/fabric-samples/demo-first
$ ./**peer2.sh**

```
root@iZ7xviqssiy2jdsfcldpc1Z:~/fabric-samples/demo-first# ./peer2.sh
Creating volume "net_peer0.org2.example.com" with default driver
Creating peer0.org2.example.com ... done
```

$ docker ps

```
root@iZ7xviqssiy2jdsfcldpc1Z:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                               COMMAND             CREATED             STATUS              PORTS                                        NAMES
e88f655db1d9        hyperledger/fabric-peer:2.2.0       "peer node start"   2 seconds ago       Up 1 second         7051/tcp, 0.0.0.0:8051->8051/tcp             peer0.org2.example.com
```

Step 8 (Run): CLI sets up the Docker container,

$ cd $HOME/fabric-samples/demo-first
$ ./**cli.sh**

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# ./cli.sh
Creating cli ... done
```

$ docker ps

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                               COMMAND             CREATED             STATUS              PORTS               NAMES
30e6fbad4c51        hyperledger/fabric-tools:2.2.0      "/bin/bash"         2 seconds ago       Up 1 second                             cli
```

1.2. Start the Network

Step 1 (Run): CLI connects to Peer0.org1.example.com:7051, creates a channel.block, and then all peers join the channel.block,

// CLI goes to `peer0.org1.example.com:7051`
$ docker exec -it cli bash

// Environment Configuration for `peer0.org1.example.com:7051`
cd scripts
./**step1.sh** // All peers join the channel

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                          COMMAND             CREATED             STATUS              PORTS               NAMES
30e6fbad4c51        hyperledger/fabric-tools:2.2.0   "/bin/bash"         2 seconds ago       Up 1 second                             cli
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# docker exec -it cli bash
bash-5.0# cd scripts
bash-5.0# ./step1.sh
2026-04-14 08:26:37.630 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:37.647 UTC [cli.common] readBlock -> INFO 002 Expect block, but got status: &{NOT_FOUND}
2026-04-14 08:26:37.649 UTC [channelCmd] InitCmdFactory -> INFO 003 Endorser and orderer connections initialized
2026-04-14 08:26:37.850 UTC [cli.common] readBlock -> INFO 004 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:37.852 UTC [channelCmd] InitCmdFactory -> INFO 005 Endorser and orderer connections initialized
2026-04-14 08:26:38.053 UTC [cli.common] readBlock -> INFO 006 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.054 UTC [channelCmd] InitCmdFactory -> INFO 007 Endorser and orderer connections initialized
2026-04-14 08:26:38.255 UTC [cli.common] readBlock -> INFO 008 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.257 UTC [channelCmd] InitCmdFactory -> INFO 009 Endorser and orderer connections initialized
2026-04-14 08:26:38.457 UTC [cli.common] readBlock -> INFO 00a Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.459 UTC [channelCmd] InitCmdFactory -> INFO 00b Endorser and orderer connections initialized
2026-04-14 08:26:38.659 UTC [cli.common] readBlock -> INFO 00c Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.661 UTC [channelCmd] InitCmdFactory -> INFO 00d Endorser and orderer connections initialized
2026-04-14 08:26:38.864 UTC [cli.common] readBlock -> INFO 00e Received block: 0
2026-04-14 08:26:38.897 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:38.921 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-14 08:26:38.953 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:38.977 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-14 08:26:39.004 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:39.015 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
2026-04-14 08:26:39.040 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:39.050 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0#
```

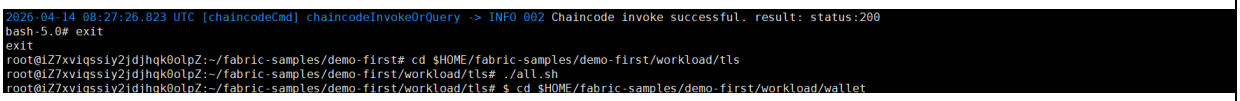
./**step2.sh** // All peers join the chaincode

```
11ac10e3978491cdaab7d56219165c0b45\022\006mycc_1" >
2026-04-14 08:27:14.093 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 002 Chaincode code package identifier: mycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cda
b7d56219165c0b45
2026-04-14 08:27:20.092 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 001 Installed remotely: response:<status:200 payload:"\nGmycc_1:9c2d699dc0bb37ea77cfbadd7567(
11ac10e3978491cdaab7d56219165c0b45\022\006mycc_1" >
2026-04-14 08:27:20.093 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 002 Chaincode code package identifier: mycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cda
b7d56219165c0b45
2026-04-14 08:27:20.160 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychannel) orderer endpoint: orderer.example.com:7050
2026-04-14 08:27:21.182 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [2e49555a3a8dd90810de8e5e90973f90a615a62711c3be05cc97ddf7b1cc0a23] committed with status (VALID) at
2026-04-14 08:27:21.218 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychannel) orderer endpoint: orderer.example.com:7050
2026-04-14 08:27:22.238 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [a720bb2d81a5e1dc2f56ab369b6df082355423c38337e372ffc9f80a515d6f26] committed with status (VALID) at
{
        "approvals": {
                "Org1MSP": true,
                "Org2MSP": true
        }
}
2026-04-14 08:27:23.402 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [f8eba34bdcfcd726a6959a47dc60c95e05cf7652a8b7f4cf80c7b44f70bd236b] committed with status (VALID) at peer0
rg1.example.com:7051
2026-04-14 08:27:23.402 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [f8eba34bdcfcd726a6959a47dc60c95e05cf7652a8b7f4cf80c7b44f70bd236b] committed with status (VALID) at peer0
rg2.example.com:8051
2026-04-14 08:27:24.693 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [862b6b71fe7bdc81df4b23af39e63d1e1017b2c3b39e34b6cdf6ac6213bb905c] committed with status (VALID) at peer0
rg1.example.com:7051
2026-04-14 08:27:24.693 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
2026-04-14 08:27:25.741 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [1722e9e0ffb97b289c38f53946f397b242b6d3ff591f2ba4f48ae6ed239311f3] committed with status (VALID) at peer0
rg1.example.com:7051
2026-04-14 08:27:25.742 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
200tps
2026-04-14 08:27:26.823 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [64c2861c9c16c9c93e549c87eab9181e87ff23827fb1207ae4c213c802b92110] committed with status (VALID) at peer0
rg2.example.com:8051
2026-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
bash-5.0#
```

1.3. Start Nodejs SDK

Step 1 (Run): CLI configures the Node.js SDK folder and tls, wallet, and workload files for all peers,

```
# exit
$ mkdir -p $HOME/fabric-samples/demo-first/workload
$ mkdir -p $HOME/fabric-samples/demo-first/workload/tls
$ mkdir -p $HOME/fabric-samples/demo-first/workload/wallet
$ mkdir -p $HOME/fabric-samples/demo-first/workload/result

$ cd $HOME/fabric-samples/demo-first/workload/tls
$ vi read_pem.py
$ vi prepare_tls_peer0_org1.sh
$ vi prepare_tls_peer0_org2.sh
$ ./all.sh
```




```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ vi prepare_wallet_user1_org1.py
$ vi prepare_wallet_user1_org2.py
$ ./all.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ mkdir sdk-peer0-org1
$ mkdir sdk-peer0-org2
$ vi invoke.js
$ vi prepare_sdk_peer0_org1.sh
$ vi prepare_sdk_peer0_org2.sh
$ ./all.sh
```

Step 2 (Prepare): CLI prepares the Nodejs SDK dependencies on CLI. Note that you may pass this step by copying the compiled nodejs source code https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/node_modules.zip,

```
$ cd $HOME/fabric-samples/demo-first/workload
$ vi package.json
$ npm install
$ vi $HOME/fabric-samples/demo-first/workload/node_modules/fabric-network/lib/transaction.js
```

Step 3 (Run): Cli prepares wallets of User1@org1.example.com and User1@org2.example.com,

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ rm -rf *.id
$ python prepare_wallet_user1_org1.py
```

$ python prepare_wallet_user1_org2.py
$ ./**all.sh**

Step 4 (Run): CLI invokes and queries transactions via Node.js,

$ cd $HOME/fabric-samples/demo-first/workload
$ ./**scp.sh** // CLI copies files to all machines

$ ./**ssh.sh** **> result/benchlog**
$ ls result -l

```
root@iZ7xviqssiy2jdsfcldpbvZ:~# cd $HOME/fabric-samples/demo-first/workload/sdk-peer0-org1
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first/workload/sdk-peer0-org1# nodejs invoke.js 60 100 10 1
Wallet path: /root/fabric-samples/demo-first/workload/wallet
Send_Endorsing_Proposal: 1776155562.442
Send_Endorsing_Proposal: 1776155562.447
Send_Endorsing_Proposal: 1776155562.45
Send_Endorsing_Proposal: 1776155562.452
Send_Endorsing_Proposal: 1776155562.455
Send_Endorsing_Proposal: 1776155562.458
Send_Endorsing_Proposal: 1776155562.46
Send_Endorsing_Proposal: 1776155562.462
Send_Endorsing_Proposal: 1776155562.464
Send_Endorsing_Proposal: 1776155562.468
Send_Endorsing_Proposal: 1776155562.471
Send_Endorsing_Proposal: 1776155562.474
Send_Endorsing_Proposal: 1776155562.476
Send_Endorsing_Proposal: 1776155562.478
Send_Endorsing_Proposal: 1776155562.48
Send_Endorsing_Proposal: 1776155562.482
Send_Endorsing_Proposal: 1776155562.484
Send_Endorsing_Proposal: 1776155562.487
Send_Endorsing_Proposal: 1776155562.488
Send_Endorsing_Proposal: 1776155562.49
Send_Endorsing_Proposal: 1776155562.492
Send_Endorsing_Proposal: 1776155562.494
Send_Endorsing_Proposal: 1776155562.495
Send_Endorsing_Proposal: 1776155562.497
Send_Endorsing_Proposal: 1776155562.499
Send_Endorsing_Proposal: 1776155562.502
Send_Endorsing_Proposal: 1776155562.504
Send_Endorsing_Proposal: 1776155562.507
```

// $ node query.js
// $ node invoke.js 30 500 20 1

Step 5 (Run): Analyze the log,

// CLI analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ python **readthroughput_p1.py** // rate of sending endorsed txs

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/result# python readthroughput_p1.py
Avg_Send_Rate_Phrase_1:
5.7471256178
```

$ sudo apt-get install python-numpy
$ python readdelay_p1.py // delay of endorsing phase

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/result# python readdelay_p1.py
Mean_Delay_Endorse:
0.0188799798489
Mean_Delay_Orderer:
0.00146238137317
Mean_Delay_Commit:
0.178739959002
CDF with the probability of 50% 0.00100016593933
CDF with the probability of 90% 0.00200009346008
CDF with the probability of 95% 0.00300002098083
```

// Orderer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& ordererlog
$ **docker logs $(docker ps -a | grep 'orderer'| awk '{print $1 }') >& ordererlog**
// $ scp ordererlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python readthroughput_p2.py // rate of generating blocks

```
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first/workload/result# python readthroughput_p2.py
Block_ID:
lastPersistedBlock=[24]

lastPersistedBlock=[25]

lastPersistedBlock=[26]

lastPersistedBlock=[27]

lastPersistedBlock=[28]

lastPersistedBlock=[29]

lastPersistedBlock=[30]

lastPersistedBlock=[31]

lastPersistedBlock=[32]
```

// Peer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& peerlog
$ **docker logs $(docker ps -a | grep 'peer node start'| awk '{print $1 }') >& peerlog**
// $ scp peerlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python2 readthroughput_p3.py

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first/workload/result# python2 readthroughput_p3.py
block id:
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
```

// Get block size
peer channel fetch 2 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem

Step 6 (Run): All nodes run the clean procedure,

$ cd $HOME/fabric-samples/demo-first
$ ./clean.sh

2. Run Multi-Nodes Fabric (Exp 2: 1 CLI + 2 Peers + 5 Orderers)

2.1. Prepare the Hosts

Step 1 (Prepare): All machines prepare the host names,

```
$ vi /etc/hosts
```

```
172.31.210.67 cli
172.31.210.70 peer0.org1.example.com
172.31.210.71 peer0.org2.example.com
172.31.210.72 orderer.example.com
172.31.210.73 orderer2.example.com
172.31.210.74 orderer3.example.com
172.31.210.75 orderer4.example.com
172.31.210.76 orderer5.example.com
```

Step 2 (Prepare): CLI prepares the Test-Network files,

```
$ cd $HOME/fabric-samples
$ mkdir demo-first
$ cd $HOME/fabric-samples/demo-first
$ vi crypto-config.yaml
$ vi configtx.yaml

$ vi peer0-org1.yaml
$ vi peer0-org2.yaml

$ vi orderer1.yaml
$ vi orderer2.yaml
$ vi orderer3.yaml
$ vi orderer4.yaml
$ vi orderer5.yaml

$ vi cli.yaml
```

Step 2 (Prepare): All machines prepare the timestamp alignment,

```
$ date +%s // check the timestamp

$ sudo apt-get install ntpdate // align the timestamp
$ sudo ntpdate cn.pool.ntp.org
$ sudo hwclock –systohc

$ date +%s // check the timestamp again
```

Step 3 (Run): CLI runs the initial block generation,

$ cd $HOME/fabric-samples/demo-first
$ chmod +x *.sh
$ ./**config.sh**

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# ./config.sh
org1.example.com
org2.example.com
2026-04-20 10:50:10.813 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.832 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 002 orderer type: etcdraft
2026-04-20 10:50:10.832 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 003 Orderer.EtcdRaft.Options unset, setting to tick_interval:"500ms" el
ction_tick:10 heartbeat_tick:1 max_inflight_blocks:5 snapshot_interval_size:16777216
2026-04-20 10:50:10.832 CST [common.tools.configtxgen.localconfig] Load -> INFO 004 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.833 CST [common.tools.configtxgen] doOutputBlock -> INFO 005 Generating genesis block
2026-04-20 10:50:10.834 CST [common.tools.configtxgen] doOutputBlock -> INFO 006 Writing genesis block
2026-04-20 10:50:10.847 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.867 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.867 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 003 Generating new channel configtx
2026-04-20 10:50:10.868 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 004 Writing new channel tx
2026-04-20 10:50:10.882 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.902 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.902 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-20 10:50:10.902 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
2026-04-20 10:50:10.916 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.935 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.935 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-20 10:50:10.936 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
```

Step 4 (Run): CLI scp the contents to all machines including peers and orderers,

$ cd $HOME/fabric-samples/demo-first
$ ./**scp.sh**

```
priv_sk                         100%  241    0.2KB/s   00:0
tlsca.example.com-cert.pem      100%  847    0.8KB/s   00:0
orderer.example.com-cert.pem    100%  769    0.8KB/s   00:0
tlsca.example.com-cert.pem      100%  847    0.8KB/s   00:0
Admin@example.com-cert.pem      100%  769    0.8KB/s   00:0
ca.example.com-cert.pem         100%  839    0.8KB/s   00:0
priv_sk                         100%  241    0.2KB/s   00:0
ca.crt                          100%  847    0.8KB/s   00:0
server.key                      100%  241    0.2KB/s   00:0
server.crt                      100%  916    0.9KB/s   00:0
Admin@example.com-cert.pem      100%  769    0.8KB/s   00:0
tlsca.example.com-cert.pem      100%  847    0.8KB/s   00:0
Admin@example.com-cert.pem      100%  769    0.8KB/s   00:0
ca.example.com-cert.pem         100%  839    0.8KB/s   00:0
priv_sk                         100%  241    0.2KB/s   00:0
ca.crt                          100%  847    0.8KB/s   00:0
client.crt                      100%  814    0.8KB/s   00:0
client.key                      100%  241    0.2KB/s   00:0
priv_sk                         100%  241    0.2KB/s   00:0
ca.example.com-cert.pem         100%  839    0.8KB/s   00:0
```

Step 5 (Run): Orderers sets up the Docker container,

$ cd $HOME/fabric-samples/demo-first
$ ./**orderer1.sh**

```
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first# ./orderer1.sh
Creating volume "net_orderer.example.com" with default driver
Creating orderer.example.com ... done
```

$ ./**orderer2.sh**

```
root@iZ7xviqssiy2jdsfcldpbwZ:~/fabric-samples/demo-first# ./orderer2.sh
Creating network "net_byfn" with the default driver
Creating volume "net_orderer2.example.com" with default driver
Creating orderer2.example.com ... done
```

$ ./**orderer3.sh**

```
root@iZ7xviqssiy2jdsfcldpbzZ:~/fabric-samples/demo-first# ./orderer3.sh
Creating network "net_byfn" with the default driver
Creating volume "net_orderer3.example.com" with default driver
Creating orderer3.example.com ... done
```

$ ./**orderer4.sh**

```
root@iZ7xviqssiy2jdsfcldpc3Z:~/fabric-samples/demo-first# ./orderer4.sh
Creating network "net_byfn" with the default driver
Creating volume "net_orderer4.example.com" with default driver
Creating orderer4.example.com ... done
```

$ ./**orderer5.sh**

```
root@iZ7xviqssiy2jdsfcldpc0Z:~/fabric-samples/demo-first# ./orderer5.sh
Creating network "net_byfn" with the default driver
Creating volume "net_orderer5.example.com" with default driver
Creating orderer5.example.com ... done
```

$ docker ps

Step 6 (Run): Peer1 sets up the Docker container,

$ cd $HOME/fabric-samples/demo-first
$ ./**peer1.sh**

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first# ./peer1.sh
Creating volume "net_peer0.org1.example.com" with default driver
Creating peer0.org1.example.com ... done
```

$ docker ps

Step 7 (Run): Peer2 sets up the Docker container,

$ cd $HOME/fabric-samples/demo-first
$ ./**peer2.sh**

```
root@iZ7xviqssiy2jdsfcldpc1Z:~/fabric-samples/demo-first# ./peer2.sh
Creating volume "net_peer0.org2.example.com" with default driver
Creating peer0.org2.example.com ... done
```

$ docker ps

Step 8 (Run): CLI sets up the Docker container,

$ cd $HOME/fabric-samples/demo-first
$ ./**cli.sh**

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# ./cli.sh
Creating cli ... done
```

$ docker ps

2.2. Start the Network

Step 1 (Run): CLI connects to Peer0.org1.example.com:7051, creates a channel.block, and then all peers join the channel.block,

// CLI goes to `peer0.org1.example.com:7051`
$ **docker exec -it cli bash**

// Environment Configuration for `peer0.org1.example.com:7051`
cd scripts
./**step1.sh** // All peers join the channel

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# docker exec -it cli bash
bash-5.0# ls
channel-artifacts  crypto            scripts
bash-5.0# cd scripts
bash-5.0# ls
mychannel.block  step1.sh        step2.sh
bash-5.0# ./step1.sh
2026-04-20 03:02:19.759 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:19.776 UTC [cli.common] readBlock -> INFO 002 Expect block, but got status: &{NOT_FOUND}
2026-04-20 03:02:19.778 UTC [channelCmd] InitCmdFactory -> INFO 003 Endorser and orderer connections initialized
2026-04-20 03:02:19.979 UTC [cli.common] readBlock -> INFO 004 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-20 03:02:19.980 UTC [channelCmd] InitCmdFactory -> INFO 005 Endorser and orderer connections initialized
2026-04-20 03:02:20.181 UTC [cli.common] readBlock -> INFO 006 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-20 03:02:20.183 UTC [channelCmd] InitCmdFactory -> INFO 007 Endorser and orderer connections initialized
2026-04-20 03:02:20.383 UTC [cli.common] readBlock -> INFO 008 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-20 03:02:20.385 UTC [channelCmd] InitCmdFactory -> INFO 009 Endorser and orderer connections initialized
2026-04-20 03:02:20.586 UTC [cli.common] readBlock -> INFO 00a Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-20 03:02:20.587 UTC [channelCmd] InitCmdFactory -> INFO 00b Endorser and orderer connections initialized
2026-04-20 03:02:20.788 UTC [cli.common] readBlock -> INFO 00c Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-20 03:02:20.790 UTC [channelCmd] InitCmdFactory -> INFO 00d Endorser and orderer connections initialized
2026-04-20 03:02:20.993 UTC [cli.common] readBlock -> INFO 00e Received block: 0
2026-04-20 03:02:21.027 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.053 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-20 03:02:21.084 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.112 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-20 03:02:21.139 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.151 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
2026-04-20 03:02:21.176 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.187 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0#
```

./**step2.sh** // All peers join the chaincode

2.3. Start Nodejs SDK

Step 1 (Run): CLI configures the Node.js SDK folder and tls, wallet, and workload files for all peers,

exit
$ mkdir -p $HOME/fabric-samples/demo-first/workload
$ mkdir -p $HOME/fabric-samples/demo-first/workload/tls
$ mkdir -p $HOME/fabric-samples/demo-first/workload/wallet
$ mkdir -p $HOME/fabric-samples/demo-first/workload/result

$ cd $HOME/fabric-samples/demo-first/workload/tls
$ vi read_pem.py
$ vi prepare_tls_peer0_org1.sh
$ vi prepare_tls_peer0_org2.sh
$ ./**all.sh**

```
2026-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
bash-5.0# exit
exit
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# cd $HOME/fabric-samples/demo-first/workload/tls
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/tls# ./all.sh
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/tls# $ cd $HOME/fabric-samples/demo-first/workload/wallet
```

$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ vi prepare_wallet_user1_org1.py
$ vi prepare_wallet_user1_org2.py
$ ./**all.sh**

$ cd $HOME/fabric-samples/demo-first/workload

```
$ mkdir sdk-peer0-org1
$ mkdir sdk-peer0-org2
$ vi invoke.js
$ vi prepare_sdk_peer0_org1.sh
$ vi prepare_sdk_peer0_org2.sh
$ ./all.sh
```

Step 2 (Prepare): -

Step 3 (Run): CLI prepares wallets of User1@org1.example.com and User1@org2.example.com,

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ python prepare_wallet_user1_org1.py
$ python prepare_wallet_user1_org2.py
$ ./all.sh
```

Step 4 (Run): CLI invokes and queries transactions via Node.js,

```
$ cd $HOME/fabric-samples/demo-first/workload
$ ./scp.sh // CLI copies files to all machines

$ ./ssh.sh > result/benchlog
$ ls result -l
```

```
root@iZ7xviqssiy2jdsfcldpbvZ:~# cd $HOME/fabric-samples/demo-first/workload/sdk-peer0-org1
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first/workload/sdk-peer0-org1# nodejs invoke.js 60 100 10 1
Wallet path: /root/fabric-samples/demo-first/workload/wallet
Send_Endorsing_Proposal: 1776155562.442
Send_Endorsing_Proposal: 1776155562.447
Send_Endorsing_Proposal: 1776155562.45
Send_Endorsing_Proposal: 1776155562.452
Send_Endorsing_Proposal: 1776155562.455
Send_Endorsing_Proposal: 1776155562.458
Send_Endorsing_Proposal: 1776155562.46
Send_Endorsing_Proposal: 1776155562.462
Send_Endorsing_Proposal: 1776155562.464
Send_Endorsing_Proposal: 1776155562.468
Send_Endorsing_Proposal: 1776155562.471
Send_Endorsing_Proposal: 1776155562.474
Send_Endorsing_Proposal: 1776155562.476
Send_Endorsing_Proposal: 1776155562.478
Send_Endorsing_Proposal: 1776155562.48
Send_Endorsing_Proposal: 1776155562.482
Send_Endorsing_Proposal: 1776155562.484
Send_Endorsing_Proposal: 1776155562.487
Send_Endorsing_Proposal: 1776155562.488
Send_Endorsing_Proposal: 1776155562.49
Send_Endorsing_Proposal: 1776155562.492
Send_Endorsing_Proposal: 1776155562.494
Send_Endorsing_Proposal: 1776155562.495
Send_Endorsing_Proposal: 1776155562.497
Send_Endorsing_Proposal: 1776155562.499
Send_Endorsing_Proposal: 1776155562.502
Send_Endorsing_Proposal: 1776155562.504
Send_Endorsing_Proposal: 1776155562.507
```

```
// $ node query.js
// $ node invoke.js 30 500 20 1
```

Step 5 (Run): Analyze the log,

```
// CLI analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ python readthroughput_p1.py // rate of sending endorsed txs
```

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/result# python readthroughput_p1.py
Avg_Send_Rate_Phrase_1:
5.7471256178
```

$ sudo apt-get install python-numpy
$ python readdelay_p1.py // delay of endorsing phase

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/result# python readdelay_p1.py
Mean_Delay_Endorse:
0.0188799798489
Mean_Delay_Orderer:
0.00146238137317
Mean_Delay_Commit:
0.178739959002
CDF with the probability of 50% 0.00100016593933
CDF with the probability of 90% 0.00200009346008
CDF with the probability of 95% 0.00300002098083
```

// Orderer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& ordererlog
$ **docker logs $(docker ps -a | grep 'orderer'| awk '{print $1 }') >& ordererlog**
// $ scp ordererlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python readthroughput_p2.py // rate of generating blocks

```
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first/workload/result# python readthroughput_p2.py
Block_ID:
lastPersistedBlock=[24]

lastPersistedBlock=[25]

lastPersistedBlock=[26]

lastPersistedBlock=[27]

lastPersistedBlock=[28]

lastPersistedBlock=[29]

lastPersistedBlock=[30]

lastPersistedBlock=[31]

lastPersistedBlock=[32]
```

// Peer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& peerlog
$ **docker logs $(docker ps -a | grep 'peer node start'| awk '{print $1 }') >& peerlog**
// $ scp peerlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python2 readthroughput_p3.py

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first/workload/result# python2 readthroughput_p3.py
block id:
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
```

// Get block size
peer channel fetch 228 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem

Step 6 (Run): All nodes run the clean procedure,

```
$ cd $HOME/fabric-samples/demo-first
$ ./clean.sh
```

3. Run Multi-Nodes Fabric (Exp 3: 1 CLI + 5 Peers + 5 Orderers)

3.1. Prepare the Hosts

Step 1 (Prepare): All machines prepare the host names,

```
$ vi /etc/hosts
```

```
172.31.210.67 cli
172.31.210.70 peer0.org1.example.com
172.31.210.71 peer0.org2.example.com
172.31.210.72 orderer.example.com
172.31.210.73 orderer2.example.com
172.31.210.74 orderer3.example.com
172.31.210.75 orderer4.example.com
172.31.210.76 orderer5.example.com
172.31.210.77 peer0.org3.example.com
172.31.210.78 peer0.org4.example.com
172.31.210.79 peer0.org5.example.com
```

Step 2 (Prepare): CLI prepares the Test-Network files,

```
$ cd $HOME/fabric-samples
$ mkdir demo-first
$ cd $HOME/fabric-samples/demo-first
$ vi crypto-config.yaml
$ vi configtx.yaml

$ vi peer0-org1.yaml
$ vi peer0-org2.yaml
$ vi peer0-org3.yaml
$ vi peer0-org4.yaml
$ vi peer0-org5.yaml

$ vi orderer1.yaml
$ vi orderer2.yaml
$ vi orderer3.yaml
$ vi orderer4.yaml
$ vi orderer5.yaml

$ vi cli.yaml
```

Step 2 (Prepare): All machines prepare the timestamp alignment,

```
$ date +%s // check the timestamp
```

```
$ sudo apt-get install ntpdate // align the timestamp
$ sudo ntpdate cn.pool.ntp.org
$ sudo hwclock –systohc

$ date +%s // check the timestamp again
```

Step 3 (Run): CLI runs the initial block generation,

```
$ cd $HOME/fabric-samples/demo-first
$ chmod +x *.sh
$ ./config.sh
```

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# ./config.sh
org1.example.com
org2.example.com
2026-04-14 16:21:05.926 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 002 orderer type: etcdraft
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 003 Orderer.EtcdRaft.Options unset, setting to tick_interval:"500ms" election_
ck:10 heartbeat_tick:1 max_inflight_blocks:5 snapshot_interval_size:16777216
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] Load -> INFO 004 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 005 Generating genesis block
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 006 Writing genesis block
2026-04-14 16:21:05.957 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.973 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.973 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 003 Generating new channel configtx
2026-04-14 16:21:05.974 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 004 Writing new channel tx
2026-04-14 16:21:05.988 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.003 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.003 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.004 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
2026-04-14 16:21:06.018 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.034 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
```

Step 4 (Run): CLI scp the contents to all machines including peers and orderers,

```
$ cd $HOME/fabric-samples/demo-first
$ ./scp.sh
```

```
priv_sk                                                                                       100%  241     0.2KB/s   00:0
tlsca.example.com-cert.pem                                                                    100%  847     0.8KB/s   00:0
orderer.example.com-cert.pem                                                                  100%  769     0.8KB/s   00:0
tlsca.example.com-cert.pem                                                                    100%  847     0.8KB/s   00:0
Admin@example.com-cert.pem                                                                    100%  769     0.8KB/s   00:0
ca.example.com-cert.pem                                                                       100%  839     0.8KB/s   00:0
priv_sk                                                                                       100%  241     0.2KB/s   00:0
ca.crt                                                                                        100%  847     0.8KB/s   00:0
server.key                                                                                    100%  241     0.2KB/s   00:0
server.crt                                                                                    100%  916     0.9KB/s   00:0
Admin@example.com-cert.pem                                                                    100%  769     0.8KB/s   00:0
tlsca.example.com-cert.pem                                                                    100%  847     0.8KB/s   00:0
Admin@example.com-cert.pem                                                                    100%  769     0.8KB/s   00:0
ca.example.com-cert.pem                                                                       100%  839     0.8KB/s   00:0
priv_sk                                                                                       100%  241     0.2KB/s   00:0
ca.crt                                                                                        100%  847     0.8KB/s   00:0
client.crt                                                                                    100%  814     0.8KB/s   00:0
client.key                                                                                    100%  241     0.2KB/s   00:0
priv_sk                                                                                       100%  241     0.2KB/s   00:0
ca.example.com-cert.pem                                                                       100%  839     0.8KB/s   00:0
```

Step 5 (Run): Orderers sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./orderer1.sh

$ ./orderer2.sh

$ ./orderer3.sh

$ ./orderer4.sh
```

```
$ ./orderer5.sh

$ docker ps
```

Step 6 (Run): Peer1 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./peer1.sh

$ docker ps
```

Step 7 (Run): Other peers set up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./peer2.sh

$ ./peer3.sh

$ ./peer4.sh

$ ./peer5.sh

$ docker ps
```

Step 8 (Run): CLI sets up the docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./cli.sh

root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# ./cli.sh
Creating cli ... done

$ docker ps
```

3.2. Start the Network

Step 1 (Run): CLI (of peer0.org1) connects to Peer0.org1.example.com:7051, creates a channel.block, and then all peers join the channel.block,

```
// CLI  goes to `peer0.org1.example.com:7051`
$ docker exec -it cli bash

// Environment Configuration for `peer0.org1.example.com:7051`
```

cd scripts
./**step1.sh** // All peers join the channel

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                          COMMAND             CREATED             STATUS              PORTS               NAMES
30e6fbad4c51        hyperledger/fabric-tools:2.2.0   "/bin/bash"         2 seconds ago       Up 1 second                             cli
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# docker exec -it cli bash
bash-5.0# cd scripts
bash-5.0# ./step1.sh
2026-04-14 08:26:37.630 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:37.647 UTC [cli.common] readBlock -> INFO 002 Expect block, but got status: &{NOT_FOUND}
2026-04-14 08:26:37.649 UTC [channelCmd] InitCmdFactory -> INFO 003 Endorser and orderer connections initialized
2026-04-14 08:26:37.850 UTC [cli.common] readBlock -> INFO 004 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:37.852 UTC [channelCmd] InitCmdFactory -> INFO 005 Endorser and orderer connections initialized
2026-04-14 08:26:38.053 UTC [cli.common] readBlock -> INFO 006 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.054 UTC [channelCmd] InitCmdFactory -> INFO 007 Endorser and orderer connections initialized
2026-04-14 08:26:38.255 UTC [cli.common] readBlock -> INFO 008 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.257 UTC [channelCmd] InitCmdFactory -> INFO 009 Endorser and orderer connections initialized
2026-04-14 08:26:38.457 UTC [cli.common] readBlock -> INFO 00a Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.459 UTC [channelCmd] InitCmdFactory -> INFO 00b Endorser and orderer connections initialized
2026-04-14 08:26:38.659 UTC [cli.common] readBlock -> INFO 00c Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.661 UTC [channelCmd] InitCmdFactory -> INFO 00d Endorser and orderer connections initialized
2026-04-14 08:26:38.864 UTC [cli.common] readBlock -> INFO 00e Received block: 0
2026-04-14 08:26:38.897 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:38.921 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-14 08:26:38.953 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:38.977 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-14 08:26:39.004 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:39.015 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
2026-04-14 08:26:39.040 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:39.050 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0#
```

./**step2.sh** // All peers join the chaincode

```
2026-04-20 09:35:34.485 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [601b6ba90405a662832c471b05eb27696a5ab493c4b38ec245009af7c9626ee6] committed with status (VALID) a

{
        "approvals": {
                "Org1MSP": true,
                "Org2MSP": true,
                "Org3MSP": true,
                "Org4MSP": true,
                "Org5MSP": true
        }
}
2026-04-20 09:35:35.665 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [bda0c8d144286359f8394ec1a7c9f993bac3a87a6a4099ba9c8b0b8c4f6fb02d] committed with status (VALID) a
 peer0.org2.example.com:7051
2026-04-20 09:35:35.667 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [bda0c8d144286359f8394ec1a7c9f993bac3a87a6a4099ba9c8b0b8c4f6fb02d] committed with status (VALID) a
 peer0.org1.example.com:7051
2026-04-20 09:35:35.668 UTC [chaincodeCmd] ClientWait -> INFO 003 txid [bda0c8d144286359f8394ec1a7c9f993bac3a87a6a4099ba9c8b0b8c4f6fb02d] committed with status (VALID) a
 peer0.org5.example.com:7051
2026-04-20 09:35:35.668 UTC [chaincodeCmd] ClientWait -> INFO 004 txid [bda0c8d144286359f8394ec1a7c9f993bac3a87a6a4099ba9c8b0b8c4f6fb02d] committed with status (VALID) a
 peer0.org4.example.com:7051
2026-04-20 09:35:35.670 UTC [chaincodeCmd] ClientWait -> INFO 005 txid [bda0c8d144286359f8394ec1a7c9f993bac3a87a6a4099ba9c8b0b8c4f6fb02d] committed with status (VALID) a
 peer0.org3.example.com:7051
2026-04-20 09:35:36.920 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [0968d89aa85538bcbac17bd89ac2438435d186162a5915195bed3c374dcebf67] committed with status (VALID) a
 peer0.org1.example.com:7051
2026-04-20 09:35:36.920 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
2026-04-20 09:35:37.970 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [207d83ba4cab05dc9c6e742b66aba3cabc5fdae5e7237cba5a93d9967945a9bc] committed with status (VALID) a
 peer0.org1.example.com:7051
2026-04-20 09:35:37.970 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
200tps
2026-04-20 09:35:39.054 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [891f28608f3b23220f2e49a556a69afcf5395fe5ca58300c0c516f4aae1e12c2] committed with status (VALID) a
 peer0.org2.example.com:7051
2026-04-20 09:35:39.055 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
3000tps
bash-5.0#
```

3.3. Start Nodejs SDK

Step 1 (Run): CLI configures the Node.js SDK folder and tls, wallet, and workload files for all peers,

```
# exit
$ mkdir -p $HOME/fabric-samples/demo-first/workload
$ mkdir -p $HOME/fabric-samples/demo-first/workload/tls
$ mkdir -p $HOME/fabric-samples/demo-first/workload/wallet
$ mkdir -p $HOME/fabric-samples/demo-first/workload/result

$ cd $HOME/fabric-samples/demo-first/workload/tls
$ vi read_pem.py
$ vi prepare_tls_peer0_org1.sh
```

$ vi prepare_tls_peer0_org2.sh
$ vi prepare_tls_peer0_org3.sh
$ vi prepare_tls_peer0_org4.sh
$ vi prepare_tls_peer0_org5.sh
$ ./**all.sh**

```
2026-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
bash-5.0# exit
exit
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# cd $HOME/fabric-samples/demo-first/workload/tls
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/tls# ./all.sh
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/tls# $ cd $HOME/fabric-samples/demo-first/workload/wallet
```

$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ vi prepare_wallet_user1_org1.py
$ vi prepare_wallet_user1_org2.py
$ vi prepare_wallet_user1_org3.py
$ vi prepare_wallet_user1_org4.py
$ vi prepare_wallet_user1_org5.py
$ ./**all.sh**

$ cd $HOME/fabric-samples/demo-first/workload
$ mkdir sdk-peer0-org1
$ mkdir sdk-peer0-org2
$ mkdir sdk-peer0-org3
$ mkdir sdk-peer0-org4
$ mkdir sdk-peer0-org5
$ vi invoke.js
$ vi prepare_sdk_peer0_org1.sh
$ vi prepare_sdk_peer0_org2.sh
$ vi prepare_sdk_peer0_org3.sh
$ vi prepare_sdk_peer0_org4.sh
$ vi prepare_sdk_peer0_org5.sh
$ ./**all.sh**

Step 2 (Prepare): -

Step 3 (Run): Cli prepares wallets of User1@org1.example.com, User1@org2.example.com, …, and User1@org5.example.com,

$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ python prepare_wallet_user1_org1.py
$ python prepare_wallet_user1_org2.py
$ python prepare_wallet_user1_org3.py
$ python prepare_wallet_user1_org4.py
$ python prepare_wallet_user1_org5.py
$ ./**all.sh**

Step 4 (Run): CLI invokes and queries transactions via Node.js,

$ cd $HOME/fabric-samples/demo-first/workload

$ ./**scp.sh** // CLI copies files to all machines

$ ./**ssh.sh** **> result/benchlog**
$ ls result -l

```
root@iZ7xviqssiy2jdsfcldpbvZ:~# cd $HOME/fabric-samples/demo-first/workload/sdk-peer0-org1
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first/workload/sdk-peer0-org1# nodejs invoke.js 60 100 10 1
Wallet path: /root/fabric-samples/demo-first/workload/wallet
Send_Endorsing_Proposal: 1776155562.442
Send_Endorsing_Proposal: 1776155562.447
Send_Endorsing_Proposal: 1776155562.45
Send_Endorsing_Proposal: 1776155562.452
Send_Endorsing_Proposal: 1776155562.455
Send_Endorsing_Proposal: 1776155562.458
Send_Endorsing_Proposal: 1776155562.46
Send_Endorsing_Proposal: 1776155562.462
Send_Endorsing_Proposal: 1776155562.464
Send_Endorsing_Proposal: 1776155562.468
Send_Endorsing_Proposal: 1776155562.471
Send_Endorsing_Proposal: 1776155562.474
Send_Endorsing_Proposal: 1776155562.476
Send_Endorsing_Proposal: 1776155562.478
Send_Endorsing_Proposal: 1776155562.48
Send_Endorsing_Proposal: 1776155562.482
Send_Endorsing_Proposal: 1776155562.484
Send_Endorsing_Proposal: 1776155562.487
Send_Endorsing_Proposal: 1776155562.488
Send_Endorsing_Proposal: 1776155562.49
Send_Endorsing_Proposal: 1776155562.492
Send_Endorsing_Proposal: 1776155562.494
Send_Endorsing_Proposal: 1776155562.495
Send_Endorsing_Proposal: 1776155562.497
Send_Endorsing_Proposal: 1776155562.499
Send_Endorsing_Proposal: 1776155562.502
Send_Endorsing_Proposal: 1776155562.504
Send_Endorsing_Proposal: 1776155562.507
```

// $ node query.js
// $ node invoke.js 30 500 20 1

Step 5 (Run): Analyze the log,

// CLI analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ python **readthroughput_p1.py** // rate of sending endorsed txs

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/result# python readthroughput_p1.py
Avg_Send_Rate_Phrase_1:
5.7471256178
```

$ sudo apt-get install python-numpy
$ python readdelay_p1.py // delay of endorsing phase

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/result# python readdelay_p1.py
Mean_Delay_Endorse:
0.0188799798489
Mean_Delay_Orderer:
0.00146238137317
Mean_Delay_Commit:
0.178739959002
CDF with the probability of 50% 0.00100016593933
CDF with the probability of 90% 0.00200009346008
CDF with the probability of 95% 0.00300002098083
```

// Orderer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& ordererlog
$ **docker logs $(docker ps -a | grep 'orderer'| awk '{print $1 }') >& ordererlog**
// $ scp ordererlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python readthroughput_p2.py // rate of generating blocks

```
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first/workload/result# python readthroughput_p2.py
Block_ID:
lastPersistedBlock=[24]

lastPersistedBlock=[25]

lastPersistedBlock=[26]

lastPersistedBlock=[27]

lastPersistedBlock=[28]

lastPersistedBlock=[29]

lastPersistedBlock=[30]

lastPersistedBlock=[31]

lastPersistedBlock=[32]
```

// Peer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& peerlog
$ **docker logs $(docker ps -a | grep 'peer node start'| awk '{print $1 }') >& peerlog**
// $ scp peerlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python2 readthroughput_p3.py

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first/workload/result# python2 readthroughput_p3.py
block id:
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
```

// Get block size
peer channel fetch 228 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem

Step 6 (Run): All nodes run the clean procedure,

$ cd $HOME/fabric-samples/demo-first
$ ./clean.sh

4. Run Multi-Nodes Fabric (Exp 4: 7 CLI + 1 Peers + 15 Orderers)

4.1. Prepare the Hosts

Step 1 (Prepare): All machines prepare the host names,

```
$ vi /etc/hosts
```

```
172.31.210.67 cli # CLI

172.31.210.70 peer0.org1.example.com # Endorsing peer 1
172.31.210.71 peer0.org2.example.com # Endorsing peer 2
172.31.210.77 peer0.org3.example.com # Endorsing peer 3
172.31.210.78 peer0.org4.example.com # Endorsing peer 4
172.31.210.79 peer0.org5.example.com # Endorsing peer 5
172.31.210.80 peer0.org6.example.com # Endorsing peer 6
172.31.210.81 peer0.org7.example.com # Endorsing peer 7

172.31.210.82 peer0.org8.example.com # A Non-Endorsing Peer

172.31.210.72 orderer.example.com # Orderer 1
172.31.210.73 orderer2.example.com # Orderer 2
172.31.210.74 orderer3.example.com # Orderer 3
172.31.210.75 orderer4.example.com # Orderer 4
172.31.210.76 orderer5.example.com # Orderer 5
172.31.210.83 orderer6.example.com # Orderer 6
172.31.210.84 orderer7.example.com # Orderer 7
172.31.210.85 orderer8.example.com # Orderer 8
172.31.210.86 orderer9.example.com # Orderer 9
172.31.210.87 orderer10.example.com # Orderer 10
172.31.210.88 orderer11.example.com # Orderer 11
172.31.210.89 orderer12.example.com # Orderer 12
172.31.210.90 orderer13.example.com # Orderer 13
172.31.210.91 orderer14.example.com # Orderer 14
172.31.210.92 orderer15.example.com # Orderer 15
```

Step 2 (Prepare): CLI prepares the Test-Network files,

```
$ cd $HOME/fabric-samples
$ mkdir demo-first
$ cd $HOME/fabric-samples/demo-first
$ vi crypto-config.yaml
$ vi configtx.yaml

$ vi peer0-org1.yaml
$ vi peer0-org2.yaml
$ vi peer0-org3.yaml
```

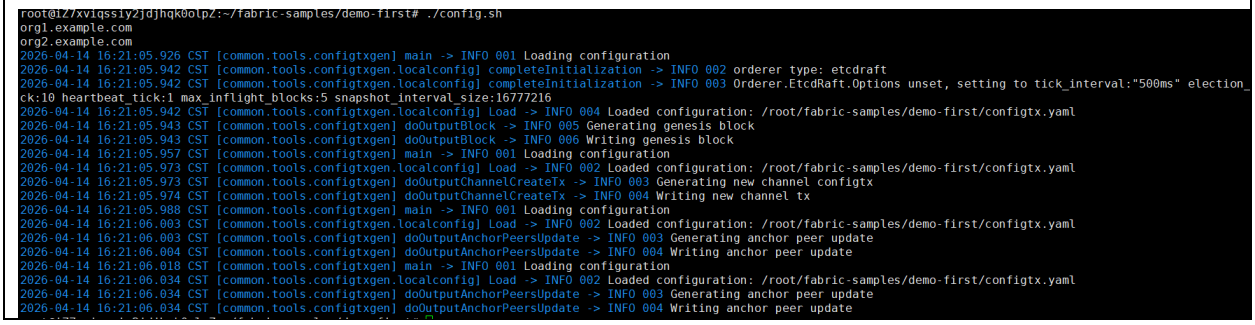
```
$ vi peer0-org4.yaml
$ vi peer0-org5.yaml
$ vi peer0-org6.yaml
$ vi peer0-org7.yaml
$ vi peer0-org8.yaml

$ vi orderer1.yaml
$ vi orderer2.yaml
$ vi orderer3.yaml
$ vi orderer4.yaml
$ vi orderer5.yaml
$ vi orderer6.yaml
$ vi orderer7.yaml
$ vi orderer8.yaml
$ vi orderer9.yaml
$ vi orderer10.yaml
$ vi orderer11.yaml
$ vi orderer12.yaml
$ vi orderer13.yaml
$ vi orderer14.yaml
$ vi orderer15.yaml

$ vi cli.yaml
```

Step 2 (Prepare): All machines prepare the timestamp alignment,

```
$ date +%s // check the timestamp

$ sudo apt-get install ntpdate // align the timestamp
$ sudo ntpdate cn.pool.ntp.org
$ sudo hwclock –systohc

$ date +%s // check the timestamp again
```

Step 3 (Run): CLI runs the initial block generation,

```
$ cd $HOME/fabric-samples/demo-first
$ chmod +x *.sh
$ ./config.sh
```



Step 4 (Run): CLI scp the contents to all machines including peers and orderers,

```
$ cd $HOME/fabric-samples/demo-first
$ ./scp.sh
```

```
priv_sk                                   100%  241    0.2KB/s   00:0
tlsca.example.com-cert.pem                100%  847    0.8KB/s   00:0
orderer.example.com-cert.pem              100%  769    0.8KB/s   00:0
tlsca.example.com-cert.pem                100%  847    0.8KB/s   00:0
Admin@example.com-cert.pem                100%  769    0.8KB/s   00:0
ca.example.com-cert.pem                   100%  839    0.8KB/s   00:0
priv_sk                                   100%  241    0.2KB/s   00:0
ca.crt                                    100%  847    0.8KB/s   00:0
server.key                                100%  241    0.2KB/s   00:0
server.crt                                100%  916    0.9KB/s   00:0
Admin@example.com-cert.pem                100%  769    0.8KB/s   00:0
tlsca.example.com-cert.pem                100%  847    0.8KB/s   00:0
Admin@example.com-cert.pem                100%  769    0.8KB/s   00:0
ca.example.com-cert.pem                   100%  839    0.8KB/s   00:0
priv_sk                                   100%  241    0.2KB/s   00:0
ca.crt                                    100%  847    0.8KB/s   00:0
client.crt                                100%  814    0.8KB/s   00:0
client.key                                100%  241    0.2KB/s   00:0
priv_sk                                   100%  241    0.2KB/s   00:0
ca.example.com-cert.pem                   100%  839    0.8KB/s   00:0
```

Step 5 (Run): Orderers sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./orderer1.sh

$ ./orderer2.sh

$ ./orderer3.sh

$ ./orderer4.sh

$ ./orderer5.sh

$ ./orderer6.sh

$ ./orderer7.sh

$ ./orderer8.sh

$ ./orderer9.sh

$ ./orderer10.sh

$ ./orderer11.sh

$ ./orderer12.sh

$ ./orderer13.sh

$ ./orderer14.sh

$ ./orderer15.sh
```

```
$ docker ps
```

Step 6 (Run): Peer1 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./peer1.sh

$ docker ps
```

Step 7 (Run): Other peers set up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./peer2.sh

$ ./peer3.sh

$ ./peer4.sh

$ ./peer5.sh

$ ./peer6.sh

$ ./peer7.sh

$ ./peer8.sh

$ docker ps
```

Step 8 (Run): CLI sets up the docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./cli.sh
```


```
$ docker ps
```

4.2. Start the Network

Step 1 (Run): CLI (of peer0.org1) connects to Peer0.org1.example.com:7051, creates a channel.block, and then all peers join the channel.block,

// CLI goes to `peer0.org1.example.com:7051`
$ **docker exec -it cli bash**

// Environment Configuration for `peer0.org1.example.com:7051`
cd scripts
./**step1.sh** // All peers join the channel

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                          COMMAND             CREATED             STATUS              PORTS               NAMES
30e6fbad4c51        hyperledger/fabric-tools:2.2.0   "/bin/bash"         2 seconds ago       Up 1 second                             cli
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# docker exec -it cli bash
bash-5.0# cd scripts
bash-5.0# ./step1.sh
2026-04-14 08:26:37.630 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:37.647 UTC [cli.common] readBlock -> INFO 002 Expect block, but got status: &{NOT_FOUND}
2026-04-14 08:26:37.649 UTC [channelCmd] InitCmdFactory -> INFO 003 Endorser and orderer connections initialized
2026-04-14 08:26:37.850 UTC [cli.common] readBlock -> INFO 004 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:37.852 UTC [channelCmd] InitCmdFactory -> INFO 005 Endorser and orderer connections initialized
2026-04-14 08:26:38.053 UTC [cli.common] readBlock -> INFO 006 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.054 UTC [channelCmd] InitCmdFactory -> INFO 007 Endorser and orderer connections initialized
2026-04-14 08:26:38.255 UTC [cli.common] readBlock -> INFO 008 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.257 UTC [channelCmd] InitCmdFactory -> INFO 009 Endorser and orderer connections initialized
2026-04-14 08:26:38.457 UTC [cli.common] readBlock -> INFO 00a Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.459 UTC [channelCmd] InitCmdFactory -> INFO 00b Endorser and orderer connections initialized
2026-04-14 08:26:38.659 UTC [cli.common] readBlock -> INFO 00c Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.661 UTC [channelCmd] InitCmdFactory -> INFO 00d Endorser and orderer connections initialized
2026-04-14 08:26:38.864 UTC [cli.common] readBlock -> INFO 00e Received block: 0
2026-04-14 08:26:38.897 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:38.921 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-14 08:26:38.953 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:38.977 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-14 08:26:39.004 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:39.015 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
2026-04-14 08:26:39.040 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:39.050 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0#
```

./**step2.sh** // All peers join the chaincode

```
2026-04-20 09:35:34.485 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [601b6ba90405a662832c471b05eb27696a5ab493c4b38ec245009af7c9626ee6] committed with status (VALID)

{
        "approvals": {
                "Org1MSP": true,
                "Org2MSP": true,
                "Org3MSP": true,
                "Org4MSP": true,
                "Org5MSP": true
        }
}
2026-04-20 09:35:35.665 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [bda0c8d144286359f8394ec1a7c9f993bac3a87a6a4099ba9c8b0b8c4f6fb02d] committed with status (VALID)
 peer0.org2.example.com:7051
2026-04-20 09:35:35.667 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [bda0c8d144286359f8394ec1a7c9f993bac3a87a6a4099ba9c8b0b8c4f6fb02d] committed with status (VALID)
 peer0.org1.example.com:7051
2026-04-20 09:35:35.668 UTC [chaincodeCmd] ClientWait -> INFO 003 txid [bda0c8d144286359f8394ec1a7c9f993bac3a87a6a4099ba9c8b0b8c4f6fb02d] committed with status (VALID)
 peer0.org5.example.com:7051
2026-04-20 09:35:35.668 UTC [chaincodeCmd] ClientWait -> INFO 004 txid [bda0c8d144286359f8394ec1a7c9f993bac3a87a6a4099ba9c8b0b8c4f6fb02d] committed with status (VALID)
 peer0.org4.example.com:7051
2026-04-20 09:35:35.670 UTC [chaincodeCmd] ClientWait -> INFO 005 txid [bda0c8d144286359f8394ec1a7c9f993bac3a87a6a4099ba9c8b0b8c4f6fb02d] committed with status (VALID)
 peer0.org3.example.com:7051
2026-04-20 09:35:36.920 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [0968d89aa85538bcbac17bd89ac2438435d186162a5915195bed3c374dcebf67] committed with status (VALID)
 peer0.org1.example.com:7051
2026-04-20 09:35:36.920 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
2026-04-20 09:35:37.970 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [207d83ba4cab05dc9c6e742b66aba3cabc5fdae5e7237cba5a93d9967945a9bc] committed with status (VALID)
 peer0.org1.example.com:7051
2026-04-20 09:35:37.970 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
200tps
2026-04-20 09:35:39.054 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [891f28608f3b23220f2e49a556a69afcf5395fe5ca58300c0c516f4aae1e12c2] committed with status (VALID)
 peer0.org2.example.com:7051
2026-04-20 09:35:39.055 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
3000tps
bash-5.0#
```

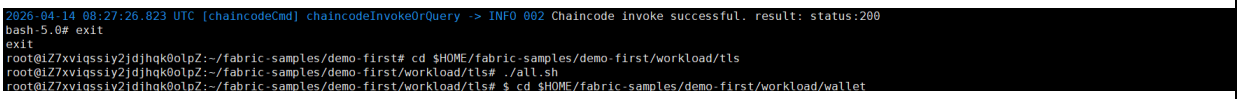
4.3. Start Nodejs SDK

Step 1 (Run): CLI configures the Node.js SDK folder and tls, wallet, and workload files for all peers,

exit
$ mkdir -p $HOME/fabric-samples/demo-first/workload

$ mkdir -p $HOME/fabric-samples/demo-first/workload/tls
$ mkdir -p $HOME/fabric-samples/demo-first/workload/wallet
$ mkdir -p $HOME/fabric-samples/demo-first/workload/result

$ **cd $HOME/fabric-samples/demo-first/workload/tls**
$ vi read_pem.py
$ vi prepare_tls_peer0_org1.sh
$ vi prepare_tls_peer0_org2.sh
$ vi prepare_tls_peer0_org3.sh
$ vi prepare_tls_peer0_org4.sh
$ vi prepare_tls_peer0_org5.sh
$ vi prepare_tls_peer0_org6.sh
$ vi prepare_tls_peer0_org7.sh
$ vi prepare_tls_peer0_org8.sh
$ ./**all.sh**



$ **cd $HOME/fabric-samples/demo-first/workload/wallet**
$ vi prepare_wallet_user1_org1.py
$ vi prepare_wallet_user1_org2.py
$ vi prepare_wallet_user1_org3.py
$ vi prepare_wallet_user1_org4.py
$ vi prepare_wallet_user1_org5.py
$ vi prepare_wallet_user1_org6.py
$ vi prepare_wallet_user1_org7.py
$ vi prepare_wallet_user1_org8.py
$ ./**all.sh**

$ **cd $HOME/fabric-samples/demo-first/workload**
$ mkdir sdk-peer0-org1
$ mkdir sdk-peer0-org2
$ mkdir sdk-peer0-org3
$ mkdir sdk-peer0-org4
$ mkdir sdk-peer0-org5
$ mkdir sdk-peer0-org6
$ mkdir sdk-peer0-org7
$ mkdir sdk-peer0-org8
$ vi invoke.js

$ vi prepare_sdk_peer0_org1.sh
$ vi prepare_sdk_peer0_org2.sh
$ vi prepare_sdk_peer0_org3.sh
$ vi prepare_sdk_peer0_org4.sh
$ vi prepare_sdk_peer0_org5.sh
$ vi prepare_sdk_peer0_org6.sh

```
$ vi prepare_sdk_peer0_org7.sh
$ vi prepare_sdk_peer0_org8.sh

$ ./all.sh
```

Step 2 (Prepare): -

Step 3 (Run): Cli prepares wallets of User1@org1.example.com, User1@org2.example.com, …, and User1@org5.example.com,

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ python prepare_wallet_user1_org1.py
$ python prepare_wallet_user1_org2.py
$ python prepare_wallet_user1_org3.py
$ python prepare_wallet_user1_org4.py
$ python prepare_wallet_user1_org5.py
$ python prepare_wallet_user1_org6.py
$ python prepare_wallet_user1_org7.py
$ python prepare_wallet_user1_org8.py

$ ./all.sh
```

Step 4 (Run): CLI invokes and queries transactions via Node.js,

```
$ cd $HOME/fabric-samples/demo-first/workload
$ ./scp.sh // CLI copies files to all machines

$ ./ssh.sh > result/benchlog
$ ls result -l
```

```
root@iZ7xviqssiy2jdsfcldpbvZ:~# cd $HOME/fabric-samples/demo-first/workload/sdk-peer0-org1
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first/workload/sdk-peer0-org1# nodejs invoke.js 60 100 10 1
Wallet path: /root/fabric-samples/demo-first/workload/wallet
Send_Endorsing_Proposal: 1776155562.442
Send_Endorsing_Proposal: 1776155562.447
Send_Endorsing_Proposal: 1776155562.45
Send_Endorsing_Proposal: 1776155562.452
Send_Endorsing_Proposal: 1776155562.455
Send_Endorsing_Proposal: 1776155562.458
Send_Endorsing_Proposal: 1776155562.46
Send_Endorsing_Proposal: 1776155562.462
Send_Endorsing_Proposal: 1776155562.464
Send_Endorsing_Proposal: 1776155562.468
Send_Endorsing_Proposal: 1776155562.471
Send_Endorsing_Proposal: 1776155562.474
Send_Endorsing_Proposal: 1776155562.476
Send_Endorsing_Proposal: 1776155562.478
Send_Endorsing_Proposal: 1776155562.48
Send_Endorsing_Proposal: 1776155562.482
Send_Endorsing_Proposal: 1776155562.484
Send_Endorsing_Proposal: 1776155562.487
Send_Endorsing_Proposal: 1776155562.488
Send_Endorsing_Proposal: 1776155562.49
Send_Endorsing_Proposal: 1776155562.492
Send_Endorsing_Proposal: 1776155562.494
Send_Endorsing_Proposal: 1776155562.495
Send_Endorsing_Proposal: 1776155562.497
Send_Endorsing_Proposal: 1776155562.499
Send_Endorsing_Proposal: 1776155562.502
Send_Endorsing_Proposal: 1776155562.504
Send_Endorsing_Proposal: 1776155562.507
```

```
// $ node query.js
// $ node invoke.js 30 500 20 1
```

Step 5 (Run): Analyze the log,

// CLI analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ python **readthroughput_p1.py** // rate of sending endorsed txs

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/result# python readthroughput_p1.py
Avg_Send_Rate_Phrase_1:
5.7471256178
```

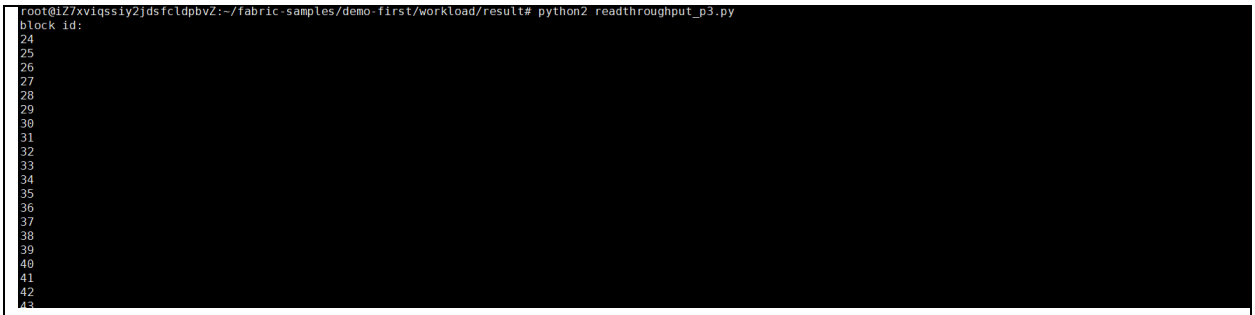
$ sudo apt-get install python-numpy
$ python readdelay_p1.py // delay of endorsing phase

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/result# python readdelay_p1.py
Mean_Delay_Endorse:
0.0188799798489
Mean_Delay_Orderer:
0.00146238137317
Mean_Delay_Commit:
0.178739959002
CDF with the probability of 50% 0.00100016593933
CDF with the probability of 90% 0.00200009346008
CDF with the probability of 95% 0.00300002098083
```

// Orderer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& ordererlog
$ **docker logs $(docker ps -a | grep 'orderer'| awk '{print $1 }') >& ordererlog**
// $ scp ordererlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python readthroughput_p2.py // rate of generating blocks

```
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first/workload/result# python readthroughput_p2.py
Block_ID:
lastPersistedBlock=[24]

lastPersistedBlock=[25]

lastPersistedBlock=[26]

lastPersistedBlock=[27]

lastPersistedBlock=[28]

lastPersistedBlock=[29]

lastPersistedBlock=[30]

lastPersistedBlock=[31]

lastPersistedBlock=[32]
```

// Peer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& peerlog
$ **docker logs $(docker ps -a | grep 'peer node start'| awk '{print $1 }') >& peerlog**
// $ scp peerlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python2 readthroughput_p3.py



// Get block size
peer channel fetch 228 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem

Step 6 (Run): All nodes run the clean procedure,

```
$ cd $HOME/fabric-samples/demo-first
$ ./clean.sh
```

5. Dynamic Adj on Single Orderer (Exp 5: 1 CLI + 2 Peers + 1 Orderer)

5.1. Prepare the Hosts

Step 1 (Prepare): All machines prepare the host names,

```
$ vi /etc/hosts
```

```
172.31.210.67 cli
172.31.210.70 peer0.org1.example.com
172.31.210.71 peer0.org2.example.com
172.31.210.72 orderer.example.com
```

Step 2 (Prepare): CLI prepares the Test-Network files,

```
$ cd $HOME/fabric-samples
$ mkdir demo-first
$ cd $HOME/fabric-samples/demo-first
$ vi crypto-config.yaml
$ vi configtx.yaml

$ vi peer0-org1.yaml
$ vi peer0-org2.yaml

$ vi orderer1.yaml

$ vi cli.yaml // (of peer0.org1)

$ vi $HOME/fabric-samples/chaincode/abstore/go/abstore.go // our proposed key-value chaincode
$ vi $HOME/fabric-samples/chaincode/abstore/go/go.mod
```

Step 2 (Prepare): All machines prepare the timestamp alignment,

```
$ date +%s // check the timestamp

$ sudo apt-get install ntpdate // align the timestamp
$ sudo ntpdate cn.pool.ntp.org
$ sudo hwclock –systohc

$ date +%s // check the timestamp again
```

Step 3 (Run): CLI runs the initial block generation,

```
$ cd $HOME/fabric-samples/demo-first
$ chmod +x *.sh
$ ./config.sh
```

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# ./config.sh
org1.example.com
org2.example.com
2026-04-14 16:21:05.926 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 002 orderer type: etcdraft
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 003 Orderer.EtcdRaft.Options unset, setting to tick_interval:"500ms" election_
ck:10 heartbeat_tick:1 max_inflight_blocks:5 snapshot_interval_size:16777216
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] Load -> INFO 004 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 005 Generating genesis block
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 006 Writing genesis block
2026-04-14 16:21:05.957 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.973 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.973 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 003 Generating new channel configtx
2026-04-14 16:21:05.974 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 004 Writing new channel tx
2026-04-14 16:21:05.988 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.003 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.003 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.004 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
2026-04-14 16:21:06.018 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.034 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
```

```
$ cd $HOME/fabric-samples/demo-first/workload/tls
$ chmod +x *.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ chmod +x *.sh
$ wget https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/node_modules.zip
$ unzip node_modules.zip

$ cd $HOME/fabric-samples/demo-first/scripts
$ chmod +x *.sh
```

Step 4 (Run): CLI scp the contents to all machines including peers and orderers,

```
$ cd $HOME/fabric-samples/demo-first
$ ./scp.sh
```

```
priv_sk                          100%  241     0.2KB/s   00:0
tlsca.example.com-cert.pem       100%  847     0.8KB/s   00:0
orderer.example.com-cert.pem     100%  769     0.8KB/s   00:0
tlsca.example.com-cert.pem       100%  847     0.8KB/s   00:0
Admin@example.com-cert.pem       100%  769     0.8KB/s   00:0
ca.example.com-cert.pem          100%  839     0.8KB/s   00:0
priv_sk                          100%  241     0.2KB/s   00:0
ca.crt                           100%  847     0.8KB/s   00:0
server.key                       100%  241     0.2KB/s   00:0
server.crt                       100%  916     0.9KB/s   00:0
Admin@example.com-cert.pem       100%  769     0.8KB/s   00:0
tlsca.example.com-cert.pem       100%  847     0.8KB/s   00:0
Admin@example.com-cert.pem       100%  769     0.8KB/s   00:0
ca.example.com-cert.pem          100%  839     0.8KB/s   00:0
priv_sk                          100%  241     0.2KB/s   00:0
ca.crt                           100%  847     0.8KB/s   00:0
client.crt                       100%  814     0.8KB/s   00:0
client.key                       100%  241     0.2KB/s   00:0
priv_sk                          100%  241     0.2KB/s   00:0
ca.example.com-cert.pem          100%  839     0.8KB/s   00:0
```

Step 5 (Run): Orderer1 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./orderer1.sh
```

```
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first# ./orderer1.sh
Creating volume "net_orderer.example.com" with default driver
Creating orderer.example.com ... done
```

$ docker ps

```
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                             COMMAND             CREATED             STATUS              PORTS                    NAMES
4a207f175833        hyperledger/fabric-orderer:2.2.0   "orderer"           8 seconds ago       Up 8 seconds        0.0.0.0:7050->7050/tcp   orderer.example.com
```

Step 6 (Run): Peer1 sets up the Docker container,

$ cd $HOME/fabric-samples/demo-first
$ ./**peer1.sh**

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first# ./peer1.sh
Creating volume "net_peer0.org1.example.com" with default driver
Creating peer0.org1.example.com ... done
```

$ docker ps

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                      COMMAND             CREATED             STATUS              PORTS                    NAMES
38fb1129b2ae        hyperledger/fabric-peer:2.2.0   "peer node start"   2 seconds ago       Up 1 second         0.0.0.0:7051->7051/tcp   peer0.org1.example.com
```

Step 7 (Run): Peer2 sets up the Docker container,

$ cd $HOME/fabric-samples/demo-first
$ ./**peer2.sh**

```
root@iZ7xviqssiy2jdsfcldpc1Z:~/fabric-samples/demo-first# ./peer2.sh
Creating volume "net_peer0.org2.example.com" with default driver
Creating peer0.org2.example.com ... done
```

$ docker ps

```
root@iZ7xviqssiy2jdsfcldpc1Z:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                      COMMAND             CREATED             STATUS              PORTS                                        NAMES
e88f655db1d9        hyperledger/fabric-peer:2.2.0   "peer node start"   2 seconds ago       Up 1 second         7051/tcp, 0.0.0.0:8051->8051/tcp   peer0.org2.example.com
```

Step 8 (Run): CLI sets up the Docker container,

$ cd $HOME/fabric-samples/demo-first
$ ./**cli.sh**

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# ./cli.sh
Creating cli ... done
```

$ docker ps

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                       COMMAND             CREATED             STATUS              PORTS               NAMES
30e6fbad4c51        hyperledger/fabric-tools:2.2.0   "/bin/bash"         2 seconds ago       Up 1 second                             cli
```

5.2. Start the Network

Step 1 (Run): CLI connects to Peer0.org1.example.com:7051, creates a channel.block, and then all peers join the channel.block,

// CLI goes to `peer0.org1.example.com:7051`
$ docker exec -it cli bash

// Environment Configuration for `peer0.org1.example.com:7051`
cd scripts
./**step1.sh** // All peers join the channel

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                          COMMAND             CREATED             STATUS              PORTS               NAMES
30e6fbad4c51        hyperledger/fabric-tools:2.2.0   "/bin/bash"         2 seconds ago       Up 1 second                             cli
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# docker exec -it cli bash
bash-5.0# cd scripts
bash-5.0# ./step1.sh
2026-04-14 08:26:37.630 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:37.647 UTC [cli.common] readBlock -> INFO 002 Expect block, but got status: &{NOT_FOUND}
2026-04-14 08:26:37.649 UTC [channelCmd] InitCmdFactory -> INFO 003 Endorser and orderer connections initialized
2026-04-14 08:26:37.850 UTC [cli.common] readBlock -> INFO 004 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:37.852 UTC [channelCmd] InitCmdFactory -> INFO 005 Endorser and orderer connections initialized
2026-04-14 08:26:38.053 UTC [cli.common] readBlock -> INFO 006 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.054 UTC [channelCmd] InitCmdFactory -> INFO 007 Endorser and orderer connections initialized
2026-04-14 08:26:38.255 UTC [cli.common] readBlock -> INFO 008 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.257 UTC [channelCmd] InitCmdFactory -> INFO 009 Endorser and orderer connections initialized
2026-04-14 08:26:38.457 UTC [cli.common] readBlock -> INFO 00a Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.459 UTC [channelCmd] InitCmdFactory -> INFO 00b Endorser and orderer connections initialized
2026-04-14 08:26:38.659 UTC [cli.common] readBlock -> INFO 00c Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.661 UTC [channelCmd] InitCmdFactory -> INFO 00d Endorser and orderer connections initialized
2026-04-14 08:26:38.864 UTC [cli.common] readBlock -> INFO 00e Received block: 0
2026-04-14 08:26:38.897 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:38.921 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-14 08:26:38.953 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:38.977 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-14 08:26:39.004 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:39.015 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
2026-04-14 08:26:39.040 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:39.050 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0#
```

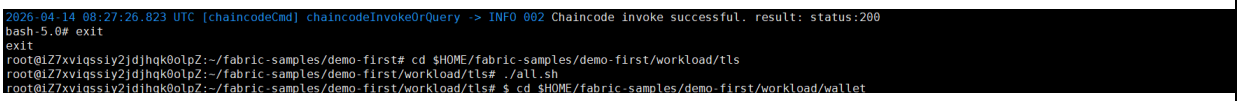
./**step2.sh** // All peers join the chaincode

```
11ac10e3978491cdaab7d56219165c0b45\022\006mycc_1" >
2026-04-14 08:27:14.093 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 002 Chaincode code package identifier: mycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cda
b7d56219165c0b45
2026-04-14 08:27:20.092 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 001 Installed remotely: response:<status:200 payload:"\nGmycc_1:9c2d699dc0bb37ea77cfbadd7567(
11ac10e3978491cdaab7d56219165c0b45\022\006mycc_1" >
2026-04-14 08:27:20.093 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 002 Chaincode code package identifier: mycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cda
b7d56219165c0b45
2026-04-14 08:27:20.160 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychannel) orderer endpoint: orderer.example.com:7050
2026-04-14 08:27:21.182 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [2e49555a3a8dd90810de8e5e90973f90a615a62711c3be05cc97ddf7b1cc0a23] committed with status (VALID) at
2026-04-14 08:27:21.218 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychannel) orderer endpoint: orderer.example.com:7050
2026-04-14 08:27:22.238 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [a720bb2d81a5e1dc2f56ab369b6df082355423c38337e372ffc9f80a515d6f26] committed with status (VALID) at
{
        "approvals": {
                "Org1MSP": true,
                "Org2MSP": true
        }
}
2026-04-14 08:27:23.402 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [f8eba34bdcfcd726a6959a47dc60c95e05cf7652a8b7f4cf80c7b44f70bd236b] committed with status (VALID) at peer0
rg1.example.com:7051
2026-04-14 08:27:23.402 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [f8eba34bdcfcd726a6959a47dc60c95e05cf7652a8b7f4cf80c7b44f70bd236b] committed with status (VALID) at peer0
rg2.example.com:8051
2026-04-14 08:27:24.693 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [862b6b71fe7bdc81df4b23af39e63d1e1017b2c3b39e34b6cdf6ac6213bb905c] committed with status (VALID) at peer0
rg1.example.com:7051
2026-04-14 08:27:24.693 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
2026-04-14 08:27:25.741 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [1722e9e0ffb97b289c38f53946f397b242b6d3ff591f2ba4f48ae6ed239311f3] committed with status (VALID) at peer0
rg1.example.com:7051
2026-04-14 08:27:25.742 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
200tps
2026-04-14 08:27:26.823 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [64c2861c9c16c9c93e549c87eab9181e87ff23827fb1207ae4c213c802b92110] committed with status (VALID) at peer0
rg2.example.com:8051
2026-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
bash-5.0#
```

5.3. Start Nodejs SDK

Step 1 (Run): CLI configures the Node.js SDK folder and tls, wallet, and workload files for all peers,

```
# exit
$ mkdir -p $HOME/fabric-samples/demo-first/workload
$ mkdir -p $HOME/fabric-samples/demo-first/workload/tls
$ mkdir -p $HOME/fabric-samples/demo-first/workload/wallet
$ mkdir -p $HOME/fabric-samples/demo-first/workload/result

$ cd $HOME/fabric-samples/demo-first/workload/tls
$ vi read_pem.py
$ vi prepare_tls_peer0_org1.sh
$ vi prepare_tls_peer0_org2.sh
$ ./all.sh
```




```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ vi prepare_wallet_user1_org1.py
$ vi prepare_wallet_user1_org2.py
$ ./all.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ mkdir sdk-peer0-org1
$ mkdir sdk-peer0-org2
$ vi invoke.js
$ vi prepare_sdk_peer0_org1.sh
$ vi prepare_sdk_peer0_org2.sh
$ ./all.sh
```

Step 2 (Prepare): CLI prepares the Nodejs SDK dependencies on CLI. Note that you may pass this step by copying the compiled nodejs source code https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/node_modules.zip,

```
$ cd $HOME/fabric-samples/demo-first/workload
$ vi package.json
$ npm install
$ vi $HOME/fabric-samples/demo-first/workload/node_modules/fabric-network/lib/transaction.js
```

Step 3 (Run): Cli prepares wallets of User1@org1.example.com and User1@org2.example.com,

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ rm -rf *.id
$ python prepare_wallet_user1_org1.py
```

$ python prepare_wallet_user1_org2.py
$ ./**all.sh**

Step 4 (Run): CLI invokes and queries transactions via Node.js,

$ cd $HOME/fabric-samples/demo-first/workload
$ ./**scp.sh** // CLI copies files to all machines

$ ./**ssh.sh** **> result/benchlog**
$ ls result -l

```
root@iZ7xviqssiy2jdsfcldpbvZ:~# cd $HOME/fabric-samples/demo-first/workload/sdk-peer0-org1
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first/workload/sdk-peer0-org1# nodejs invoke.js 60 100 10 1
Wallet path: /root/fabric-samples/demo-first/workload/wallet
Send_Endorsing_Proposal: 1776155562.442
Send_Endorsing_Proposal: 1776155562.447
Send_Endorsing_Proposal: 1776155562.45
Send_Endorsing_Proposal: 1776155562.452
Send_Endorsing_Proposal: 1776155562.455
Send_Endorsing_Proposal: 1776155562.458
Send_Endorsing_Proposal: 1776155562.46
Send_Endorsing_Proposal: 1776155562.462
Send_Endorsing_Proposal: 1776155562.464
Send_Endorsing_Proposal: 1776155562.468
Send_Endorsing_Proposal: 1776155562.471
Send_Endorsing_Proposal: 1776155562.474
Send_Endorsing_Proposal: 1776155562.476
Send_Endorsing_Proposal: 1776155562.478
Send_Endorsing_Proposal: 1776155562.48
Send_Endorsing_Proposal: 1776155562.482
Send_Endorsing_Proposal: 1776155562.484
Send_Endorsing_Proposal: 1776155562.487
Send_Endorsing_Proposal: 1776155562.488
Send_Endorsing_Proposal: 1776155562.49
Send_Endorsing_Proposal: 1776155562.492
Send_Endorsing_Proposal: 1776155562.494
Send_Endorsing_Proposal: 1776155562.495
Send_Endorsing_Proposal: 1776155562.497
Send_Endorsing_Proposal: 1776155562.499
Send_Endorsing_Proposal: 1776155562.502
Send_Endorsing_Proposal: 1776155562.504
Send_Endorsing_Proposal: 1776155562.507
```

// $ node query.js
// $ node invoke.js 30 500 20 1

Step 5 (Run): Analyze the log,

// CLI analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ python **readthroughput_p1.py** // rate of sending endorsed txs

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/result# python readthroughput_p1.py
Avg_Send_Rate_Phrase_1:
5.7471256178
```

$ sudo apt-get install python-numpy
$ python readdelay_p1.py // delay of endorsing phase

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/result# python readdelay_p1.py
Mean_Delay_Endorse:
0.0188799798489
Mean_Delay_Orderer:
0.00146238137317
Mean_Delay_Commit:
0.178739959002
CDF with the probability of 50% 0.00100016593933
CDF with the probability of 90% 0.00200009346008
CDF with the probability of 95% 0.00300002098083
```

// Orderer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& ordererlog
$ **docker logs $(docker ps -a | grep 'orderer'| awk '{print $1 }') >& ordererlog**
// $ scp ordererlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python readthroughput_p2.py // rate of generating blocks

```
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first/workload/result# python readthroughput_p2.py
Block_ID:
lastPersistedBlock=[24]

lastPersistedBlock=[25]

lastPersistedBlock=[26]

lastPersistedBlock=[27]

lastPersistedBlock=[28]

lastPersistedBlock=[29]

lastPersistedBlock=[30]

lastPersistedBlock=[31]

lastPersistedBlock=[32]
```

// Peer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& peerlog
$ **docker logs $(docker ps -a | grep 'peer node start'| awk '{print $1 }') >& peerlog**
// $ scp peerlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python2 readthroughput_p3.py

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first/workload/result# python2 readthroughput_p3.py
block id:
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
```

// For example, we can get block size
peer channel fetch 2 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem

```
bash-5.0# peer channel fetch 2 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
2026-05-07 08:15:21.880 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-07 08:15:21.882 UTC [cli.common] readBlock -> INFO 002 Received block: 2
```

5.4. Dynamic Adjust Block Parameters on Single OSN

Step 1 (Run): Orderer fetches the latest block configuration, where orderer pulls the channel configuration in protobuf format and create a file named config_block.pb, as follows,

```
$ docker exec -it cli bash
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp
CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload# docker exec -it cli bash
bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp CORE_PEER_ADDRES
=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/e
mple.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel --tls
-cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pe
2026-05-09 02:53:08.455 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-09 02:53:08.457 UTC [cli.common] readBlock -> INFO 002 Received block: 13
2026-05-09 02:53:08.457 UTC [channelCmd] fetch -> INFO 003 Retrieving last config block: 2
2026-05-09 02:53:08.458 UTC [cli.common] readBlock -> INFO 004 Received block: 2
bash-5.0# ls
channel-artifacts  config_block.pb  crypto  scripts
```

Step 2 (Run): We convert the configuration file named config_block.pb to JSON file named config.json, as follows,

```
# configtxlator proto_decode --input config_block.pb --type common.Block |
jq .data.data[0].payload.data.config > config.json
```

```
bash-5.0# configtxlator proto_decode --input config_block.pb --type common.Block | jq .data.data[0].payload.data.config > config.json
bash-5.0# ls
channel-artifacts  config.json  config_block.pb  crypto  scripts
```

We can check that the initial BatchSize is 60,

```
    },
    "values": {
      "BatchSize": {
        "mod_policy": "Admins",
        "value": {
          "absolute_max_bytes": 103809024,
          "max_message_count": 60,
          "preferred_max_bytes": 524288
        },
        "version": "0"
```

Step 3 (Run): We update the BatchSize value from 60 to 80, and save the modified json file as modified_config.json, as follows,

```
# BS=".channel_group.groups.Orderer.values.BatchSize.value.max_message_count"
# jq "$BS" config.json
```

```
bash-5.0# BS=".channel_group.groups.Orderer.values.BatchSize.value.max_message_count"
bash-5.0# jq "$BS" config.json
60
```

```
# jq "$BS = 80" config.json > modified_config.json
# jq "$BS" modified_config.json
```

```
bash-5.0# jq "$BS = 80" config.json > modified_config.json
bash-5.0# jq "$BS" modified_config.json
80
```

Step 4 (Run): We convert the initial config.json to config.pb and convert the modified_config.json to modified_config.pb, as follows,

```
# configtxlator proto_encode --input config.json --type common.Config --output config.pb
# configtxlator proto_encode --input modified_config.json --type common.Config --output
modified_config.pb
```

```
bash-5.0# configtxlator proto_encode --input config.json --type common.Config --output config.pb
bash-5.0# configtxlator proto_encode --input modified_config.json --type common.Config --output modified_config.pb
bash-5.0# ls
channel-artifacts     config_block.pb       modified_config.pb
config.json           crypto                scripts
config.pb             modified_config.json
bash-5.0#
```

Step 5 (Run): We calculate the difference between config.pb and modified_config.pb. The difference to be appended is named file_update.pb, as follows,

```
# configtxlator compute_update --channel_id mychannel --original config.pb --updated
modified_config.pb --output final_update.pb
```

```
bash-5.0# configtxlator compute_update --channel_id mychannel --original config.pb --updated modified_config.pb --output final_update.pb
bash-5.0# ls
channel-artifacts     config.pb             crypto                modified_config.json  scripts
config.json           config_block.pb       final_update.pb       modified_config.pb
bash-5.0#
```

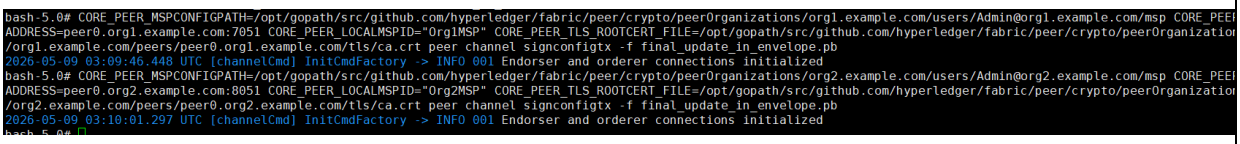
Step 6 (Run): We update the difference to the envelope, and append the changes to the config, as follows,

```
# configtxlator proto_decode --input final_update.pb --type common.ConfigUpdate | jq . >
final_update.json
# echo "{\"payload\":{\"header\":{\"channel_header\":{\"channel_id\":\"mychannel\",
\"type\":2}},\"data\":{\"config_update\":"$(cat final_update.json)"}}}" | jq . >
header_in_envolope.json
# configtxlator proto_encode --input header_in_envolope.json --type common.Envelope --output
final_update_in_envelope.pb
```

```
bash-5.0# configtxlator proto_decode --input final_update.pb --type common.ConfigUpdate | jq . > final_update.json
bash-5.0# echo "{\"payload\":{\"header\":{\"channel_header\":{\"channel_id\":\"mychannel\", \"type\":2}},\"data\":{\"config_update\":"$(cat final_update.json)"}}}" | jq
 >  header_in_envolope.json
bash-5.0# configtxlator proto_encode --input header_in_envolope.json --type common.Envelope --output final_update_in_envelope.pb
bash-5.0# ls
channel-artifacts           config_block.pb             final_update.pb             modified_config.json
config.json                 crypto                      final_update_in_envelope.pb  modified_config.pb
config.pb                   final_update.json           header_in_envolope.json     scripts
bash-5.0#
```

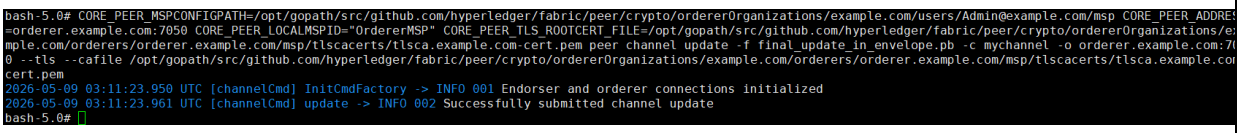
Step 7 (Run): All endorsing peers sign this update proto, as follows,

```
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peer
Organizations/org1.example.com/users/Admin@org1.example.com/msp
CORE_PEER_ADDRESS=peer0.org1.example.com:7051 CORE_PEER_LOCALMSPID="Org1MSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/pe
erOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt peer channel
signconfigtx -f final_update_in_envelope.pb

#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peer
Organizations/org2.example.com/users/Admin@org2.example.com/msp
CORE_PEER_ADDRESS=peer0.org2.example.com:8051 CORE_PEER_LOCALMSPID="Org2MSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/pe
erOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt peer channel
signconfigtx -f final_update_in_envelope.pb
```



Step 7 (Run): Orderer update the channel as follows,

```
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/orde
rerOrganizations/example.com/users/Admin@example.com/msp
CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/or
dererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.co
m-cert.pem peer channel update -f final_update_in_envelope.pb -c mychannel -o
orderer.example.com:7050 --tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/
orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```



Step 8 (Validation): We can validate that the BatchSize is now from 60 to 80, as follows,

```
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/orde
rerOrganizations/example.com/users/Admin@example.com/msp
```

CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem

 # configtxlator proto_decode --input config_block.pb --type common.Block | jq .data.data[0].payload.data.config > config.json
 # jq "$BS" config.json

```
bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp CORE_PEER_ADDRE
=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/e
mple.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel --tls
-cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pe
2026-05-09 03:13:24.871 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-09 03:13:24.873 UTC [cli.common] readBlock -> INFO 002 Received block: 14
2026-05-09 03:13:24.873 UTC [channelCmd] fetch -> INFO 003 Retrieving last config block: 14
2026-05-09 03:13:24.874 UTC [cli.common] readBlock -> INFO 004 Received block: 14
bash-5.0# configtxlator proto_decode --input config_block.pb --type common.Block | jq .data.data[0].payload.data.config > config.json
bash-5.0# jq "$BS" config.json
80
bash-5.0#
```

Step 9 (Run): Nodes run the clean procedure,

```
$ cd $HOME/fabric-samples/demo-first
$ ./clean.sh
```

6. Dynamic Adj on Multi Orderers (Exp 6: 1 CLI + 2 Peers + 5 Orderers)

6.1. Prepare the Hosts

Step 1 (Prepare): All machines prepare the host names,

```
$ vi /etc/hosts
```

```
172.31.210.67 cli
172.31.210.70 peer0.org1.example.com
172.31.210.71 peer0.org2.example.com
172.31.210.72 orderer.example.com
172.31.210.73 orderer2.example.com
172.31.210.74 orderer3.example.com
172.31.210.75 orderer4.example.com
172.31.210.76 orderer5.example.com
```

Step 2 (Prepare): CLI prepares the Test-Network files,

```
$ cd $HOME/fabric-samples
$ mkdir demo-first
$ cd $HOME/fabric-samples/demo-first
$ vi crypto-config.yaml
$ vi configtx.yaml

$ vi peer0-org1.yaml
$ vi peer0-org2.yaml

$ vi orderer1.yaml
$ vi orderer2.yaml
$ vi orderer3.yaml
$ vi orderer4.yaml
$ vi orderer5.yaml

$ vi cli.yaml
```

Step 2 (Prepare): All machines prepare the timestamp alignment,

```
$ date +%s // check the timestamp

$ sudo apt-get install ntpdate // align the timestamp
$ sudo ntpdate cn.pool.ntp.org
$ sudo hwclock –systohc

$ date +%s // check the timestamp again
```

Step 3 (Run): CLI runs the initial block generation,

$ cd $HOME/fabric-samples/demo-first
$ chmod +x *.sh
$ ./**config.sh**

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# ./config.sh
org1.example.com
org2.example.com
2026-04-20 10:50:10.813 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.832 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 002 orderer type: etcdraft
2026-04-20 10:50:10.832 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 003 Orderer.EtcdRaft.Options unset, setting to tick_interval:"500ms" e
ction_tick:10 heartbeat_tick:1 max_inflight_blocks:5 snapshot_interval_size:16777216
2026-04-20 10:50:10.832 CST [common.tools.configtxgen.localconfig] Load -> INFO 004 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.833 CST [common.tools.configtxgen] doOutputBlock -> INFO 005 Generating genesis block
2026-04-20 10:50:10.834 CST [common.tools.configtxgen] doOutputBlock -> INFO 006 Writing genesis block
2026-04-20 10:50:10.847 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.867 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.867 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 003 Generating new channel configtx
2026-04-20 10:50:10.868 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 004 Writing new channel tx
2026-04-20 10:50:10.882 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.902 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.902 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-20 10:50:10.902 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
2026-04-20 10:50:10.916 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-20 10:50:10.935 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-20 10:50:10.935 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-20 10:50:10.936 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
```

Step 4 (Run): CLI scp the contents to all machines including peers and orderers,

$ cd $HOME/fabric-samples/demo-first
$ ./**scp.sh**

```
priv_sk                          100%  241   0.2KB/s   00:0
tlsca.example.com-cert.pem       100%  847   0.8KB/s   00:0
orderer.example.com-cert.pem     100%  769   0.8KB/s   00:0
tlsca.example.com-cert.pem       100%  847   0.8KB/s   00:0
Admin@example.com-cert.pem       100%  769   0.8KB/s   00:0
ca.example.com-cert.pem          100%  839   0.8KB/s   00:0
priv_sk                          100%  241   0.2KB/s   00:0
ca.crt                           100%  847   0.8KB/s   00:0
server.key                       100%  241   0.2KB/s   00:0
server.crt                       100%  916   0.9KB/s   00:0
Admin@example.com-cert.pem       100%  769   0.8KB/s   00:0
tlsca.example.com-cert.pem       100%  847   0.8KB/s   00:0
Admin@example.com-cert.pem       100%  769   0.8KB/s   00:0
ca.example.com-cert.pem          100%  839   0.8KB/s   00:0
priv_sk                          100%  241   0.2KB/s   00:0
ca.crt                           100%  847   0.8KB/s   00:0
client.crt                       100%  814   0.8KB/s   00:0
client.key                       100%  241   0.2KB/s   00:0
priv_sk                          100%  241   0.2KB/s   00:0
ca.example.com-cert.pem          100%  839   0.8KB/s   00:0
```

Step 5 (Run): Orderers sets up the Docker container,

$ cd $HOME/fabric-samples/demo-first
$ ./**orderer1.sh**

```
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first# ./orderer1.sh
Creating volume "net_orderer.example.com" with default driver
Creating orderer.example.com ... done
```

$ ./**orderer2.sh**

```
root@iZ7xviqssiy2jdsfcldpbwZ:~/fabric-samples/demo-first# ./orderer2.sh
Creating network "net_byfn" with the default driver
Creating volume "net_orderer2.example.com" with default driver
Creating orderer2.example.com ... done
```

$ ./**orderer3.sh**

```
root@iZ7xviqssiy2jdsfcldpbzZ:~/fabric-samples/demo-first# ./orderer3.sh
Creating network "net_byfn" with the default driver
Creating volume "net_orderer3.example.com" with default driver
Creating orderer3.example.com ... done
```

$ ./**orderer4.sh**

```
root@iZ7xviqssiy2jdsfcldpc3Z:~/fabric-samples/demo-first# ./orderer4.sh
Creating network "net_byfn" with the default driver
Creating volume "net_orderer4.example.com" with default driver
Creating orderer4.example.com ... done
```

$ ./**orderer5.sh**

```
root@iZ7xviqssiy2jdsfcldpc0Z:~/fabric-samples/demo-first# ./orderer5.sh
Creating network "net_byfn" with the default driver
Creating volume "net_orderer5.example.com" with default driver
Creating orderer5.example.com ... done
```

$ docker ps

Step 6 (Run): Peer1 sets up the Docker container,

$ cd $HOME/fabric-samples/demo-first
$ ./**peer1.sh**

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first# ./peer1.sh
Creating volume "net_peer0.org1.example.com" with default driver
Creating peer0.org1.example.com ... done
```

$ docker ps

Step 7 (Run): Peer2 sets up the Docker container,

$ cd $HOME/fabric-samples/demo-first
$ ./**peer2.sh**

```
root@iZ7xviqssiy2jdsfcldpc1Z:~/fabric-samples/demo-first# ./peer2.sh
Creating volume "net_peer0.org2.example.com" with default driver
Creating peer0.org2.example.com ... done
```

$ docker ps

Step 8 (Run): CLI sets up the Docker container,

$ cd $HOME/fabric-samples/demo-first
$ ./**cli.sh**

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# ./cli.sh
Creating cli ... done
```

$ docker ps

6.2. Start the Network

Step 1 (Run): CLI connects to Peer0.org1.example.com:7051, creates a channel.block, and then all peers join the channel.block,

// CLI goes to `peer0.org1.example.com:7051`
$ **docker exec -it cli bash**

// Environment Configuration for `peer0.org1.example.com:7051`
cd scripts
./**step1.sh** // All peers join the channel

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# docker exec -it cli bash
bash-5.0# ls
channel-artifacts  crypto            scripts
bash-5.0# cd scripts
bash-5.0# ls
mychannel.block  step1.sh        step2.sh
bash-5.0# ./step1.sh
2026-04-20 03:02:19.759 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:19.776 UTC [cli.common] readBlock -> INFO 002 Expect block, but got status: &{NOT_FOUND}
2026-04-20 03:02:19.778 UTC [channelCmd] InitCmdFactory -> INFO 003 Endorser and orderer connections initialized
2026-04-20 03:02:19.979 UTC [cli.common] readBlock -> INFO 004 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-20 03:02:19.980 UTC [channelCmd] InitCmdFactory -> INFO 005 Endorser and orderer connections initialized
2026-04-20 03:02:20.181 UTC [cli.common] readBlock -> INFO 006 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-20 03:02:20.183 UTC [channelCmd] InitCmdFactory -> INFO 007 Endorser and orderer connections initialized
2026-04-20 03:02:20.383 UTC [cli.common] readBlock -> INFO 008 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-20 03:02:20.385 UTC [channelCmd] InitCmdFactory -> INFO 009 Endorser and orderer connections initialized
2026-04-20 03:02:20.586 UTC [cli.common] readBlock -> INFO 00a Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-20 03:02:20.587 UTC [channelCmd] InitCmdFactory -> INFO 00b Endorser and orderer connections initialized
2026-04-20 03:02:20.788 UTC [cli.common] readBlock -> INFO 00c Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-20 03:02:20.790 UTC [channelCmd] InitCmdFactory -> INFO 00d Endorser and orderer connections initialized
2026-04-20 03:02:20.993 UTC [cli.common] readBlock -> INFO 00e Received block: 0
2026-04-20 03:02:21.027 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.053 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-20 03:02:21.084 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.112 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-20 03:02:21.139 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.151 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
2026-04-20 03:02:21.176 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-20 03:02:21.187 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0#
```

./**step2.sh** // All peers join the chaincode

6.3. Start Nodejs SDK

Step 1 (Run): CLI configures the Node.js SDK folder and tls, wallet, and workload files for all peers,

exit
$ mkdir -p $HOME/fabric-samples/demo-first/workload
$ mkdir -p $HOME/fabric-samples/demo-first/workload/tls
$ mkdir -p $HOME/fabric-samples/demo-first/workload/wallet
$ mkdir -p $HOME/fabric-samples/demo-first/workload/result

$ cd $HOME/fabric-samples/demo-first/workload/tls
$ vi read_pem.py
$ vi prepare_tls_peer0_org1.sh
$ vi prepare_tls_peer0_org2.sh
$ ./**all.sh**

```
2026-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
bash-5.0# exit
exit
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# cd $HOME/fabric-samples/demo-first/workload/tls
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/tls# ./all.sh
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/tls# $ cd $HOME/fabric-samples/demo-first/workload/wallet
```

$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ vi prepare_wallet_user1_org1.py
$ vi prepare_wallet_user1_org2.py
$ ./**all.sh**

$ cd $HOME/fabric-samples/demo-first/workload

```
$ mkdir sdk-peer0-org1
$ mkdir sdk-peer0-org2
$ vi invoke.js
$ vi prepare_sdk_peer0_org1.sh
$ vi prepare_sdk_peer0_org2.sh
$ ./all.sh
```

Step 2 (Prepare): -

Step 3 (Run): CLI prepares wallets of User1@org1.example.com and User1@org2.example.com,

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ python prepare_wallet_user1_org1.py
$ python prepare_wallet_user1_org2.py
$ ./all.sh
```

Step 4 (Run): CLI invokes and queries transactions via Node.js,

```
$ cd $HOME/fabric-samples/demo-first/workload
$ ./scp.sh // CLI copies files to all machines

$ ./ssh.sh > result/benchlog
$ ls result -l
```

```
root@iZ7xviqssiy2jdsfcldpbvZ:~# cd $HOME/fabric-samples/demo-first/workload/sdk-peer0-org1
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first/workload/sdk-peer0-org1# nodejs invoke.js 60 100 10 1
Wallet path: /root/fabric-samples/demo-first/workload/wallet
Send_Endorsing_Proposal: 1776155562.442
Send_Endorsing_Proposal: 1776155562.447
Send_Endorsing_Proposal: 1776155562.45
Send_Endorsing_Proposal: 1776155562.452
Send_Endorsing_Proposal: 1776155562.455
Send_Endorsing_Proposal: 1776155562.458
Send_Endorsing_Proposal: 1776155562.46
Send_Endorsing_Proposal: 1776155562.462
Send_Endorsing_Proposal: 1776155562.464
Send_Endorsing_Proposal: 1776155562.468
Send_Endorsing_Proposal: 1776155562.471
Send_Endorsing_Proposal: 1776155562.474
Send_Endorsing_Proposal: 1776155562.476
Send_Endorsing_Proposal: 1776155562.478
Send_Endorsing_Proposal: 1776155562.48
Send_Endorsing_Proposal: 1776155562.482
Send_Endorsing_Proposal: 1776155562.484
Send_Endorsing_Proposal: 1776155562.487
Send_Endorsing_Proposal: 1776155562.488
Send_Endorsing_Proposal: 1776155562.49
Send_Endorsing_Proposal: 1776155562.492
Send_Endorsing_Proposal: 1776155562.494
Send_Endorsing_Proposal: 1776155562.495
Send_Endorsing_Proposal: 1776155562.497
Send_Endorsing_Proposal: 1776155562.499
Send_Endorsing_Proposal: 1776155562.502
Send_Endorsing_Proposal: 1776155562.504
Send_Endorsing_Proposal: 1776155562.507
```

```
// $ node query.js
// $ node invoke.js 30 500 20 1
```

Step 5 (Run): Analyze the log,

```
// CLI analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ python readthroughput_p1.py // rate of sending endorsed txs
```

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/result# python readthroughput_p1.py
Avg_Send_Rate_Phrase_1:
5.7471256178
```

$ sudo apt-get install python-numpy
$ python readdelay_p1.py // delay of endorsing phase

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/result# python readdelay_p1.py
Mean_Delay_Endorse:
0.0188799798489
Mean_Delay_Orderer:
0.00146238137317
Mean_Delay_Commit:
0.178739959002
CDF with the probability of 50% 0.00100016593933
CDF with the probability of 90% 0.00200009346008
CDF with the probability of 95% 0.00300002098083
```

// Orderer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& ordererlog
$ **docker logs $(docker ps -a | grep 'orderer'| awk '{print $1 }') >& ordererlog**
// $ scp ordererlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python readthroughput_p2.py // rate of generating blocks

```
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first/workload/result# python readthroughput_p2.py
Block_ID:
lastPersistedBlock=[24]

lastPersistedBlock=[25]

lastPersistedBlock=[26]

lastPersistedBlock=[27]

lastPersistedBlock=[28]

lastPersistedBlock=[29]

lastPersistedBlock=[30]

lastPersistedBlock=[31]

lastPersistedBlock=[32]
```

// Peer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& peerlog
$ **docker logs $(docker ps -a | grep 'peer node start'| awk '{print $1 }') >& peerlog**
// $ scp peerlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python2 readthroughput_p3.py

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first/workload/result# python2 readthroughput_p3.py
block id:
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
```

// Get block size
peer channel fetch 228 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem

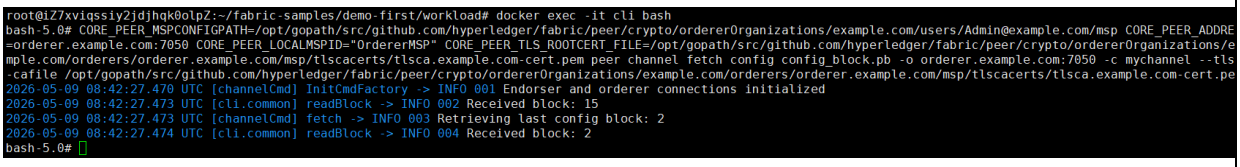
Step 6 (Run): All nodes run the clean procedure,

```
$ cd $HOME/fabric-samples/demo-first
$ ./clean.sh
```

6.4. Dynamic Adjust Block Parameters on Multiple OSNs

Step 1 (Run): Orderer 1 fetches the latest block configuration, where orderer pulls the channel configuration in protobuf format and create a file named config_block.pb, as follows,

```
$ docker exec -it cli bash
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp
CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel --tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```



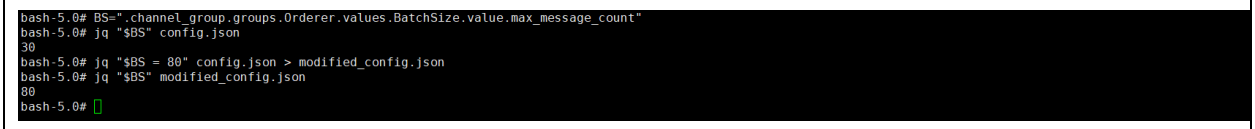
Step 2 (Run): We convert the configuration file named config_block.pb to JSON file named config.json, as follows,

```
# configtxlator proto_decode --input config_block.pb --type common.Block |
jq .data.data[0].payload.data.config > config.json
```

Step 3 (Run): We update the BatchSize value from 30 to 80, and save the modified json file as modified_config.json, as follows,

```
# BS=".channel_group.groups.Orderer.values.BatchSize.value.max_message_count"
# jq "$BS" config.json
```


```
# jq "$BS = 80" config.json > modified_config.json
# jq "$BS" modified_config.json
```



Step 4 (Run): We convert the initial config.json to config.pb and convert the modified_config.json to modified_config.pb, as follows,

```
# configtxlator proto_encode --input config.json --type common.Config --output config.pb
# configtxlator proto_encode --input modified_config.json --type common.Config --output
modified_config.pb
```

Step 5 (Run): We calculate the difference between config.pb and modified_config.pb. The difference to be appended is named file_update.pb, as follows,

```
# configtxlator compute_update --channel_id mychannel --original config.pb --updated
modified_config.pb --output final_update.pb
```

Step 6 (Run): We update the difference to the envelope, and append the changes to the config, as follows,

```
# configtxlator proto_decode --input final_update.pb --type common.ConfigUpdate | jq . >
final_update.json
# echo "{\"payload\":{\"header\":{\"channel_header\":{\"channel_id\":\"mychannel\",
\"type\":2}},\"data\":{\"config_update\":"$(cat final_update.json)"}}}" | jq . >
header_in_envolope.json
# configtxlator proto_encode --input header_in_envolope.json --type common.Envelope --output
final_update_in_envelope.pb
```

```
bash-5.0# configtxlator proto_encode --input config.json --type common.Config --output config.pb
bash-5.0# configtxlator proto_encode --input modified_config.json --type common.Config --output modified_config.pb
bash-5.0# configtxlator compute_update --channel_id mychannel --original config.pb --updated modified_config.pb --output final_update.pb
bash-5.0# configtxlator proto_decode --input final_update.pb --type common.ConfigUpdate | jq . > final_update.json
bash-5.0# echo "{\"payload\":{\"header\":{\"channel_header\":{\"channel_id\":\"mychannel\", \"type\":2}},\"data\":{\"config_update\":"$(cat final_update.json)"}}}" | jq
 >  header_in_envolope.json
bash-5.0# configtxlator proto_encode --input header_in_envolope.json --type common.Envelope --output final_update_in_envelope.pb
bash-5.0#
```

Step 7 (Run): All endorsing peers sign this update proto, as follows,

```
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peer
Organizations/org1.example.com/users/Admin@org1.example.com/msp
CORE_PEER_ADDRESS=peer0.org1.example.com:7051 CORE_PEER_LOCALMSPID="Org1MSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/pe
erOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt peer channel
signconfigtx -f final_update_in_envelope.pb

#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peer
Organizations/org2.example.com/users/Admin@org2.example.com/msp
CORE_PEER_ADDRESS=peer0.org2.example.com:8051 CORE_PEER_LOCALMSPID="Org2MSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/pe
erOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt peer channel
signconfigtx -f final_update_in_envelope.pb
```

```
bash-5.0# configtxlator proto_encode --input header_in_envolope.json --type common.Envelope --output final_update_in_envelope.pb
bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp CORE_PEE
ADDRESS=peer0.org1.example.com:7051 CORE_PEER_LOCALMSPID="Org1MSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizatio
/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt peer channel signconfigtx -f final_update_in_envelope.pb
2026-05-09 08:47:17.445 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org2.example.com/users/Admin@org2.example.com/msp CORE_PEE
ADDRESS=peer0.org2.example.com:8051 CORE_PEER_LOCALMSPID="Org2MSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizatio
/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt peer channel signconfigtx -f final_update_in_envelope.pb
2026-05-09 08:47:27.614 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
bash-5.0#
```

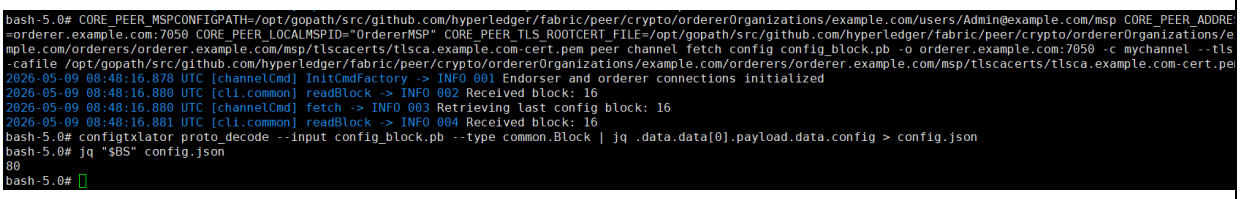
Step 7 (Run): Orderer 1 updates the channel as follows,

```
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/orde
rerOrganizations/example.com/users/Admin@example.com/msp
CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/or
dererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.co
m-cert.pem peer channel update -f final_update_in_envelope.pb -c mychannel -o
orderer.example.com:7050 --tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/
orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

```
2026-05-09 08:47:27.614 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp CORE_PEER_ADDRE
=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/e
mple.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel update -f final_update_in_envelope.pb -c mychannel -o orderer.example.com:7
0 --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.co
cert.pem
2026-05-09 08:47:51.077 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-09 08:47:51.089 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0#
```

Step 8 (Validation): We can validate that the BatchSize is now from 60 to 80, as follows,

```
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp
CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel --tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem

# configtxlator proto_decode --input config_block.pb --type common.Block |
jq .data.data[0].payload.data.config > config.json
# jq "$BS" config.json
```



Step 9 (Run): Nodes run the clean procedure,

```
$ cd $HOME/fabric-samples/demo-first
$ ./clean.sh
```

7. Dynamic Adj on Single Peer (Exp 7: 1 CLI + 1 Peer + 1 Orderer)

7.1. Prepare the Hosts

Step 1 (Prepare): All machines prepare the host names,

```
$ vi /etc/hosts
```

```
172.31.210.67 cli
172.31.210.70 peer0.org1.example.com
172.31.210.72 orderer.example.com
```

Step 2 (Prepare): CLI prepares the Test-Network files,

```
$ cd $HOME/fabric-samples
$ mkdir demo-first
$ cd $HOME/fabric-samples/demo-first
$ vi crypto-config.yaml
$ vi configtx.yaml

$ vi peer0-org1.yaml

$ vi orderer1.yaml

$ vi cli.yaml // (of peer0.org1)

$ vi $HOME/fabric-samples/chaincode/abstore/go/abstore.go // our proposed key-value chaincode
$ vi $HOME/fabric-samples/chaincode/abstore/go/go.mod
```

Step 2 (Prepare): All machines prepare the timestamp alignment,

```
$ date +%s // check the timestamp

$ sudo apt-get install ntpdate // align the timestamp
$ sudo ntpdate cn.pool.ntp.org
$ sudo hwclock –systohc

$ date +%s // check the timestamp again
```

Step 3 (Run): CLI runs the initial block generation,

```
$ cd $HOME/fabric-samples/demo-first
$ chmod +x *.sh
$ ./config.sh
```

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# ./config.sh
org1.example.com
org2.example.com
2026-04-14 16:21:05.926 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 002 orderer type: etcdraft
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 003 Orderer.EtcdRaft.Options unset, setting to tick_interval:"500ms" election_
ck:10 heartbeat_tick:1 max_inflight_blocks:5 snapshot_interval_size:16777216
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] Load -> INFO 004 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 005 Generating genesis block
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 006 Writing genesis block
2026-04-14 16:21:05.957 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.973 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.973 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 003 Generating new channel configtx
2026-04-14 16:21:05.974 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 004 Writing new channel tx
2026-04-14 16:21:05.988 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.003 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.003 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.004 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
2026-04-14 16:21:06.018 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.034 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
```

```
$ cd $HOME/fabric-samples/demo-first/workload/tls
$ chmod +x *.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ chmod +x *.sh
$ wget https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/node_modules.zip
$ unzip node_modules.zip

$ cd $HOME/fabric-samples/demo-first/scripts
$ chmod +x *.sh
```

Step 4 (Run): CLI scp the contents to all machines including peers and orderers,

```
$ cd $HOME/fabric-samples/demo-first
$ ./scp.sh
```

```
priv_sk                                 100%  241     0.2KB/s   00:0
tlsca.example.com-cert.pem              100%  847     0.8KB/s   00:0
orderer.example.com-cert.pem            100%  769     0.8KB/s   00:0
tlsca.example.com-cert.pem              100%  847     0.8KB/s   00:0
Admin@example.com-cert.pem              100%  769     0.8KB/s   00:0
ca.example.com-cert.pem                 100%  839     0.8KB/s   00:0
priv_sk                                 100%  241     0.2KB/s   00:0
ca.crt                                  100%  847     0.8KB/s   00:0
server.key                              100%  241     0.2KB/s   00:0
server.crt                              100%  916     0.9KB/s   00:0
Admin@example.com-cert.pem              100%  769     0.8KB/s   00:0
tlsca.example.com-cert.pem              100%  847     0.8KB/s   00:0
Admin@example.com-cert.pem              100%  769     0.8KB/s   00:0
ca.example.com-cert.pem                 100%  839     0.8KB/s   00:0
priv_sk                                 100%  241     0.2KB/s   00:0
ca.crt                                  100%  847     0.8KB/s   00:0
client.crt                              100%  814     0.8KB/s   00:0
client.key                              100%  241     0.2KB/s   00:0
priv_sk                                 100%  241     0.2KB/s   00:0
ca.example.com-cert.pem                 100%  839     0.8KB/s   00:0
```

Step 5 (Run): Orderer1 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./orderer1.sh
```

```
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first# ./orderer1.sh
Creating volume "net_orderer.example.com" with default driver
Creating orderer.example.com ... done
```

```
$ docker ps
```

```
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first# docker ps
CONTAINER ID   IMAGE                               COMMAND     CREATED         STATUS         PORTS                    NAMES
4a207f175833   hyperledger/fabric-orderer:2.2.0    "orderer"   8 seconds ago   Up 8 seconds   0.0.0.0:7050->7050/tcp   orderer.example.com
```

Step 6 (Run): Peer1 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./peer1.sh
```

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first# ./peer1.sh
Creating volume "net_peer0.org1.example.com" with default driver
Creating peer0.org1.example.com ... done
```

```
$ docker ps
```

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                          COMMAND             CREATED             STATUS              PORTS                    NAMES
38fb1129b2ae        hyperledger/fabric-peer:2.2.0   "peer node start"   2 seconds ago       Up 1 second         0.0.0.0:7051->7051/tcp   peer0.org1.example.com
```

Step 7 (Run): -,

Step 8 (Run): CLI sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./cli.sh
```

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# ./cli.sh
Creating cli ... done
```

```
$ docker ps
```

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                          COMMAND             CREATED             STATUS              PORTS               NAMES
30e6fbad4c51        hyperledger/fabric-tools:2.2.0   "/bin/bash"         2 seconds ago       Up 1 second                             cli
```

7.2. Start the Network

Step 1 (Run): CLI connects to Peer0.org1.example.com:7051, creates a channel.block, and then all peers join the channel.block,

```
// CLI  goes to `peer0.org1.example.com:7051`
$ docker exec -it cli bash

// Environment Configuration for `peer0.org1.example.com:7051`
# cd scripts
# ./step1.sh // All peers join the channel
```

```
mychannel.block  step1.sh         step2.sh
bash-5.0# ./step1.sh
2026-05-10 02:37:25.066 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-10 02:37:25.081 UTC [cli.common] readBlock -> INFO 002 Expect block, but got status: &{NOT_FOUND}
2026-05-10 02:37:25.083 UTC [channelCmd] InitCmdFactory -> INFO 003 Endorser and orderer connections initialized
2026-05-10 02:37:25.283 UTC [cli.common] readBlock -> INFO 004 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-05-10 02:37:25.285 UTC [channelCmd] InitCmdFactory -> INFO 005 Endorser and orderer connections initialized
2026-05-10 02:37:25.486 UTC [cli.common] readBlock -> INFO 006 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-05-10 02:37:25.487 UTC [channelCmd] InitCmdFactory -> INFO 007 Endorser and orderer connections initialized
2026-05-10 02:37:25.688 UTC [cli.common] readBlock -> INFO 008 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-05-10 02:37:25.690 UTC [channelCmd] InitCmdFactory -> INFO 009 Endorser and orderer connections initialized
2026-05-10 02:37:25.890 UTC [cli.common] readBlock -> INFO 00a Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-05-10 02:37:25.892 UTC [channelCmd] InitCmdFactory -> INFO 00b Endorser and orderer connections initialized
2026-05-10 02:37:26.092 UTC [cli.common] readBlock -> INFO 00c Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-05-10 02:37:26.093 UTC [channelCmd] InitCmdFactory -> INFO 00d Endorser and orderer connections initialized
2026-05-10 02:37:26.295 UTC [cli.common] readBlock -> INFO 00e Received block: 0
2026-05-10 02:37:26.327 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-10 02:37:26.354 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-05-10 02:37:26.380 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-10 02:37:26.390 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0#
```

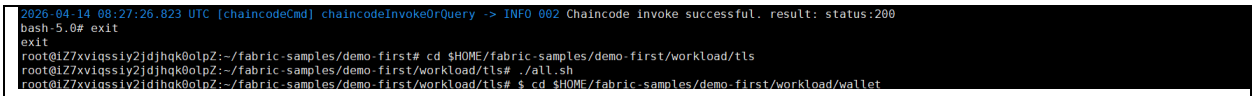
./**step2.sh** // All peers join the chaincode

```
go: downloading github.com/xeipuuv/gojsonreference v0.0.0-20180127040603-bd5ef7bd5415
go: downloading github.com/go-openapi/swag v0.19.5
go: downloading github.com/go-openapi/jsonpointer v0.19.3
go: downloading github.com/xeipuuv/gojsonpointer v0.0.0-20180127040702-4e3ac2762d5f
go: downloading github.com/mailru/easyjson v0.0.0-20190626092158-b2ccc519800e
go: downloading github.com/go-openapi/jsonreference v0.19.2
go: downloading gopkg.in/yaml.v2 v2.2.8
go: downloading google.golang.org/genproto v0.0.0-20180831171423-11092d34479b
go: downloading golang.org/x/net v0.0.0-20190827160401-ba9fcec4b297
go: downloading golang.org/x/sys v0.0.0-20190710143415-6ec70d6a5542
go: downloading github.com/PuerkitoBio/purell v1.1.1
go: downloading github.com/PuerkitoBio/urlesc v0.0.0-20170810143723-de5bf2ad4578
go: downloading golang.org/x/text v0.3.2
2026-05-10 02:38:25.043 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 001 Installed remotely: response:<status:200 payload:"\nGmycc_1:9c2d699dc0bb37ea77cfb
dd75670011ac10e3978491cdaab7d56219165c0b45\022\006mycc_1" >
2026-05-10 02:38:25.043 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 002 Chaincode code package identifier: mycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3
8491cdaab7d56219165c0b45
2026-05-10 02:38:25.111 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychannel) orderer endpoint: orderer.example.com:7050
2026-05-10 02:38:26.132 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [30bb212defe8d4d076eaba0d797ece8a84471f5ce9c8d167acba9600849715c3] committed with status (VALID)

{
        "approvals": {
                "Org1MSP": true
        }
}
2026-05-10 02:38:27.287 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [404ce595265b340173ee48b2d6ba3aa468f43f782e964586c8d104754ffbe38d] committed with status (VALID)
 peer0.org1.example.com:7051
2026-05-10 02:38:28.543 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [1b603633fccf3d5b09308053d7cd1cf540cd7e2c7c7a394c339def5beab759bf] committed with status (VALID)
 peer0.org1.example.com:7051
2026-05-10 02:38:28.543 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
2026-05-10 02:38:29.590 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [25c5d6a910d52fe7648f638c52780ba45e2ebcac3c7d6abcdb6ca6af8af2341e] committed with status (VALID)
 peer0.org1.example.com:7051
2026-05-10 02:38:29.590 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
200tps
bash-5.0#
```

7.3. Start Nodejs SDK

Step 1 (Run): CLI configures the Node.js SDK folder and tls, wallet, and workload files for all peers,

```
# exit
$ mkdir -p $HOME/fabric-samples/demo-first/workload
$ mkdir -p $HOME/fabric-samples/demo-first/workload/tls
$ mkdir -p $HOME/fabric-samples/demo-first/workload/wallet
$ mkdir -p $HOME/fabric-samples/demo-first/workload/result

$ cd $HOME/fabric-samples/demo-first/workload/tls
$ vi read_pem.py
$ vi prepare_tls_peer0_org1.sh
$ vi prepare_tls_peer0_org2.sh
$ ./all.sh
```



$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ vi prepare_wallet_user1_org1.py
$ vi prepare_wallet_user1_org2.py
$ **./all.sh**

$ cd $HOME/fabric-samples/demo-first/workload
$ mkdir sdk-peer0-org1
$ mkdir sdk-peer0-org2
$ vi invoke.js
$ vi prepare_sdk_peer0_org1.sh
$ vi prepare_sdk_peer0_org2.sh
$ ./**all.sh**

Step 2 (Prepare): CLI prepares the Nodejs SDK dependencies on CLI. Note that you may pass this step by copying the compiled nodejs source code https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/node_modules.zip,

$ cd $HOME/fabric-samples/demo-first/workload
$ vi package.json
$ npm install
$ vi $HOME/fabric-samples/demo-first/workload/node_modules/fabric-network/lib/transaction.js

Step 3 (Run): Cli prepares wallets of User1@org1.example.com and User1@org2.example.com,

$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ rm -rf *.id
$ python prepare_wallet_user1_org1.py
$ python prepare_wallet_user1_org2.py
$ ./**all.sh**

Step 4 (Run): CLI invokes and queries transactions via Node.js,

$ cd $HOME/fabric-samples/demo-first/workload
$ ./**scp.sh** // CLI copies files to all machines

$ ./**ssh.sh** **> result/benchlog**
$ ls result -l

```
Delay_Endorsing: 0.018999814987182617
Get_Endorsed_Proposal: 1778380771.862
Delay_Endorsing: 0.020999908447265625
Get_Ordered_Transaction: 1778380771.863
Delay_Ordering: 0.0009999275207519531
Get_Endorsed_Proposal: 1778380771.865
Delay_Endorsing: 0.023999929428100586
Get_Endorsed_Proposal: 1778380771.867
Delay_Endorsing: 0.026000022888183594
Get_Ordered_Transaction: 1778380771.867
Delay_Ordering: 0
Get_Ordered_Transaction: 1778380771.868
Delay_Ordering: 0.0009999275207519531
Get_Ordered_Transaction: 1778380771.868
Delay_Ordering: 0.0009999275207519531
Get_Ordered_Transaction: 1778380771.868
Delay_Ordering: 0.0009999275207519531
Get_Ordered_Transaction: 1778380771.868
Delay_Ordering: 0.0009999275207519531
Get_Ordered_Transaction: 1778380771.868
Delay_Ordering: 0.0009999275207519531
Get_Ordered_Transaction: 1778380771.868
Delay_Ordering: 0.0009999275207519531
Get_Ordered_Transaction: 1778380771.868
Delay_Ordering: 0.0009999275207519531
Send_Endorsing_Proposal: 1778380772.206
Send_Endorsing_Proposal: 1778380772.208
Send_Endorsing_Proposal: 1778380772.209
Send_Endorsing_Proposal: 1778380772.211
Send_Endorsing_Proposal: 1778380772.212
Send_Endorsing_Proposal: 1778380772.214
Send_Endorsing_Proposal: 1778380772.215
Send_Endorsing_Proposal: 1778380772.217
```

// $ node query.js
// $ node invoke.js 30 500 20 1

Step 5 (Run): Analyze the log,

// CLI analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ python **readthroughput_p1.py** // rate of sending endorsed txs

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/result# python readthroughput_p1.py
Avg_Send_Rate_Phrase_1:
5.7471256178
```

$ sudo apt-get install python-numpy
$ python readdelay_p1.py // delay of endorsing phase

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/result# python readdelay_p1.py
Mean_Delay_Endorse:
0.0188799798489
Mean_Delay_Orderer:
0.00146238137317
Mean_Delay_Commit:
0.178739959002
CDF with the probability of 50% 0.00100016593933
CDF with the probability of 90% 0.00200009346008
CDF with the probability of 95% 0.00300002098083
```

// Orderer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& ordererlog
$ **docker logs $(docker ps -a | grep 'orderer'| awk '{print $1 }') >& ordererlog**
// $ scp ordererlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python readthroughput_p2.py // rate of generating blocks

// Peer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& peerlog

$ **docker logs $(docker ps -a | grep 'peer node start'| awk '{print $1 }') >& peerlog**
// $ scp peerlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python2 readthroughput_p3.py

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first/workload/result# python2 readthroughput_p3.py
block id:
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
```

// For example, we can get block size
peer channel fetch 2 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem

```
bash-5.0# peer channel fetch 2 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
2026-05-07 08:15:21.880 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-07 08:15:21.882 UTC [cli.common] readBlock -> INFO 002 Received block: 2
```

7.4. Dynamic Adjust Block Parameters on Single Peer

Step 1 (Run): Orderer fetches the latest block configuration, where orderer pulls the channel configuration in protobuf format and create a file named config_block.pb, as follows,

$ **docker exec -it cli bash**
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload# docker exec -it cli bash
bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
2026-05-10 02:40:38.206 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-10 02:40:38.208 UTC [cli.common] readBlock -> INFO 002 Received block: 9
2026-05-10 02:40:38.208 UTC [channelCmd] fetch -> INFO 003 Retrieving last config block: 1
2026-05-10 02:40:38.209 UTC [cli.common] readBlock -> INFO 004 Received block: 1
bash-5.0#
```

Step 2 (Run): We convert the configuration file named config_block.pb to JSON file named config.json, as follows,

```
# configtxlator proto_decode --input config_block.pb --type common.Block |
jq .data.data[0].payload.data.config > config.json
```

```
bash-5.0# configtxlator proto_decode --input config_block.pb --type common.Block | jq .data.data[0].payload.data.config > config.json
bash-5.0# ls
channel-artifacts  config.json        config_block.pb    crypto             scripts
```

We can check that the initial BatchSize is 60,

```
        "Writers": {
          "mod_policy": "Admins",
          "policy": {
            "type": 3,
            "value": {
              "rule": "ANY",
              "sub_policy": "Writers"
            }
          },
          "version": "0"
        }
      },
      "values": {
        "BatchSize": {
          "mod_policy": "Admins",
          "value": {
            "absolute_max_bytes": 103809024,
            "max_message_count": 60,
            "preferred_max_bytes": 524288
          },
          "version": "0"
        },
        "BatchTimeout": {
          "mod_policy": "Admins",
```

Step 3 (Run): We update the BatchSize value from 60 to 80, and save the modified json file as modified_config.json, as follows,

```
# BS=".channel_group.groups.Orderer.values.BatchSize.value.max_message_count"
# jq "$BS" config.json
```

```
bash-5.0# BS=".channel_group.groups.Orderer.values.BatchSize.value.max_message_count"
bash-5.0# jq "$BS" config.json
60
```

```
# jq "$BS = 80" config.json > modified_config.json
# jq "$BS" modified_config.json
```

```
bash-5.0# jq "$BS = 80" config.json > modified_config.json
bash-5.0# jq "$BS" modified_config.json
80
```

Step 4 (Run): We convert the initial config.json to config.pb and convert the modified_config.json to modified_config.pb, as follows,

```
# configtxlator proto_encode --input config.json --type common.Config --output config.pb
# configtxlator proto_encode --input modified_config.json --type common.Config --output
modified_config.pb
```

```
bash-5.0# configtxlator proto_encode --input config.json --type common.Config --output config.pb
bash-5.0# configtxlator proto_encode --input modified_config.json --type common.Config --output modified_config.pb
bash-5.0# ls
channel-artifacts     config_block.pb       modified_config.pb
config.json           crypto                scripts
config.pb             modified_config.json
bash-5.0#
```

Step 5 (Run): We calculate the difference between config.pb and modified_config.pb. The difference to be appended is named file_update.pb, as follows,

```
# configtxlator compute_update --channel_id mychannel --original config.pb --updated modified_config.pb --output final_update.pb
```

```
bash-5.0# configtxlator compute_update --channel_id mychannel --original config.pb --updated modified_config.pb --output final_update.pb
bash-5.0# ls
channel-artifacts     config.pb             crypto                modified_config.json  scripts
config.json           config_block.pb       final_update.pb       modified_config.pb
bash-5.0#
```

Step 6 (Run): We update the difference to the envelope, and append the changes to the config, as follows,

```
# configtxlator proto_decode --input final_update.pb --type common.ConfigUpdate | jq . > final_update.json
# echo "{\"payload\":{\"header\":{\"channel_header\":{\"channel_id\":\"mychannel\", \"type\":2}},\"data\":{\"config_update\":"$(cat final_update.json)"}}}" | jq . > header_in_envolope.json
# configtxlator proto_encode --input header_in_envolope.json --type common.Envelope --output final_update_in_envelope.pb
```

```
bash-5.0# configtxlator proto_decode --input final_update.pb --type common.ConfigUpdate | jq . > final_update.json
bash-5.0# echo "{\"payload\":{\"header\":{\"channel_header\":{\"channel_id\":\"mychannel\", \"type\":2}},\"data\":{\"config_update\":"$(cat final_update.json)"}}}" | jq
 >  header_in_envolope.json
bash-5.0# configtxlator proto_encode --input header_in_envolope.json --type common.Envelope --output final_update_in_envelope.pb
bash-5.0# ls
channel-artifacts           config_block.pb             final_update.pb             modified_config.json
config.json                 crypto                      final_update_in_envelope.pb  modified_config.pb
config.pb                   final_update.json           header_in_envolope.json     scripts
bash-5.0#
```

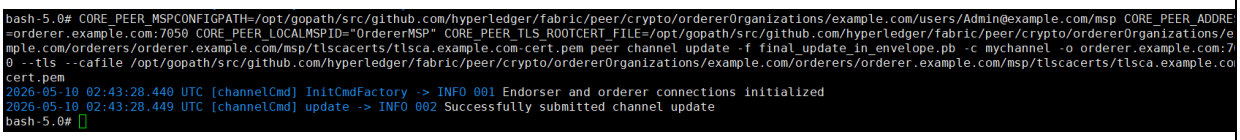
Step 7 (Run): All endorsing peers sign this update proto, as follows,

```
# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp CORE_PEER_ADDRESS=peer0.org1.example.com:7051 CORE_PEER_LOCALMSPID="Org1MSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt peer channel signconfigtx -f final_update_in_envelope.pb
```

```
bash-5.0# configtxlator proto_encode --input config.json --type common.Config --output config.pb
bash-5.0# configtxlator proto_encode --input modified_config.json --type common.Config --output modified_config.pb
bash-5.0# configtxlator compute_update --channel_id mychannel --original config.pb --updated modified_config.pb --output final_update.pb
bash-5.0# configtxlator proto_decode --input final_update.pb --type common.ConfigUpdate | jq . > final_update.json
bash-5.0# echo "{\"payload\":{\"header\":{\"channel_header\":{\"channel_id\":\"mychannel\", \"type\":2}},\"data\":{\"config_update\":"$(cat final_update.json)"}}}" | jq
 >  header_in_envolope.json
bash-5.0# configtxlator proto_encode --input header_in_envolope.json --type common.Envelope --output final_update_in_envelope.pb
bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp CORE_PEER
ADDRESS=peer0.org1.example.com:7051 CORE_PEER_LOCALMSPID="Org1MSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizatio
/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt peer channel signconfigtx -f final_update_in_envelope.pb
2026-05-10 02:42:58.035 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
bash-5.0#
```

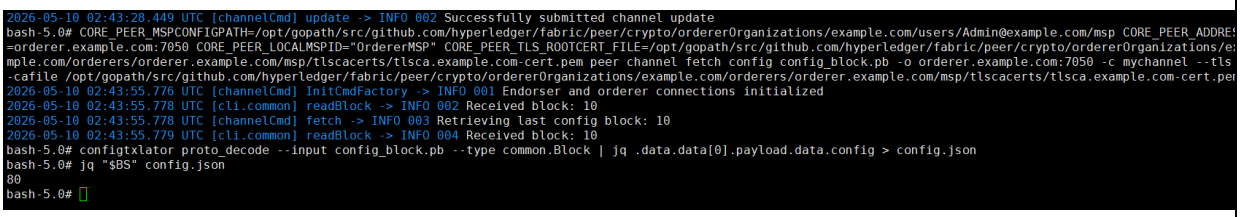
Step 7 (Run): Orderer update the channel as follows,

```
# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel update -f final_update_in_envelope.pb -c mychannel -o orderer.example.com:7050 --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```



Step 8 (Validation): We can validate that the BatchSize is now from 60 to 80, as follows,

```
# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem

# configtxlator proto_decode --input config_block.pb --type common.Block | jq .data.data[0].payload.data.config > config.json
# jq "$BS" config.json
```



Step 9 (Run): Nodes run the clean procedure,

```
$ cd $HOME/fabric-samples/demo-first
$ ./clean.sh
```

8. Dynamic Adj on Multi Peers (Exp 8: 1 CLI + 2 Peers + 1 Orderer)

8.1. Prepare the Hosts

Step 1 (Prepare): All machines prepare the host names,

```
$ vi /etc/hosts
```

```
172.31.210.67 cli
172.31.210.70 peer0.org1.example.com
172.31.210.71 peer0.org2.example.com
172.31.210.72 orderer.example.com
```

Step 2 (Prepare): CLI prepares the Test-Network files,

```
$ cd $HOME/fabric-samples
$ mkdir demo-first
$ cd $HOME/fabric-samples/demo-first
$ vi crypto-config.yaml
$ vi configtx.yaml

$ vi peer0-org1.yaml
$ vi peer0-org2.yaml

$ vi orderer1.yaml

$ vi cli.yaml // (of peer0.org1)

$ vi $HOME/fabric-samples/chaincode/abstore/go/abstore.go // our proposed key-value chaincode
$ vi $HOME/fabric-samples/chaincode/abstore/go/go.mod
```

Step 2 (Prepare): All machines prepare the timestamp alignment,

```
$ date +%s // check the timestamp

$ sudo apt-get install ntpdate // align the timestamp
$ sudo ntpdate cn.pool.ntp.org
$ sudo hwclock –systohc

$ date +%s // check the timestamp again
```

Step 3 (Run): CLI runs the initial block generation,

```
$ cd $HOME/fabric-samples/demo-first
$ chmod +x *.sh
$ ./config.sh
```

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# ./config.sh
org1.example.com
org2.example.com
2026-04-14 16:21:05.926 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 002 orderer type: etcdraft
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] completeInitialization -> INFO 003 Orderer.EtcdRaft.Options unset, setting to tick_interval:"500ms" election_
ck:10 heartbeat_tick:1 max_inflight_blocks:5 snapshot_interval_size:16777216
2026-04-14 16:21:05.942 CST [common.tools.configtxgen.localconfig] Load -> INFO 004 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 005 Generating genesis block
2026-04-14 16:21:05.943 CST [common.tools.configtxgen] doOutputBlock -> INFO 006 Writing genesis block
2026-04-14 16:21:05.957 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:05.973 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:05.973 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 003 Generating new channel configtx
2026-04-14 16:21:05.974 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 004 Writing new channel tx
2026-04-14 16:21:05.988 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.003 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.003 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.004 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
2026-04-14 16:21:06.018 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-04-14 16:21:06.034 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /root/fabric-samples/demo-first/configtx.yaml
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating anchor peer update
2026-04-14 16:21:06.034 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor peer update
```

$ cd $HOME/fabric-samples/demo-first/workload/tls
$ chmod +x *.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ chmod +x *.sh
$ wget https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/node_modules.zip
$ unzip node_modules.zip

$ cd $HOME/fabric-samples/demo-first/scripts
$ chmod +x *.sh

Step 4 (Run): CLI scp the contents to all machines including peers and orderers,

$ cd $HOME/fabric-samples/demo-first
$ **./scp.sh**

```
priv_sk                        100%  241   0.2KB/s   00:0
tlsca.example.com-cert.pem     100%  847   0.8KB/s   00:0
orderer.example.com-cert.pem   100%  769   0.8KB/s   00:0
tlsca.example.com-cert.pem     100%  847   0.8KB/s   00:0
Admin@example.com-cert.pem     100%  769   0.8KB/s   00:0
ca.example.com-cert.pem        100%  839   0.8KB/s   00:0
priv_sk                        100%  241   0.2KB/s   00:0
ca.crt                         100%  847   0.8KB/s   00:0
server.key                     100%  241   0.2KB/s   00:0
server.crt                     100%  916   0.9KB/s   00:0
Admin@example.com-cert.pem     100%  769   0.8KB/s   00:0
tlsca.example.com-cert.pem     100%  847   0.8KB/s   00:0
Admin@example.com-cert.pem     100%  769   0.8KB/s   00:0
ca.example.com-cert.pem        100%  839   0.8KB/s   00:0
priv_sk                        100%  241   0.2KB/s   00:0
ca.crt                         100%  847   0.8KB/s   00:0
client.crt                     100%  814   0.8KB/s   00:0
client.key                     100%  241   0.2KB/s   00:0
priv_sk                        100%  241   0.2KB/s   00:0
ca.example.com-cert.pem        100%  839   0.8KB/s   00:0
```

Step 5 (Run): Orderer1 sets up the Docker container,

$ cd $HOME/fabric-samples/demo-first
$ **./orderer1.sh**

```
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first# ./orderer1.sh
Creating volume "net_orderer.example.com" with default driver
Creating orderer.example.com ... done
```

$ docker ps

```
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                             COMMAND             CREATED             STATUS              PORTS                    NAMES
4a207f175833        hyperledger/fabric-orderer:2.2.0   "orderer"           8 seconds ago       Up 8 seconds        0.0.0.0:7050->7050/tcp   orderer.example.com
```

Step 6 (Run): Peer1 sets up the Docker container,

$ cd $HOME/fabric-samples/demo-first
$ ./**peer1.sh**

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first# ./peer1.sh
Creating volume "net_peer0.org1.example.com" with default driver
Creating peer0.org1.example.com ... done
```

$ docker ps

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                      COMMAND             CREATED             STATUS              PORTS                    NAMES
38fb1129b2ae        hyperledger/fabric-peer:2.2.0   "peer node start"   2 seconds ago       Up 1 second         0.0.0.0:7051->7051/tcp   peer0.org1.example.com
```

Step 7 (Run): Peer2 sets up the Docker container,

$ cd $HOME/fabric-samples/demo-first
$ ./**peer2.sh**

```
root@iZ7xviqssiy2jdsfcldpc1Z:~/fabric-samples/demo-first# ./peer2.sh
Creating volume "net_peer0.org2.example.com" with default driver
Creating peer0.org2.example.com ... done
```

$ docker ps

```
root@iZ7xviqssiy2jdsfcldpc1Z:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                      COMMAND             CREATED             STATUS              PORTS                                        NAMES
e88f655db1d9        hyperledger/fabric-peer:2.2.0   "peer node start"   2 seconds ago       Up 1 second         7051/tcp, 0.0.0.0:8051->8051/tcp   peer0.org2.example.com
```

Step 8 (Run): CLI sets up the Docker container,

$ cd $HOME/fabric-samples/demo-first
$ ./**cli.sh**

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# ./cli.sh
Creating cli ... done
```

$ docker ps

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                       COMMAND             CREATED             STATUS              PORTS               NAMES
30e6fbad4c51        hyperledger/fabric-tools:2.2.0   "/bin/bash"         2 seconds ago       Up 1 second                             cli
```

8.2. Start the Network

Step 1 (Run): CLI connects to Peer0.org1.example.com:7051, creates a channel.block, and then all peers join the channel.block,

// CLI goes to `peer0.org1.example.com:7051`
$ docker exec -it cli bash

// Environment Configuration for `peer0.org1.example.com:7051`
cd scripts
./**step1.sh** // All peers join the channel

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                          COMMAND             CREATED             STATUS              PORTS               NAMES
30e6fbad4c51        hyperledger/fabric-tools:2.2.0   "/bin/bash"         2 seconds ago       Up 1 second                             cli
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# docker exec -it cli bash
bash-5.0# cd scripts
bash-5.0# ./step1.sh
2026-04-14 08:26:37.630 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:37.647 UTC [cli.common] readBlock -> INFO 002 Expect block, but got status: &{NOT_FOUND}
2026-04-14 08:26:37.649 UTC [channelCmd] InitCmdFactory -> INFO 003 Endorser and orderer connections initialized
2026-04-14 08:26:37.850 UTC [cli.common] readBlock -> INFO 004 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:37.852 UTC [channelCmd] InitCmdFactory -> INFO 005 Endorser and orderer connections initialized
2026-04-14 08:26:38.053 UTC [cli.common] readBlock -> INFO 006 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.054 UTC [channelCmd] InitCmdFactory -> INFO 007 Endorser and orderer connections initialized
2026-04-14 08:26:38.255 UTC [cli.common] readBlock -> INFO 008 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.257 UTC [channelCmd] InitCmdFactory -> INFO 009 Endorser and orderer connections initialized
2026-04-14 08:26:38.457 UTC [cli.common] readBlock -> INFO 00a Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.459 UTC [channelCmd] InitCmdFactory -> INFO 00b Endorser and orderer connections initialized
2026-04-14 08:26:38.659 UTC [cli.common] readBlock -> INFO 00c Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.661 UTC [channelCmd] InitCmdFactory -> INFO 00d Endorser and orderer connections initialized
2026-04-14 08:26:38.864 UTC [cli.common] readBlock -> INFO 00e Received block: 0
2026-04-14 08:26:38.897 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:38.921 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-14 08:26:38.953 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:38.977 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-14 08:26:39.004 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:39.015 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
2026-04-14 08:26:39.040 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:39.050 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0#
```

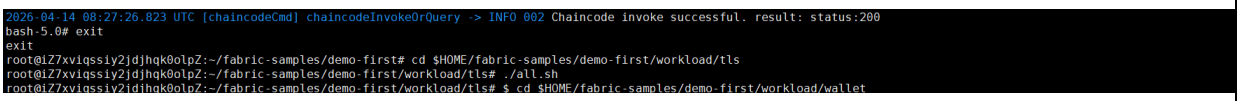
./**step2.sh** // All peers join the chaincode

```
11ac10e3978491cdaab7d56219165c0b45\022\006mycc_1" >
2026-04-14 08:27:14.093 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 002 Chaincode code package identifier: mycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cda
b7d56219165c0b45
2026-04-14 08:27:20.092 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 001 Installed remotely: response:<status:200 payload:"\nGmycc_1:9c2d699dc0bb37ea77cfbadd7567(
11ac10e3978491cdaab7d56219165c0b45\022\006mycc_1" >
2026-04-14 08:27:20.093 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 002 Chaincode code package identifier: mycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cda
b7d56219165c0b45
2026-04-14 08:27:20.160 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychannel) orderer endpoint: orderer.example.com:7050
2026-04-14 08:27:21.182 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [2e49555a3a8dd90810de8e5e90973f90a615a62711c3be05cc97ddf7b1cc0a23] committed with status (VALID) at
2026-04-14 08:27:21.218 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychannel) orderer endpoint: orderer.example.com:7050
2026-04-14 08:27:22.238 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [a720bb2d81a5e1dc2f56ab369b6df082355423c38337e372ffc9f80a515d6f26] committed with status (VALID) at
{
        "approvals": {
                "Org1MSP": true,
                "Org2MSP": true
        }
}
2026-04-14 08:27:23.402 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [f8eba34bdcfcd726a6959a47dc60c95e05cf7652a8b7f4cf80c7b44f70bd236b] committed with status (VALID) at peer0
rg1.example.com:7051
2026-04-14 08:27:23.402 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [f8eba34bdcfcd726a6959a47dc60c95e05cf7652a8b7f4cf80c7b44f70bd236b] committed with status (VALID) at peer0
rg2.example.com:8051
2026-04-14 08:27:24.693 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [862b6b71fe7bdc81df4b23af39e63d1e1017b2c3b39e34b6cdf6ac6213bb905c] committed with status (VALID) at peer0
rg1.example.com:7051
2026-04-14 08:27:24.693 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
2026-04-14 08:27:25.741 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [1722e9e0ffb97b289c38f53946f397b242b6d3ff591f2ba4f48ae6ed239311f3] committed with status (VALID) at peer0
rg1.example.com:7051
2026-04-14 08:27:25.742 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
200tps
2026-04-14 08:27:26.823 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [64c2861c9c16c9c93e549c87eab9181e87ff23827fb1207ae4c213c802b92110] committed with status (VALID) at peer0
rg2.example.com:8051
2026-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
bash-5.0#
```

8.3. Start Nodejs SDK

Step 1 (Run): CLI configures the Node.js SDK folder and tls, wallet, and workload files for all peers,

```
# exit
$ mkdir -p $HOME/fabric-samples/demo-first/workload
$ mkdir -p $HOME/fabric-samples/demo-first/workload/tls
$ mkdir -p $HOME/fabric-samples/demo-first/workload/wallet
$ mkdir -p $HOME/fabric-samples/demo-first/workload/result

$ cd $HOME/fabric-samples/demo-first/workload/tls
$ vi read_pem.py
$ vi prepare_tls_peer0_org1.sh
$ vi prepare_tls_peer0_org2.sh
$ ./all.sh
```




```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ vi prepare_wallet_user1_org1.py
$ vi prepare_wallet_user1_org2.py
$ ./all.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ mkdir sdk-peer0-org1
$ mkdir sdk-peer0-org2
$ vi invoke.js
$ vi prepare_sdk_peer0_org1.sh
$ vi prepare_sdk_peer0_org2.sh
$ ./all.sh
```

Step 2 (Prepare): CLI prepares the Nodejs SDK dependencies on CLI. Note that you may pass this step by copying the compiled nodejs source code https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/node_modules.zip,

```
$ cd $HOME/fabric-samples/demo-first/workload
$ vi package.json
$ npm install
$ vi $HOME/fabric-samples/demo-first/workload/node_modules/fabric-network/lib/transaction.js
```

Step 3 (Run): Cli prepares wallets of User1@org1.example.com and User1@org2.example.com,

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ rm -rf *.id
$ python prepare_wallet_user1_org1.py
```

$ python prepare_wallet_user1_org2.py
$ ./**all.sh**

Step 4 (Run): CLI invokes and queries transactions via Node.js,

$ cd $HOME/fabric-samples/demo-first/workload
$ ./**scp.sh** // CLI copies files to all machines

$ ./**ssh.sh** **> result/benchlog**
$ ls result -l

```
root@iZ7xviqssiy2jdsfcldpbvZ:~# cd $HOME/fabric-samples/demo-first/workload/sdk-peer0-org1
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first/workload/sdk-peer0-org1# nodejs invoke.js 60 100 10 1
Wallet path: /root/fabric-samples/demo-first/workload/wallet
Send_Endorsing_Proposal: 1776155562.442
Send_Endorsing_Proposal: 1776155562.447
Send_Endorsing_Proposal: 1776155562.45
Send_Endorsing_Proposal: 1776155562.452
Send_Endorsing_Proposal: 1776155562.455
Send_Endorsing_Proposal: 1776155562.458
Send_Endorsing_Proposal: 1776155562.46
Send_Endorsing_Proposal: 1776155562.462
Send_Endorsing_Proposal: 1776155562.464
Send_Endorsing_Proposal: 1776155562.468
Send_Endorsing_Proposal: 1776155562.471
Send_Endorsing_Proposal: 1776155562.474
Send_Endorsing_Proposal: 1776155562.476
Send_Endorsing_Proposal: 1776155562.478
Send_Endorsing_Proposal: 1776155562.48
Send_Endorsing_Proposal: 1776155562.482
Send_Endorsing_Proposal: 1776155562.484
Send_Endorsing_Proposal: 1776155562.487
Send_Endorsing_Proposal: 1776155562.488
Send_Endorsing_Proposal: 1776155562.49
Send_Endorsing_Proposal: 1776155562.492
Send_Endorsing_Proposal: 1776155562.494
Send_Endorsing_Proposal: 1776155562.495
Send_Endorsing_Proposal: 1776155562.497
Send_Endorsing_Proposal: 1776155562.499
Send_Endorsing_Proposal: 1776155562.502
Send_Endorsing_Proposal: 1776155562.504
Send_Endorsing_Proposal: 1776155562.507
```

// $ node query.js
// $ node invoke.js 30 500 20 1

Step 5 (Run): Analyze the log,

// CLI analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ python **readthroughput_p1.py** // rate of sending endorsed txs

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/result# python readthroughput_p1.py
Avg_Send_Rate_Phrase_1:
5.7471256178
```

$ sudo apt-get install python-numpy
$ python readdelay_p1.py // delay of endorsing phase

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/result# python readdelay_p1.py
Mean_Delay_Endorse:
0.0188799798489
Mean_Delay_Orderer:
0.00146238137317
Mean_Delay_Commit:
0.178739959002
CDF with the probability of 50% 0.00100016593933
CDF with the probability of 90% 0.00200009346008
CDF with the probability of 95% 0.00300002098083
```

// Orderer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& ordererlog
$ **docker logs $(docker ps -a | grep 'orderer'| awk '{print $1 }') >& ordererlog**
// $ scp ordererlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python readthroughput_p2.py // rate of generating blocks

```
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first/workload/result# python readthroughput_p2.py
Block_ID:
lastPersistedBlock=[24]

lastPersistedBlock=[25]

lastPersistedBlock=[26]

lastPersistedBlock=[27]

lastPersistedBlock=[28]

lastPersistedBlock=[29]

lastPersistedBlock=[30]

lastPersistedBlock=[31]

lastPersistedBlock=[32]
```

// Peer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& peerlog
$ **docker logs $(docker ps -a | grep 'peer node start'| awk '{print $1 }') >& peerlog**
// $ scp peerlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python2 readthroughput_p3.py

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first/workload/result# python2 readthroughput_p3.py
block id:
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
```

// For example, we can get block size
peer channel fetch 2 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem

```
bash-5.0# peer channel fetch 2 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
2026-05-07 08:15:21.880 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-07 08:15:21.882 UTC [cli.common] readBlock -> INFO 002 Received block: 2
```

8.4. Dynamic Adjust Block Parameters on Multiple Peers

Step 1 (Run): Orderer fetches the latest block configuration, where orderer pulls the channel configuration in protobuf format and create a file named config_block.pb, as follows,

```
$ docker exec -it cli bash
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp
CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload# docker exec -it cli bash
bash-5.0# CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/users/Admin@example.com/msp CORE_PEER_ADDRES
=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP" CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/e
mple.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel --tls
-cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pe
2026-05-09 02:53:08.455 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-09 02:53:08.457 UTC [cli.common] readBlock -> INFO 002 Received block: 13
2026-05-09 02:53:08.457 UTC [channelCmd] fetch -> INFO 003 Retrieving last config block: 2
2026-05-09 02:53:08.458 UTC [cli.common] readBlock -> INFO 004 Received block: 2
bash-5.0# ls
channel-artifacts  config_block.pb    crypto            scripts
bash-5.0#
```

Step 2 (Run): We convert the configuration file named config_block.pb to JSON file named config.json, as follows,

```
# configtxlator proto_decode --input config_block.pb --type common.Block |
jq .data.data[0].payload.data.config > config.json
```

```
bash-5.0# configtxlator proto_decode --input config_block.pb --type common.Block | jq .data.data[0].payload.data.config > config.json
bash-5.0# ls
channel-artifacts  config.json        config_block.pb    crypto            scripts
```

We can check that the initial BatchSize is 60,

```
    },
    "values": {
      "BatchSize": {
        "mod_policy": "Admins",
        "value": {
          "absolute_max_bytes": 103809024,
          "max_message_count": 60,
          "preferred_max_bytes": 524288
        },
        "version": "0"
```

Step 3 (Run): We update the BatchSize value from 60 to 80, and save the modified json file as modified_config.json, as follows,

```
# BS=".channel_group.groups.Orderer.values.BatchSize.value.max_message_count"
# jq "$BS" config.json
```

```
bash-5.0# BS=".channel_group.groups.Orderer.values.BatchSize.value.max_message_count"
bash-5.0# jq "$BS" config.json
60
```

```
# jq "$BS = 80" config.json > modified_config.json
# jq "$BS" modified_config.json
```

```
bash-5.0# jq "$BS = 80" config.json > modified_config.json
bash-5.0# jq "$BS" modified_config.json
80
```

Step 4 (Run): We convert the initial config.json to config.pb and convert the modified_config.json to modified_config.pb, as follows,

```
# configtxlator proto_encode --input config.json --type common.Config --output config.pb
# configtxlator proto_encode --input modified_config.json --type common.Config --output
modified_config.pb
```

```
bash-5.0# configtxlator proto_encode --input config.json --type common.Config --output config.pb
bash-5.0# configtxlator proto_encode --input modified_config.json --type common.Config --output modified_config.pb
bash-5.0# ls
channel-artifacts     config_block.pb       modified_config.pb
config.json           crypto                scripts
config.pb             modified_config.json
bash-5.0#
```

Step 5 (Run): We calculate the difference between config.pb and modified_config.pb. The difference to be appended is named file_update.pb, as follows,

```
# configtxlator compute_update --channel_id mychannel --original config.pb --updated
modified_config.pb --output final_update.pb
```

```
bash-5.0# configtxlator compute_update --channel_id mychannel --original config.pb --updated modified_config.pb --output final_update.pb
bash-5.0# ls
channel-artifacts     config.pb             crypto                modified_config.json  scripts
config.json           config_block.pb       final_update.pb       modified_config.pb
bash-5.0#
```

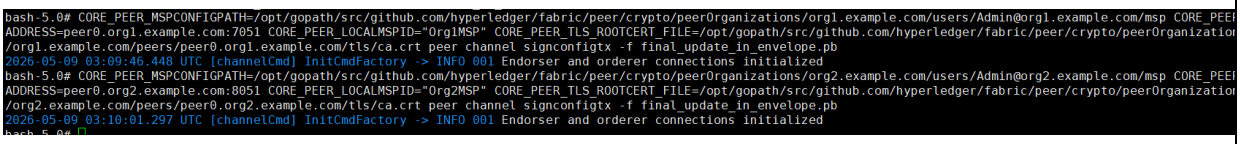
Step 6 (Run): We update the difference to the envelope, and append the changes to the config, as follows,

```
# configtxlator proto_decode --input final_update.pb --type common.ConfigUpdate | jq . >
final_update.json
# echo "{\"payload\":{\"header\":{\"channel_header\":{\"channel_id\":\"mychannel\",
\"type\":2}},\"data\":{\"config_update\":"$(cat final_update.json)"}}}" | jq . >
header_in_envolope.json
# configtxlator proto_encode --input header_in_envolope.json --type common.Envelope --output
final_update_in_envelope.pb
```

```
bash-5.0# configtxlator proto_decode --input final_update.pb --type common.ConfigUpdate | jq . > final_update.json
bash-5.0# echo "{\"payload\":{\"header\":{\"channel_header\":{\"channel_id\":\"mychannel\", \"type\":2}},\"data\":{\"config_update\":"$(cat final_update.json)"}}}" | jq
 >  header_in_envolope.json
bash-5.0# configtxlator proto_encode --input header_in_envolope.json --type common.Envelope --output final_update_in_envelope.pb
bash-5.0# ls
channel-artifacts           config_block.pb             final_update.pb             modified_config.json
config.json                 crypto                      final_update_in_envelope.pb  modified_config.pb
config.pb                   final_update.json           header_in_envolope.json     scripts
bash-5.0#
```

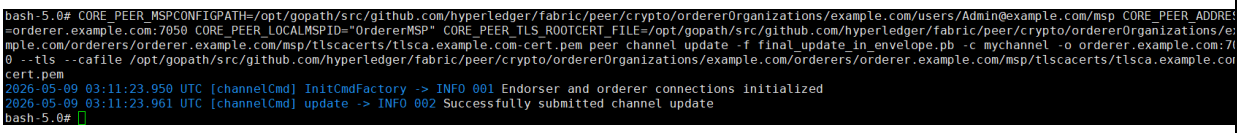
Step 7 (Run): All endorsing peers sign this update proto, as follows,

```
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peer
Organizations/org1.example.com/users/Admin@org1.example.com/msp
CORE_PEER_ADDRESS=peer0.org1.example.com:7051 CORE_PEER_LOCALMSPID="Org1MSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/pe
erOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt peer channel
signconfigtx -f final_update_in_envelope.pb

#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peer
Organizations/org2.example.com/users/Admin@org2.example.com/msp
CORE_PEER_ADDRESS=peer0.org2.example.com:8051 CORE_PEER_LOCALMSPID="Org2MSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/pe
erOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt peer channel
signconfigtx -f final_update_in_envelope.pb
```



Step 7 (Run): Orderer update the channel as follows,

```
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/orde
rerOrganizations/example.com/users/Admin@example.com/msp
CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/or
dererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.co
m-cert.pem peer channel update -f final_update_in_envelope.pb -c mychannel -o
orderer.example.com:7050 --tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/
orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```



Step 8 (Validation): We can validate that the BatchSize is now from 60 to 80, as follows,

```
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/orde
rerOrganizations/example.com/users/Admin@example.com/msp
```

```
CORE_PEER_ADDRESS=orderer.example.com:7050 CORE_PEER_LOCALMSPID="OrdererMSP"
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem peer channel fetch config config_block.pb -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem

# configtxlator proto_decode --input config_block.pb --type common.Block | jq .data.data[0].payload.data.config > config.json
# jq "$BS" config.json
```

Step 9 (Run): Nodes run the clean procedure,

```
$ cd $HOME/fabric-samples/demo-first
$ ./clean.sh
```

9. Different Transaction Payoff Sizes (Exp 9: 1 CLI + 2 Peers + 1 Orderer)

9.1. Prepare the Hosts

Step 1 (Prepare): All machines prepare the host names,

```
$ vi /etc/hosts
```

```
172.31.210.67 cli
172.31.210.70 peer0.org1.example.com
172.31.210.71 peer0.org2.example.com
172.31.210.72 orderer.example.com
```

Step 2 (Prepare): CLI prepares the Test-Network files,

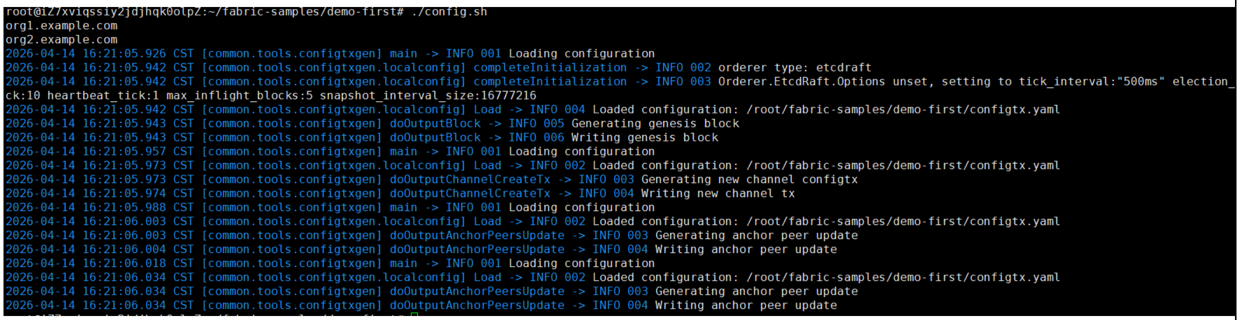
```
$ cd $HOME/fabric-samples
$ mkdir demo-first
$ cd $HOME/fabric-samples/demo-first
$ vi crypto-config.yaml
$ vi configtx.yaml

$ vi peer0-org1.yaml
$ vi peer0-org2.yaml

$ vi orderer1.yaml

$ vi cli.yaml // (of peer0.org1)

$ vi $HOME/fabric-samples/chaincode/abstore/go/abstore.go // our proposed key-value chaincode
$ vi $HOME/fabric-samples/chaincode/abstore/go/go.mod
```

Step 2 (Prepare): All machines prepare the timestamp alignment,

```
$ date +%s // check the timestamp

$ sudo apt-get install ntpdate // align the timestamp
$ sudo ntpdate cn.pool.ntp.org
$ sudo hwclock –systohc

$ date +%s // check the timestamp again
```

Step 3 (Run): CLI runs the initial block generation,

```
$ cd $HOME/fabric-samples/demo-first
$ chmod +x *.sh
$ ./config.sh
```



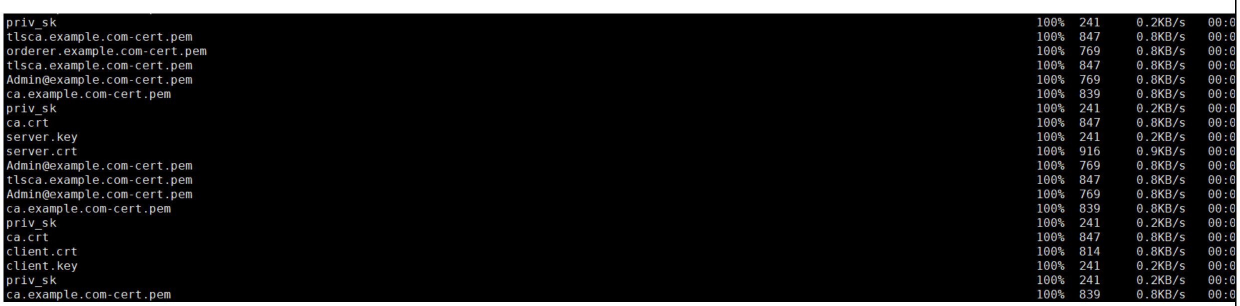
$ cd $HOME/fabric-samples/demo-first/workload/tls
$ chmod +x *.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ chmod +x *.sh
$ wget https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/node_modules.zip
$ unzip node_modules.zip

$ cd $HOME/fabric-samples/demo-first/scripts
$ chmod +x *.sh

Step 4 (Run): CLI scp the contents to all machines including peers and orderers,

$ cd $HOME/fabric-samples/demo-first
$ **./scp.sh**



Step 5 (Run): Orderer1 sets up the Docker container,

$ cd $HOME/fabric-samples/demo-first
$ **./orderer1.sh**

$ docker ps

```
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                               COMMAND             CREATED             STATUS              PORTS                    NAMES
4a207f175833        hyperledger/fabric-orderer:2.2.0    "orderer"           8 seconds ago       Up 8 seconds        0.0.0.0:7050->7050/tcp   orderer.example.com
```

Step 6 (Run): Peer1 sets up the Docker container,

$ cd $HOME/fabric-samples/demo-first
$ ./**peer1.sh**

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first# ./peer1.sh
Creating volume "net_peer0.org1.example.com" with default driver
Creating peer0.org1.example.com ... done
```

$ docker ps

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                               COMMAND             CREATED             STATUS              PORTS                    NAMES
38fb1129b2ae        hyperledger/fabric-peer:2.2.0       "peer node start"   2 seconds ago       Up 1 second         0.0.0.0:7051->7051/tcp   peer0.org1.example.com
```

Step 7 (Run): Peer2 sets up the Docker container,

$ cd $HOME/fabric-samples/demo-first
$ ./**peer2.sh**

```
root@iZ7xviqssiy2jdsfcldpc1Z:~/fabric-samples/demo-first# ./peer2.sh
Creating volume "net_peer0.org2.example.com" with default driver
Creating peer0.org2.example.com ... done
```

$ docker ps

```
root@iZ7xviqssiy2jdsfcldpc1Z:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                               COMMAND             CREATED             STATUS              PORTS                                        NAMES
e88f655db1d9        hyperledger/fabric-peer:2.2.0       "peer node start"   2 seconds ago       Up 1 second         7051/tcp, 0.0.0.0:8051->8051/tcp             peer0.org2.example.com
```

Step 8 (Run): CLI sets up the Docker container,

$ cd $HOME/fabric-samples/demo-first
$ ./**cli.sh**

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# ./cli.sh
Creating cli ... done
```

$ docker ps

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                               COMMAND             CREATED             STATUS              PORTS               NAMES
30e6fbad4c51        hyperledger/fabric-tools:2.2.0      "/bin/bash"         2 seconds ago       Up 1 second                             cli
```

9.2. Start the Network

Step 1 (Run): CLI connects to Peer0.org1.example.com:7051, creates a channel.block, and then all peers join the channel.block,

// CLI goes to `peer0.org1.example.com:7051`
$ docker exec -it cli bash

// Environment Configuration for `peer0.org1.example.com:7051`
cd scripts
./**step1.sh** // All peers join the channel

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                          COMMAND             CREATED             STATUS              PORTS               NAMES
30e6fbad4c51        hyperledger/fabric-tools:2.2.0   "/bin/bash"         2 seconds ago       Up 1 second                             cli
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# docker exec -it cli bash
bash-5.0# cd scripts
bash-5.0# ./step1.sh
2026-04-14 08:26:37.630 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:37.647 UTC [cli.common] readBlock -> INFO 002 Expect block, but got status: &{NOT_FOUND}
2026-04-14 08:26:37.649 UTC [channelCmd] InitCmdFactory -> INFO 003 Endorser and orderer connections initialized
2026-04-14 08:26:37.850 UTC [cli.common] readBlock -> INFO 004 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:37.852 UTC [channelCmd] InitCmdFactory -> INFO 005 Endorser and orderer connections initialized
2026-04-14 08:26:38.053 UTC [cli.common] readBlock -> INFO 006 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.054 UTC [channelCmd] InitCmdFactory -> INFO 007 Endorser and orderer connections initialized
2026-04-14 08:26:38.255 UTC [cli.common] readBlock -> INFO 008 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.257 UTC [channelCmd] InitCmdFactory -> INFO 009 Endorser and orderer connections initialized
2026-04-14 08:26:38.457 UTC [cli.common] readBlock -> INFO 00a Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.459 UTC [channelCmd] InitCmdFactory -> INFO 00b Endorser and orderer connections initialized
2026-04-14 08:26:38.659 UTC [cli.common] readBlock -> INFO 00c Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.661 UTC [channelCmd] InitCmdFactory -> INFO 00d Endorser and orderer connections initialized
2026-04-14 08:26:38.864 UTC [cli.common] readBlock -> INFO 00e Received block: 0
2026-04-14 08:26:38.897 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:38.921 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-14 08:26:38.953 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:38.977 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-14 08:26:39.004 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:39.015 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
2026-04-14 08:26:39.040 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:39.050 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0#
```

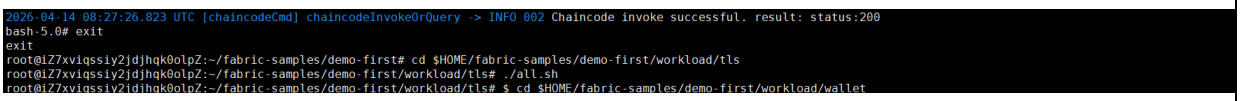
./**step2.sh** // All peers join the chaincode

```
11ac10e3978491cdaab7d56219165c0b45\022\006mycc_1" >
2026-04-14 08:27:14.093 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 002 Chaincode code package identifier: mycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cda
b7d56219165c0b45
2026-04-14 08:27:20.092 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 001 Installed remotely: response:<status:200 payload:"\nGmycc_1:9c2d699dc0bb37ea77cfbadd7567(
11ac10e3978491cdaab7d56219165c0b45\022\006mycc_1" >
2026-04-14 08:27:20.093 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 002 Chaincode code package identifier: mycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cda
b7d56219165c0b45
2026-04-14 08:27:20.160 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychannel) orderer endpoint: orderer.example.com:7050
2026-04-14 08:27:21.182 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [2e49555a3a8dd90810de8e5e90973f90a615a62711c3be05cc97ddf7b1cc0a23] committed with status (VALID) at
2026-04-14 08:27:21.218 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychannel) orderer endpoint: orderer.example.com:7050
2026-04-14 08:27:22.238 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [a720bb2d81a5e1dc2f56ab369b6df082355423c38337e372ffc9f80a515d6f26] committed with status (VALID) at
{
        "approvals": {
                "Org1MSP": true,
                "Org2MSP": true
        }
}
2026-04-14 08:27:23.402 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [f8eba34bdcfcd726a6959a47dc60c95e05cf7652a8b7f4cf80c7b44f70bd236b] committed with status (VALID) at peer0
rg1.example.com:7051
2026-04-14 08:27:23.402 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [f8eba34bdcfcd726a6959a47dc60c95e05cf7652a8b7f4cf80c7b44f70bd236b] committed with status (VALID) at peer0
rg2.example.com:8051
2026-04-14 08:27:24.693 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [862b6b71fe7bdc81df4b23af39e63d1e1017b2c3b39e34b6cdf6ac6213bb905c] committed with status (VALID) at peer0
rg1.example.com:7051
2026-04-14 08:27:24.693 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
2026-04-14 08:27:25.741 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [1722e9e0ffb97b289c38f53946f397b242b6d3ff591f2ba4f48ae6ed239311f3] committed with status (VALID) at peer0
rg1.example.com:7051
2026-04-14 08:27:25.742 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
200tps
2026-04-14 08:27:26.823 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [64c2861c9c16c9c93e549c87eab9181e87ff23827fb1207ae4c213c802b92110] committed with status (VALID) at peer0
rg2.example.com:8051
2026-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
bash-5.0#
```

9.3. Start Nodejs SDK

Step 1 (Run): CLI configures the Node.js SDK folder and tls, wallet, and workload files for all peers,

```
# exit
$ mkdir -p $HOME/fabric-samples/demo-first/workload
$ mkdir -p $HOME/fabric-samples/demo-first/workload/tls
$ mkdir -p $HOME/fabric-samples/demo-first/workload/wallet
$ mkdir -p $HOME/fabric-samples/demo-first/workload/result

$ cd $HOME/fabric-samples/demo-first/workload/tls
$ vi read_pem.py
$ vi prepare_tls_peer0_org1.sh
$ vi prepare_tls_peer0_org2.sh
$ ./all.sh
```




```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ vi prepare_wallet_user1_org1.py
$ vi prepare_wallet_user1_org2.py
$ ./all.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ mkdir sdk-peer0-org1
$ mkdir sdk-peer0-org2
$ vi invoke.js
$ vi prepare_sdk_peer0_org1.sh
$ vi prepare_sdk_peer0_org2.sh
$ ./all.sh
```

Step 2 (Prepare): CLI prepares the Nodejs SDK dependencies on CLI. Note that you may pass this step by copying the compiled nodejs source code https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/node_modules.zip,

```
$ cd $HOME/fabric-samples/demo-first/workload
$ vi package.json
$ npm install
$ vi $HOME/fabric-samples/demo-first/workload/node_modules/fabric-network/lib/transaction.js
```

Step 3 (Run): Cli prepares wallets of User1@org1.example.com and User1@org2.example.com,

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ rm -rf *.id
$ python prepare_wallet_user1_org1.py
```

$ python prepare_wallet_user1_org2.py
$ ./**all.sh**

Step 4 (Run): CLI invokes and queries transactions via Node.js,

$ cd $HOME/fabric-samples/demo-first/workload
$ ./**scp.sh** // CLI copies files to all machines

$ ./**ssh.sh** **> result/benchlog**
$ ls result -l

```
root@iZ7xviqssiy2jdsfcldpbvZ:~# cd $HOME/fabric-samples/demo-first/workload/sdk-peer0-org1
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first/workload/sdk-peer0-org1# nodejs invoke.js 60 100 10 1
Wallet path: /root/fabric-samples/demo-first/workload/wallet
Send_Endorsing_Proposal: 1776155562.442
Send_Endorsing_Proposal: 1776155562.447
Send_Endorsing_Proposal: 1776155562.45
Send_Endorsing_Proposal: 1776155562.452
Send_Endorsing_Proposal: 1776155562.455
Send_Endorsing_Proposal: 1776155562.458
Send_Endorsing_Proposal: 1776155562.46
Send_Endorsing_Proposal: 1776155562.462
Send_Endorsing_Proposal: 1776155562.464
Send_Endorsing_Proposal: 1776155562.468
Send_Endorsing_Proposal: 1776155562.471
Send_Endorsing_Proposal: 1776155562.474
Send_Endorsing_Proposal: 1776155562.476
Send_Endorsing_Proposal: 1776155562.478
Send_Endorsing_Proposal: 1776155562.48
Send_Endorsing_Proposal: 1776155562.482
Send_Endorsing_Proposal: 1776155562.484
Send_Endorsing_Proposal: 1776155562.487
Send_Endorsing_Proposal: 1776155562.488
Send_Endorsing_Proposal: 1776155562.49
Send_Endorsing_Proposal: 1776155562.492
Send_Endorsing_Proposal: 1776155562.494
Send_Endorsing_Proposal: 1776155562.495
Send_Endorsing_Proposal: 1776155562.497
Send_Endorsing_Proposal: 1776155562.499
Send_Endorsing_Proposal: 1776155562.502
Send_Endorsing_Proposal: 1776155562.504
Send_Endorsing_Proposal: 1776155562.507
```

// $ node query.js
// $ node invoke.js 30 500 20 1

Step 5 (Run): Analyze the log,

// CLI analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ python **readthroughput_p1.py** // rate of sending endorsed txs

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/result# python readthroughput_p1.py
Avg_Send_Rate_Phrase_1:
5.7471256178
```

$ sudo apt-get install python-numpy
$ python readdelay_p1.py // delay of endorsing phase

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/result# python readdelay_p1.py
Mean_Delay_Endorse:
0.0188799798489
Mean_Delay_Orderer:
0.00146238137317
Mean_Delay_Commit:
0.178739959002
CDF with the probability of 50% 0.00100016593933
CDF with the probability of 90% 0.00200009346008
CDF with the probability of 95% 0.00300002098083
```

// Orderer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy

//$ docker logs containername >& ordererlog
$ **docker logs $(docker ps -a | grep 'orderer'| awk '{print $1 }') >& ordererlog**
// $ scp ordererlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python readthroughput_p2.py // rate of generating blocks

```
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first/workload/result# python readthroughput_p2.py
Block_ID:
lastPersistedBlock=[24]

lastPersistedBlock=[25]

lastPersistedBlock=[26]

lastPersistedBlock=[27]

lastPersistedBlock=[28]

lastPersistedBlock=[29]

lastPersistedBlock=[30]

lastPersistedBlock=[31]

lastPersistedBlock=[32]
```

// Peer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& peerlog
$ **docker logs $(docker ps -a | grep 'peer node start'| awk '{print $1 }') >& peerlog**
// $ scp peerlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python2 readthroughput_p3.py

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first/workload/result# python2 readthroughput_p3.py
block id:
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
```

// For example, we can get block size
peer channel fetch 2 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem

```
bash-5.0# peer channel fetch 2 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
2026-05-07 08:15:21.880 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-07 08:15:21.882 UTC [cli.common] readBlock -> INFO 002 Received block: 2
```

9.4. Different Transaction Payoff Sizes

Step 1 (Run): CLI invokes and queries transactions via Node.js,

$ cd $HOME/fabric-samples/demo-first/workload

$ ./ssh.sh **> result/benchlog** // Transaction payoff size: 1 byte
$ ls result -l

```
Get_Endorsed_Proposal: 1778484488.036
Delay_Endorsing: 0.02500009536743164
Get_Ordered_Transaction: 1778484488.037
Delay_Ordering: 0.0009999275207519531
Get_Ordered_Transaction: 1778484488.037
Delay_Ordering: 0.0009999275207519531
Get_Ordered_Transaction: 1778484488.037
Delay_Ordering: 0.0009999275207519531
Get_Ordered_Transaction: 1778484488.037
Delay_Ordering: 0.0009999275207519531
Get_Ordered_Transaction: 1778484488.037
Delay_Ordering: 0.0009999275207519531
Get_Ordered_Transaction: 1778484488.037
Delay_Ordering: 0.0009999275207519531
Get_Ordered_Transaction: 1778484488.038
Delay_Ordering: 0.002000093460083008
Get_Committed_Transaction: 1778484489.038
Delay_Commit: 1
Get_Committed_Transaction: 1778484489.038
Delay_Commit: 1
Get_Committed_Transaction: 1778484489.038
Delay_Commit: 1
Get_Committed_Transaction: 1778484489.038
Delay_Commit: 1
Get_Committed_Transaction: 1778484489.038
Delay_Commit: 1
Get_Committed_Transaction: 1778484489.039
Delay_Commit: 1.000999927520752
Get_Committed_Transaction: 1778484489.039
Delay_Commit: 1.000999927520752
Get_Committed_Transaction: 1778484489.039
Delay_Commit: 1.000999927520752
Get_Committed_Transaction: 1778484489.039
Delay_Commit: 1.000999927520752
Get_Committed_Transaction: 1778484489.039
Delay_Commit: 1.000999927520752
```

Step 2 (Run): CLI invokes and queries transactions via Node.js,

$ cd $HOME/fabric-samples/demo-first/workload

$ ./ssh.sh **> result/benchlog** // Transaction payoff size: 10 byte
$ ls result -l

```
Get_Ordered_Transaction: 1778484589.447
Delay_Ordering: 0
Get_Ordered_Transaction: 1778484589.447
Delay_Ordering: 0
Get_Ordered_Transaction: 1778484589.447
Delay_Ordering: 0
Get_Ordered_Transaction: 1778484589.447
Delay_Ordering: 0
Get_Ordered_Transaction: 1778484589.447
Delay_Ordering: 0
Get_Ordered_Transaction: 1778484589.448
Delay_Ordering: 0.0009999275207519531
Get_Ordered_Transaction: 1778484589.448
Delay_Ordering: 0.0009999275207519531
Get_Ordered_Transaction: 1778484589.448
Delay_Ordering: 0.0009999275207519531
Get_Committed_Transaction: 1778484590.447
Delay_Commit: 0.999000072479248
Get_Committed_Transaction: 1778484590.447
Delay_Commit: 0.999000072479248
Get_Committed_Transaction: 1778484590.447
Delay_Commit: 0.999000072479248
Get_Committed_Transaction: 1778484590.447
Delay_Commit: 0.999000072479248
Get_Committed_Transaction: 1778484590.447
Delay_Commit: 0.999000072479248
Get_Committed_Transaction: 1778484590.447
Delay_Commit: 0.999000072479248
Get_Committed_Transaction: 1778484590.447
Delay_Commit: 0.999000072479248
Get_Committed_Transaction: 1778484590.447
Delay_Commit: 0.999000072479248
Get_Committed_Transaction: 1778484590.447
Delay_Commit: 0.999000072479248
Get_Committed_Transaction: 1778484590.447
Delay_Commit: 0.999000072479248
```

Step 3 (Run): CLI invokes and queries transactions via Node.js,

$ cd $HOME/fabric-samples/demo-first/workload

$ ./ssh.sh **> result/benchlog** // Transaction payoff size: 100 byte
$ ls result -l

```
Get_Committed_Transaction: 1778484727.197
Delay_Commit: 0.9579999446868896
Get_Committed_Transaction: 1778484727.197
Delay_Commit: 0.9579999446868896
Get_Committed_Transaction: 1778484727.197
Delay_Commit: 0.9579999446868896
Get_Committed_Transaction: 1778484727.197
Delay_Commit: 0.9579999446868896
Get_Committed_Transaction: 1778484727.197
Delay_Commit: 0.9579999446868896
Get_Committed_Transaction: 1778484727.197
Delay_Commit: 0.9579999446868896
Get_Committed_Transaction: 1778484727.197
Delay_Commit: 0.9579999446868896
Get_Committed_Transaction: 1778484727.197
Delay_Commit: 0.9579999446868896
Get_Committed_Transaction: 1778484727.197
Delay_Commit: 0.9579999446868896
Get_Committed_Transaction: 1778484727.197
Delay_Commit: 0.9579999446868896
Get_Committed_Transaction: 1778484727.197
Delay_Commit: 0.9579999446868896
Get_Committed_Transaction: 1778484727.197
Delay_Commit: 0.9579999446868896
Get_Committed_Transaction: 1778484727.197
Delay_Commit: 0.9579999446868896
Get_Committed_Transaction: 1778484727.197
Delay_Commit: 0.9579999446868896
Get_Committed_Transaction: 1778484727.197
Delay_Commit: 0.9579999446868896
Get_Committed_Transaction: 1778484727.197
Delay_Commit: 0.9579999446868896
Get_Committed_Transaction: 1778484727.197
Delay_Commit: 0.9579999446868896
Get_Committed_Transaction: 1778484727.197
Delay_Commit: 0.9579999446868896
```

10. Different Chaincodes (Exp 10: 1 CLI + 2 Peers + 1 Orderer)

10.1. Prepare the Hosts

Step 1 (Prepare): All machines prepare the host names,

```
$ vi /etc/hosts
```

```
172.31.210.67 cli
172.31.210.70 peer0.org1.example.com
172.31.210.71 peer0.org2.example.com
172.31.210.72 orderer.example.com
```

Step 2 (Prepare): CLI prepares the Test-Network files,

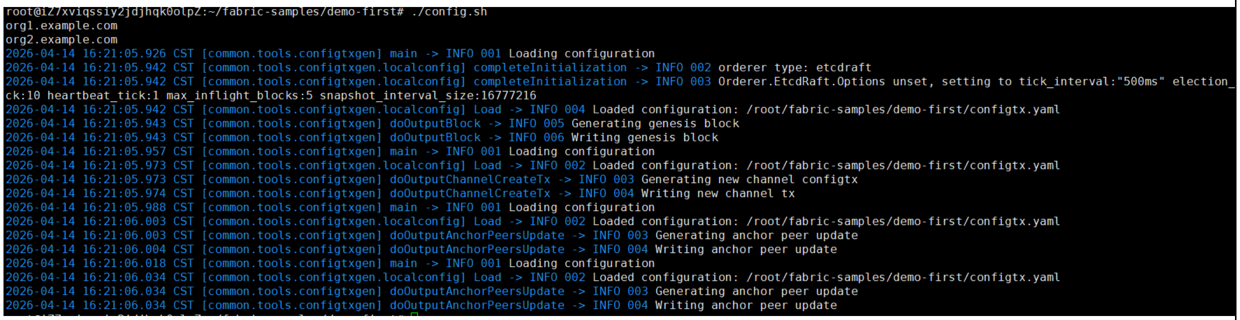
```
$ cd $HOME/fabric-samples
$ mkdir demo-first
$ cd $HOME/fabric-samples/demo-first
$ vi crypto-config.yaml
$ vi configtx.yaml

$ vi peer0-org1.yaml
$ vi peer0-org2.yaml

$ vi orderer1.yaml

$ vi cli.yaml // (of peer0.org1)

$ vi $HOME/fabric-samples/chaincode/abstore/go/abstore.go // our proposed key-value chaincode
$ vi $HOME/fabric-samples/chaincode/abstore/go/go.mod
```

Step 2 (Prepare): All machines prepare the timestamp alignment,

```
$ date +%s // check the timestamp

$ sudo apt-get install ntpdate // align the timestamp
$ sudo ntpdate cn.pool.ntp.org
$ sudo hwclock –systohc

$ date +%s // check the timestamp again
```

Step 3 (Run): CLI runs the initial block generation,

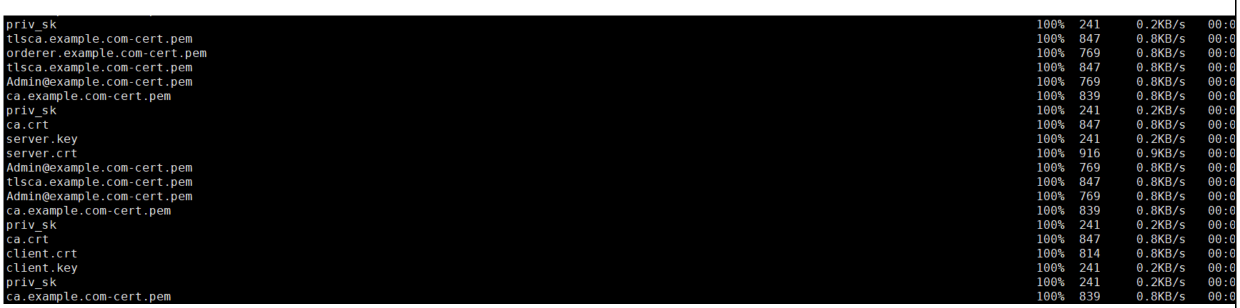
```
$ cd $HOME/fabric-samples/demo-first
$ chmod +x *.sh
$ ./config.sh
```




```
$ cd $HOME/fabric-samples/demo-first/workload/tls
$ chmod +x *.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ chmod +x *.sh
$ wget https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/node_modules.zip
$ unzip node_modules.zip

$ cd $HOME/fabric-samples/demo-first/scripts
$ chmod +x *.sh
```

Step 4 (Run): CLI scp the contents to all machines including peers and orderers,

```
$ cd $HOME/fabric-samples/demo-first
$ ./scp.sh
```



Step 5 (Run): Orderer1 sets up the Docker container,

```
$ cd $HOME/fabric-samples/demo-first
$ ./orderer1.sh
```


```
$ docker ps
```

```
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                               COMMAND             CREATED             STATUS              PORTS                    NAMES
4a207f175833        hyperledger/fabric-orderer:2.2.0    "orderer"           8 seconds ago       Up 8 seconds        0.0.0.0:7050->7050/tcp   orderer.example.com
```

Step 6 (Run): Peer1 sets up the Docker container,

$ cd $HOME/fabric-samples/demo-first
$ ./**peer1.sh**

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first# ./peer1.sh
Creating volume "net_peer0.org1.example.com" with default driver
Creating peer0.org1.example.com ... done
```

$ docker ps

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                               COMMAND             CREATED             STATUS              PORTS                    NAMES
38fb1129b2ae        hyperledger/fabric-peer:2.2.0       "peer node start"   2 seconds ago       Up 1 second         0.0.0.0:7051->7051/tcp   peer0.org1.example.com
```

Step 7 (Run): Peer2 sets up the Docker container,

$ cd $HOME/fabric-samples/demo-first
$ ./**peer2.sh**

```
root@iZ7xviqssiy2jdsfcldpc1Z:~/fabric-samples/demo-first# ./peer2.sh
Creating volume "net_peer0.org2.example.com" with default driver
Creating peer0.org2.example.com ... done
```

$ docker ps

```
root@iZ7xviqssiy2jdsfcldpc1Z:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                               COMMAND             CREATED             STATUS              PORTS                                        NAMES
e88f655db1d9        hyperledger/fabric-peer:2.2.0       "peer node start"   2 seconds ago       Up 1 second         7051/tcp, 0.0.0.0:8051->8051/tcp             peer0.org2.example.com
```

Step 8 (Run): CLI sets up the Docker container,

$ cd $HOME/fabric-samples/demo-first
$ ./**cli.sh**

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# ./cli.sh
Creating cli ... done
```

$ docker ps

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                               COMMAND             CREATED             STATUS              PORTS               NAMES
30e6fbad4c51        hyperledger/fabric-tools:2.2.0      "/bin/bash"         2 seconds ago       Up 1 second                             cli
```

10.2. Start the Network

Step 1 (Run): CLI connects to Peer0.org1.example.com:7051, creates a channel.block, and then all peers join the channel.block,

// CLI goes to `peer0.org1.example.com:7051`
$ docker exec -it cli bash

// Environment Configuration for `peer0.org1.example.com:7051`
cd scripts
./**step1.sh** // All peers join the channel

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# docker ps
CONTAINER ID        IMAGE                          COMMAND             CREATED             STATUS              PORTS               NAMES
30e6fbad4c51        hyperledger/fabric-tools:2.2.0   "/bin/bash"         2 seconds ago       Up 1 second                             cli
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# docker exec -it cli bash
bash-5.0# cd scripts
bash-5.0# ./step1.sh
2026-04-14 08:26:37.630 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:37.647 UTC [cli.common] readBlock -> INFO 002 Expect block, but got status: &{NOT_FOUND}
2026-04-14 08:26:37.649 UTC [channelCmd] InitCmdFactory -> INFO 003 Endorser and orderer connections initialized
2026-04-14 08:26:37.850 UTC [cli.common] readBlock -> INFO 004 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:37.852 UTC [channelCmd] InitCmdFactory -> INFO 005 Endorser and orderer connections initialized
2026-04-14 08:26:38.053 UTC [cli.common] readBlock -> INFO 006 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.054 UTC [channelCmd] InitCmdFactory -> INFO 007 Endorser and orderer connections initialized
2026-04-14 08:26:38.255 UTC [cli.common] readBlock -> INFO 008 Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.257 UTC [channelCmd] InitCmdFactory -> INFO 009 Endorser and orderer connections initialized
2026-04-14 08:26:38.457 UTC [cli.common] readBlock -> INFO 00a Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.459 UTC [channelCmd] InitCmdFactory -> INFO 00b Endorser and orderer connections initialized
2026-04-14 08:26:38.659 UTC [cli.common] readBlock -> INFO 00c Expect block, but got status: &{SERVICE_UNAVAILABLE}
2026-04-14 08:26:38.661 UTC [channelCmd] InitCmdFactory -> INFO 00d Endorser and orderer connections initialized
2026-04-14 08:26:38.864 UTC [cli.common] readBlock -> INFO 00e Received block: 0
2026-04-14 08:26:38.897 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:38.921 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-14 08:26:38.953 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:38.977 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join channel
2026-04-14 08:26:39.004 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:39.015 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
2026-04-14 08:26:39.040 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-04-14 08:26:39.050 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0#
```

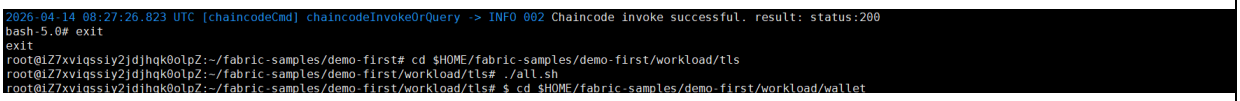
./**step2.sh** // All peers join the chaincode

```
11ac10e3978491cdaab7d56219165c0b45\022\006mycc_1" >
2026-04-14 08:27:14.093 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 002 Chaincode code package identifier: mycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cda
b7d56219165c0b45
2026-04-14 08:27:20.092 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 001 Installed remotely: response:<status:200 payload:"\nGmycc_1:9c2d699dc0bb37ea77cfbadd7567(
11ac10e3978491cdaab7d56219165c0b45\022\006mycc_1" >
2026-04-14 08:27:20.093 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 002 Chaincode code package identifier: mycc_1:9c2d699dc0bb37ea77cfbadd75670011ac10e3978491cda
b7d56219165c0b45
2026-04-14 08:27:20.160 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychannel) orderer endpoint: orderer.example.com:7050
2026-04-14 08:27:21.182 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [2e49555a3a8dd90810de8e5e90973f90a615a62711c3be05cc97ddf7b1cc0a23] committed with status (VALID) at
2026-04-14 08:27:21.218 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychannel) orderer endpoint: orderer.example.com:7050
2026-04-14 08:27:22.238 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [a720bb2d81a5e1dc2f56ab369b6df082355423c38337e372ffc9f80a515d6f26] committed with status (VALID) at
{
        "approvals": {
                "Org1MSP": true,
                "Org2MSP": true
        }
}
2026-04-14 08:27:23.402 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [f8eba34bdcfcd726a6959a47dc60c95e05cf7652a8b7f4cf80c7b44f70bd236b] committed with status (VALID) at peer0
rg1.example.com:7051
2026-04-14 08:27:23.402 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [f8eba34bdcfcd726a6959a47dc60c95e05cf7652a8b7f4cf80c7b44f70bd236b] committed with status (VALID) at peer0
rg2.example.com:8051
2026-04-14 08:27:24.693 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [862b6b71fe7bdc81df4b23af39e63d1e1017b2c3b39e34b6cdf6ac6213bb905c] committed with status (VALID) at peer0
rg1.example.com:7051
2026-04-14 08:27:24.693 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
2026-04-14 08:27:25.741 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [1722e9e0ffb97b289c38f53946f397b242b6d3ff591f2ba4f48ae6ed239311f3] committed with status (VALID) at peer0
rg1.example.com:7051
2026-04-14 08:27:25.742 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
200tps
2026-04-14 08:27:26.823 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [64c2861c9c16c9c93e549c87eab9181e87ff23827fb1207ae4c213c802b92110] committed with status (VALID) at peer0
rg2.example.com:8051
2026-04-14 08:27:26.823 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. result: status:200
bash-5.0#
```

10.3. Start Nodejs SDK

Step 1 (Run): CLI configures the Node.js SDK folder and tls, wallet, and workload files for all peers,

```
# exit
$ mkdir -p $HOME/fabric-samples/demo-first/workload
$ mkdir -p $HOME/fabric-samples/demo-first/workload/tls
$ mkdir -p $HOME/fabric-samples/demo-first/workload/wallet
$ mkdir -p $HOME/fabric-samples/demo-first/workload/result

$ cd $HOME/fabric-samples/demo-first/workload/tls
$ vi read_pem.py
$ vi prepare_tls_peer0_org1.sh
$ vi prepare_tls_peer0_org2.sh
$ ./all.sh
```




```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ vi prepare_wallet_user1_org1.py
$ vi prepare_wallet_user1_org2.py
$ ./all.sh

$ cd $HOME/fabric-samples/demo-first/workload
$ mkdir sdk-peer0-org1
$ mkdir sdk-peer0-org2
$ vi invoke.js
$ vi prepare_sdk_peer0_org1.sh
$ vi prepare_sdk_peer0_org2.sh
$ ./all.sh
```

Step 2 (Prepare): CLI prepares the Nodejs SDK dependencies on CLI. Note that you may pass this step by copying the compiled nodejs source code https://gitee.com/joe32065/fabric_2_2/releases/download/fabric-v2.2.0/node_modules.zip,

```
$ cd $HOME/fabric-samples/demo-first/workload
$ vi package.json
$ npm install
$ vi $HOME/fabric-samples/demo-first/workload/node_modules/fabric-network/lib/transaction.js
```

Step 3 (Run): Cli prepares wallets of User1@org1.example.com and User1@org2.example.com,

```
$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ rm -rf *.id
$ python prepare_wallet_user1_org1.py
```

$ python prepare_wallet_user1_org2.py
$ ./**all.sh**

Step 4 (Run): CLI invokes and queries transactions via Node.js,

$ cd $HOME/fabric-samples/demo-first/workload
$ ./**scp.sh** // CLI copies files to all machines

$ ./**ssh.sh** **> result/benchlog**
$ ls result -l

```
root@iZ7xviqssiy2jdsfcldpbvZ:~# cd $HOME/fabric-samples/demo-first/workload/sdk-peer0-org1
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first/workload/sdk-peer0-org1# nodejs invoke.js 60 100 10 1
Wallet path: /root/fabric-samples/demo-first/workload/wallet
Send_Endorsing_Proposal: 1776155562.442
Send_Endorsing_Proposal: 1776155562.447
Send_Endorsing_Proposal: 1776155562.45
Send_Endorsing_Proposal: 1776155562.452
Send_Endorsing_Proposal: 1776155562.455
Send_Endorsing_Proposal: 1776155562.458
Send_Endorsing_Proposal: 1776155562.46
Send_Endorsing_Proposal: 1776155562.462
Send_Endorsing_Proposal: 1776155562.464
Send_Endorsing_Proposal: 1776155562.468
Send_Endorsing_Proposal: 1776155562.471
Send_Endorsing_Proposal: 1776155562.474
Send_Endorsing_Proposal: 1776155562.476
Send_Endorsing_Proposal: 1776155562.478
Send_Endorsing_Proposal: 1776155562.48
Send_Endorsing_Proposal: 1776155562.482
Send_Endorsing_Proposal: 1776155562.484
Send_Endorsing_Proposal: 1776155562.487
Send_Endorsing_Proposal: 1776155562.488
Send_Endorsing_Proposal: 1776155562.49
Send_Endorsing_Proposal: 1776155562.492
Send_Endorsing_Proposal: 1776155562.494
Send_Endorsing_Proposal: 1776155562.495
Send_Endorsing_Proposal: 1776155562.497
Send_Endorsing_Proposal: 1776155562.499
Send_Endorsing_Proposal: 1776155562.502
Send_Endorsing_Proposal: 1776155562.504
Send_Endorsing_Proposal: 1776155562.507
```

// $ node query.js
// $ node invoke.js 30 500 20 1

Step 5 (Run): Analyze the log,

// CLI analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ python **readthroughput_p1.py** // rate of sending endorsed txs

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/result# python readthroughput_p1.py
Avg_Send_Rate_Phrase_1:
5.7471256178
```

$ sudo apt-get install python-numpy
$ python readdelay_p1.py // delay of endorsing phase

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first/workload/result# python readdelay_p1.py
Mean_Delay_Endorse:
0.0188799798489
Mean_Delay_Orderer:
0.00146238137317
Mean_Delay_Commit:
0.178739959002
CDF with the probability of 50% 0.00100016593933
CDF with the probability of 90% 0.00200009346008
CDF with the probability of 95% 0.00300002098083
```

// Orderer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy

//$ docker logs containername >& ordererlog
$ **docker logs $(docker ps -a | grep 'orderer'| awk '{print $1 }') >& ordererlog**
// $ scp ordererlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python readthroughput_p2.py // rate of generating blocks

```
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first/workload/result# python readthroughput_p2.py
Block_ID:
lastPersistedBlock=[24]

lastPersistedBlock=[25]

lastPersistedBlock=[26]

lastPersistedBlock=[27]

lastPersistedBlock=[28]

lastPersistedBlock=[29]

lastPersistedBlock=[30]

lastPersistedBlock=[31]

lastPersistedBlock=[32]
```

// Peer analyzes the log
$ cd $HOME/fabric-samples/demo-first/workload/result
$ sudo apt-get install python-numpy
//$ docker logs containername >& peerlog
$ **docker logs $(docker ps -a | grep 'peer node start'| awk '{print $1 }') >& peerlog**
// $ scp peerlog chwang@1xx.1xx.7xx.1xx:/home/chwang
$ python2 readthroughput_p3.py

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first/workload/result# python2 readthroughput_p3.py
block id:
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
```

// For example, we can get block size
peer channel fetch 2 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem

```
bash-5.0# peer channel fetch 2 last.block -o orderer.example.com:7050 -c mychannel --tls --cafile /opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
2026-05-07 08:15:21.880 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initialized
2026-05-07 08:15:21.882 UTC [cli.common] readBlock -> INFO 002 Received block: 2
```

10.4. Different Chaincodes

Step 1 (Run): CLI uses a different chaincode as follows,

$ cd $HOME/fabric-samples/chaincode/abstore/go
$ **vi abstore.go**

$ ls

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/chaincode/abstore/go# ls
abstore.go  go.mod  go.sum  mycc.tar.gz  vendor
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/chaincode/abstore/go#
```

Step 2 (Run): CLI compiles the chaincode in Go, and ensures that the chaincode in Go has no grammar errors, as follows,

$ **go mod vendor**
$ **go build**

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/chaincode/abstore/go# go mod vendor
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/chaincode/abstore/go# go build
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/chaincode/abstore/go# ls
abstore.go  go  go.mod  go.sum  mycc.tar.gz  vendor
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/chaincode/abstore/go#
```

Step 3 (Run): CLI modifies the step2.sh, and ensures that the new chaincode can be initialized, as follows,

$ **cd $HOME/fabric-samples/demo-first/scripts**
$ **vi step2.sh**

```
# Peer0.org1.example.com init
peer chaincode invoke -o orderer.example.com:7050 --tls --cafile /opt/gopath/src/github.com/hyperledger/fabri
/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-c
rt.pem -C $CHANNEL_NAME -n mycc --peerAddresses peer0.org1.example.com:7051 --tlsRootCertFiles /opt/gopath/sr
/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tl
/ca.crt -c '{"Args":["Init","a","100","b","50"]}' --waitForEvent

# Peer0.org1.example.com invoke
peer chaincode invoke -o orderer.example.com:7050 --tls --cafile /opt/gopath/src/github.com/hyperledger/fabri
/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-c
rt.pem -C $CHANNEL_NAME -n mycc --peerAddresses peer0.org1.example.com:7051 --tlsRootCertFiles /opt/gopath/sr
/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tl
/ca.crt -c '{"Args":["Invoke","a","b","200string"]}' --waitForEvent

peer chaincode query -C $CHANNEL_NAME -n mycc -c '{"Args":["Query","a"]}'
peer chaincode query -C $CHANNEL_NAME -n mycc -c '{"Args":["Query","b"]}'
```

Step 4 (Run): CLI modifies all peers' workload generator, as follows,

$ **cd $HOME/fabric-samples/demo-first/workload**
$ vi prepare_sdk_peer0_org1.sh

```
// ---------------------------------------------
// After load, submit a transaction
// ---------------------------------------------
function submit_a_tx_after_the_setting(){
    var t1 = Date.now()/1000;
    //console.log('Send_Endorsing_Proposal: %j', t1);
        //contract.submitTransaction('invoke', 'a', '330dota');
    contract.submitTransaction('invoke', 'a', 'b', makeid(process.argv[5]));
    //var t2 = Date.now()/1000;
    //console.log('Get_Endorsed_Proposal: %j', t2);
        //console.log("This transaction Finished on: ", new Date, "\n")
}
                                                                    103,1         48%
```

$ **vi prepare_sdk_peer0_org2.sh**

```
        result += characters.charAt(Math.floor(Math.random() * charactersLength));
    }
    return result;
}
// ---------------------------------------------
// After load, submit a transaction
// ---------------------------------------------
function submit_a_tx_after_the_setting(){
    var t1 = Date.now()/1000;
    //console.log('Send_Endorsing_Proposal: %j', t1);
        //contract.submitTransaction('invoke', 'a', '330dota');
    contract.submitTransaction('invoke', 'a', 'b',makeid(process.argv[5]));
    //var t2 = Date.now()/1000;
    //console.log('Get_Endorsed_Proposal: %j', t2);
        //console.log("This transaction Finished on: ", new Date, "\n")
}
function workload(transactions){
    try{
        for(var i = 0; i < transactions; i++){
                                                                    102,1         50%
```

Step 5 (Run): CLI runs the initial block generation,

$ cd $HOME/fabric-samples/demo-first
$ chmod +x *.sh
$ **./config.sh**

```
2026-05-12 09:09:47.439 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /ro
t/fabric-samples/demo-first/configtx.yaml
2026-05-12 09:09:47.439 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 003 Generating new cha
nel configtx
2026-05-12 09:09:47.440 CST [common.tools.configtxgen] doOutputChannelCreateTx -> INFO 004 Writing new channe
 tx
2026-05-12 09:09:47.454 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-05-12 09:09:47.471 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /ro
t/fabric-samples/demo-first/configtx.yaml
2026-05-12 09:09:47.471 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating ancho
 peer update
2026-05-12 09:09:47.472 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor p
er update
2026-05-12 09:09:47.486 CST [common.tools.configtxgen] main -> INFO 001 Loading configuration
2026-05-12 09:09:47.502 CST [common.tools.configtxgen.localconfig] Load -> INFO 002 Loaded configuration: /ro
t/fabric-samples/demo-first/configtx.yaml
2026-05-12 09:09:47.502 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 003 Generating ancho
 peer update
2026-05-12 09:09:47.503 CST [common.tools.configtxgen] doOutputAnchorPeersUpdate -> INFO 004 Writing anchor p
er update
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first#
```

Step 6 (Run): CLI scp the contents to all machines including peers and orderers,

```
$ cd $HOME/fabric-samples/demo-first
$ ./scp.sh
```

Step 7 (Run): Set up all components as follows,

```
$ cd $HOME/fabric-samples/demo-first
$ ./orderer1.sh
```

```
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first# ./orderer1.sh
Creating volume "net_orderer.example.com" with default driver
Creating orderer.example.com ... done
root@iZ7xviqssiy2jdsfcldpc2Z:~/fabric-samples/demo-first#
```

```
$ cd $HOME/fabric-samples/demo-first
$ ./peer1.sh
```

```
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first# ./peer1.sh
Creating volume "net_peer0.org1.example.com" with default driver
Creating peer0.org1.example.com ... done
root@iZ7xviqssiy2jdsfcldpbvZ:~/fabric-samples/demo-first#
```

```
$ cd $HOME/fabric-samples/demo-first
$ ./peer2.sh
```

```
root@iZ7xviqssiy2jdsfcldpc1Z:~/fabric-samples/demo-first# ./peer2.sh
Creating volume "net_peer0.org2.example.com" with default driver
Creating peer0.org2.example.com ... done
root@iZ7xviqssiy2jdsfcldpc1Z:~/fabric-samples/demo-first#
```

```
$ cd $HOME/fabric-samples/demo-first
$ ./cli.sh
```

```
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first# ./cli.sh
Creating cli ... done
root@iZ7xviqssiy2jdjhqk0olpZ:~/fabric-samples/demo-first#
```

Step 8 (Run): CLI connects to Peer0.org1.example.com:7051, creates a channel.block, and then all peers join the channel.block,

```
// CLI  goes to `peer0.org1.example.com:7051`
$ docker exec -it cli bash

// Environment Configuration for `peer0.org1.example.com:7051`
```

cd scripts
./**step1.sh** // All peers join the channel

```
ed
2026-05-12 01:16:46.280 UTC [cli.common] readBlock -> INFO 00c Expect block, but got status: &{SERVICE_UNAVAI
ABLE}
2026-05-12 01:16:46.282 UTC [channelCmd] InitCmdFactory -> INFO 00d Endorser and orderer connections initiali
ed
2026-05-12 01:16:46.485 UTC [cli.common] readBlock -> INFO 00e Received block: 0
2026-05-12 01:16:46.515 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initiali
ed
2026-05-12 01:16:46.537 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join chan
el
2026-05-12 01:16:46.566 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initiali
ed
2026-05-12 01:16:46.587 UTC [channelCmd] executeJoin -> INFO 002 Successfully submitted proposal to join chan
el
2026-05-12 01:16:46.615 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initiali
ed
2026-05-12 01:16:46.626 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
2026-05-12 01:16:46.653 UTC [channelCmd] InitCmdFactory -> INFO 001 Endorser and orderer connections initiali
ed
2026-05-12 01:16:46.665 UTC [channelCmd] update -> INFO 002 Successfully submitted channel update
bash-5.0#
```

./**step2.sh** // All peers join the chaincode

```
2026-05-12 01:17:23.109 UTC [cli.lifecycle.chaincode] submitInstallProposal -> INFO 002 Chaincode code package
 identifier: mycc_1:408cde5495ec4730e6b47f1b0732d0972d64bbc14955409866c36123019985dc
2026-05-12 01:17:23.179 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychanne
l) orderer endpoint: orderer.example.com:7050
2026-05-12 01:17:24.203 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [a0fa36507f5b99c6f2fb0811bda8bc2b538528
5f12a50c0656c8ad6e232862da] committed with status (VALID) at
2026-05-12 01:17:24.239 UTC [cli.lifecycle.chaincode] setOrdererClient -> INFO 001 Retrieved channel (mychanne
l) orderer endpoint: orderer.example.com:7050
2026-05-12 01:17:25.259 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [9d3b0183ef40b8c0b8914bd5bd8fa32f6cfd25
fcafa9bcee8e31da2227266390] committed with status (VALID) at
{
        "approvals": {
                "Org1MSP": true,
                "Org2MSP": true
        }
}
2026-05-12 01:17:26.541 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [76fdac61e10ef75f8b48f3f2ed6559ad000614
88ded2a988e714eb6e709363fe] committed with status (VALID) at peer0.org2.example.com:8051
2026-05-12 01:17:26.543 UTC [chaincodeCmd] ClientWait -> INFO 002 txid [76fdac61e10ef75f8b48f3f2ed6559ad000614
88ded2a988e714eb6e709363fe] committed with status (VALID) at peer0.org1.example.com:7051
2026-05-12 01:17:27.704 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [5b9c53e1f4f0ee94b023948fcf5ca14c99639c
bac7602e95d2f7a1b677d9ebc5] committed with status (VALID) at peer0.org1.example.com:7051
2026-05-12 01:17:27.704 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. res
ult: status:200
2026-05-12 01:17:28.753 UTC [chaincodeCmd] ClientWait -> INFO 001 txid [c9385d3760dfab3d970038c64c90f2875e3581
0c2b23ffda8f49005a07fb9273] committed with status (VALID) at peer0.org1.example.com:7051
2026-05-12 01:17:28.753 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 002 Chaincode invoke successful. res
ult: status:200
200string
200string
bash-5.0#
```

Step 9 (Run): CLI configures the Node.js SDK folder and tls, wallet, and workload files for all peers,

exit

$ cd $HOME/fabric-samples/demo-first/workload/tls
$ ./**all.sh**

$ cd $HOME/fabric-samples/demo-first/workload/wallet
$ **./all.sh**

$ cd $HOME/fabric-samples/demo-first/workload
$ ./**all.sh**

Step 10 (Run): CLI invokes and queries transactions via Node.js,

$ cd $HOME/fabric-samples/demo-first/workload
$ ./**scp.sh** // CLI copies files to all machines

$ ./**ssh.sh** **> result/benchlog**
$ ls result -l

```
Delay_Endorsing: 0.00800013542175293
Get_Endorsed_Proposal: 1778548899.528
Delay_Endorsing: 0.010000228881835938
Get_Endorsed_Proposal: 1778548899.53
Delay_Endorsing: 0.012000083923339844
Get_Ordered_Transaction: 1778548899.53
Delay_Ordering: 0
Get_Endorsed_Proposal: 1778548899.533
Delay_Endorsing: 0.015000104904174805
Get_Endorsed_Proposal: 1778548899.535
Delay_Endorsing: 0.017000198364257812
Get_Endorsed_Proposal: 1778548899.537
Delay_Endorsing: 0.01900005340576172
Get_Endorsed_Proposal: 1778548899.538
Delay_Endorsing: 0.020000219345092773
Get_Endorsed_Proposal: 1778548899.54
Delay_Endorsing: 0.02200007438659668
Get_Endorsed_Proposal: 1778548899.542
Delay_Endorsing: 0.024000167846679688
Get_Ordered_Transaction: 1778548899.543
Delay_Ordering: 0.0009999275207519531
Get_Ordered_Transaction: 1778548899.543
Delay_Ordering: 0.0009999275207519531
Get_Ordered_Transaction: 1778548899.543
Delay_Ordering: 0.0009999275207519531
Get_Ordered_Transaction: 1778548899.543
Delay_Ordering: 0.0009999275207519531
Get_Ordered_Transaction: 1778548899.543
Delay_Ordering: 0.0009999275207519531
```